دانشگاه تهران

دانشکدگان فارابی

دانشکده‌ی مهندسی

گروه مهندسی کامپیوتر

طراحی و پیاده‌سازی دستیار هوشمند بالینی (Clinical AI Copilot) مبتنی بر تلفیق پرونده بیمار و دانش پزشکی معتبر

نگارش

عرفان کربالئی

استاد راهنما

دکتر کاظم فولادی قلعه

گزارش پروژه برای دریافت درجه‌ی کارشناسی

در رشته‌ی مهندسی کامپیوتر

ماه و سال تحویل: .....

صورت‌جلسه‌ی داوری

محل درج فرم رسمی داوری پس از دریافت از دانشگاه

چکیده

این پروژه یک نمونه‌ی دانشگاهی تحت وب برای ثبت و مرور داده‌های بیمار و استفاده از ابزارهای آماده‌ی هوش مصنوعی است. هسته‌ی سمت سرور با FastAPI و PostgreSQL اطلاعات بالینی، اسناد، یادداشت‌های صوتی و گفتگو را به بیمار مشخص مرتبط می‌کند. فایل اولیه در MinIO نگهداری می‌شود و متن استخراج‌شده، تحلیل سند و متن پاک‌سازی‌شده‌ی صوت از اصل منبع جدا می‌مانند. زمینه‌ی بیمار از رکوردهای جاری ساخته می‌شود و تایم‌لاین ترتیب رخدادهای دارای منبع را عرضه می‌کند.

تمرکز گزارش بر مسئولیت‌های Backend و AI عرفان کربالئی است؛ مسئولیت اصلی رابط طبق پروپوزال با ایمان کربلائی و معماری و یکپارچه‌سازی مشترک است. اتصال مدل متن از سه adapter، گفتگوی ماندگار، کنترل شناسه‌ی شواهد، سه مسیر گفتار به متن، احراز هویت و اجازه‌ی پرونده در کد وجود دارند. مسیر فعلی تحلیل ایمنی دارو بر مدل تکیه دارد و از موتور محدود قواعد قدیمی مستقل است؛ این تفاوت با طرح اولیه به‌طور صریح بررسی می‌شود.

نسخه‌ی موجود استخراج ساختاریافته‌ی خودکار مقادیر آزمایش، بازیابی بیرونی مقاله، اتصال عملیاتی بیمارستانی یا ارزیابی بالینی تأییدشده ندارد. در اجرای منتخب تازه، ۵۹ آزمون موفق و دو آزمون به دلیل آماده نبودن MinIO دچار خطای setup شدند. این نتیجه درباره‌ی صحت قراردادهای آزموده‌شده است و به کیفیت پزشکی مدل تعمیم داده نمی‌شود. دستاورد پروژه، مبنایی قابل بررسی برای یکپارچه‌سازی داده و سرویس است؛ کارهای آینده بر امنیت عملیاتی، وضعیت پردازش پایدار، منشأ دقیق و ارزیابی مبتنی بر داده‌ی مرجع متمرکزند.

کلیدواژه‌ها: دستیار هوشمند بالینی، زمینه‌ی بیمار، FastAPI، پردازش سند، گفتار به متن، شواهد پرونده، ایمنی دارو.

فهرست مطالب

فصل اول مقدمه و معرفی پروژه ۱۱
۱-۱ مسئله و جایگاه پروژه ۱۱
۱-۲ هدف‌ها و مرز نمونه‌ی اولیه ۱۱
۱-۳ تفکیک مسئولیت و سهم عرفان ۱۲
۱-۴ وضعیت بررسی‌شده و معیار خواندن گزارش ۱۳
فصل دوم تعریف مسئله و مبانی طراحی ۱۵
۲-۱ گردش کار بالینی و نیازهای عملکردی ۱۵
۲-۲ نیازهای کیفی و معیار پذیرش ۱۵
۲-۳ قرارداد API و جدایی لایه‌ها ۱۶
۲-۴ پایگاه داده و ذخیره‌سازی شیء ۱۷
۲-۵ مدل زبانی و زمینه‌ی کنترل‌شده ۱۸
۲-۶ معنای ارزیابی فنی و بالینی ۱۸
فصل سوم معماری کلان و جریان داده ۱۹
۳-۱ اجزای اجرا و مسئولیت آن‌ها ۱۹
۳-۲ چرخه‌ی درخواست و تزریق وابستگی ۱۹
۳-۳ چرخه‌ی عمر داده و خروجی مشتق ۲۰
۳-۴ مرز Frontend و Backend ۲۱
۳-۵ مرزهای اعتماد و نقاط شکست ۲۱
فصل چهارم پیاده‌سازی Backend و مدل داده ۲۳
۴-۱ ساخت برنامه و سازمان مسیرها ۲۳
۴-۲ schema و اعتبارسنجی ۲۳
۴-۳ بیمار و مدیریت دسترسی هنگام ایجاد ۲۴
۴-۴ دارو و داده‌ی بالینی ۲۴
۴-۵ روابط و رفتار حذف ۲۵
۴-۶ Session و migration ۲۶
۴-۷ خطا و یکپارچگی عملیات ۲۶
فصل پنجم پرونده بیمار زمینه و تایم‌لاین ۲۹

۵-۱ تجمیع داده بدون تولید اطلاعات تازه ۲۹
۵-۲ ترتیب و سقف اطلاعات ۲۹
۵-۳ overview در برابر Context ۳۰
۵-۴ تایم‌لاین مشتق از رکورد ۳۰
۵-۵ کیفیت داده و تفسیر نبود اطلاعات ۳۱
۵-۶ آزمون‌های زمینه و مسیر آینده ۳۲
فصل ششم بارگذاری پردازش و تحلیل اسناد ۳۳
۶-۱ قرارداد بارگذاری و ثبت متادیتا ۳۳
۶-۲ کلید شیء و مسئولیت ذخیره‌ساز ۳۳
۶-۳ استخراج متن و وضعیت کار ۳۴
۶-۴ PDF متنی و OCR ۳۴
۶-۵ تحلیل AI سند ۳۵
۶-۶ دانلود حذف و نقاط یکپارچه‌سازی ۳۶
۶-۷ شواهد آزمون و توسعه‌ی واقع‌بینانه ۳۶
فصل هفتم مدل زبانی و گفتگوی بالینی ۳۷
۷-۱ قرارداد ارائه‌دهنده و پیکربندی ۳۷
۷-۲ درخواست OpenAI-compatible و OpenRouter ۳۷
۷-۳ Gemini و تفاوت قالب پیام ۳۸
۷-۴ تکرار تماس و خطای شبکه ۳۸
۷-۵ ساخت ورودی مدل و محدودیت زمینه ۳۹
۷-۶ گفتگوی بدون ماندگاری ۳۹
۷-۷ گفتگوی ماندگار و ترتیب commit ۴۰
۷-۸ کاتالوگ و رابط گفتگو ۴۰
۷-۹ دامنه‌ی تضمین و ارزیابی لازم ۴۱
فصل هشتم صوت و تبدیل گفتار به متن ۴۳
۸-۱ ثبت صوت و وضعیت‌های مستقل ۴۳
۸-۲ قرارداد STT و انتخاب ارائه‌دهنده ۴۳
۸-۳ Speechmatics و کار batch ۴۴
۸-۴ Google web Speech و آماده‌سازی فایل ۴۴

۸-۵ OpenRouter برای ورودی صوت ۴۵
۸-۶ زنجیره‌ی جایگزین و معنای موفقیت ۴۶
۸-۷ فایل موقت و نکته‌ی Windows ۴۶
۸-۸ پاک‌سازی اختیاری و تأیید یادداشت ۴۶
۸-۹ آزمون و سنجش کیفیت ۴۷
فصل نهم ایمنی دارو شواهد و عامل آزمایشی ۴۹
۹-۱ تغییر مسیر ایمنی نسبت به طرح اولیه ۴۹
۹-۲ قرارداد تحلیل ساختاریافته ۵۰
۹-۳ حالت ایمنی در چت ۵۰
۹-۴ موتور قواعد محدود ۵۰
۹-۵ شواهد پرونده و شناسه‌ی پایدار ۵۱
۹-۶ parse پاسخ و نمایش منبع ۵۲
۹-۷ MCP محلی و عامل ۵۲
۹-۸ امنیت و محدودیت آزمایش عامل ۵۳
فصل دهم احراز هویت امنیت و حریم خصوصی ۵۵
۱۰-۱ حساب کاربری و توکن ۵۵
۱۰-۲ اجازه‌ی پرونده و منابع فرزند ۵۵
۱۰-۳ ورود یک‌بارمصرف ۵۶
۱۰-۴ نگهداری توکن در رابط ۵۷
۱۰-۵ فایل داده و log ۵۷
۱۰-۶ کنترل‌های موجود و کارهای باقی‌مانده ۵۸
فصل یازدهم آزمون یکپارچه‌سازی و استقرار ۵۹
۱۱-۱ ساختار آزمون‌ها ۵۹
۱۱-۲ اجرای تازه برای این گزارش ۵۹
۱۱-۳ نتایج تاریخی و تفسیر آن‌ها ۶۰
۱۱-۴ طرح ارزیابی و معیارهای غایب ۶۰
۱۱-۵ زیرساخت توسعه ۶۱
۱۱-۶ تصویر Backend و Frontend ۶۱
۱۱-۷ توپولوژی production و startup ۶۲

۸-۱۱ Nginx و نشانی امضاشده ۶۲
۹-۱۱ رویه‌ی راه‌اندازی و محدودیت اثبات ۶۳
فصل دوازدهم خلاصه نتیجه‌گیری و کارهای آینده ۶۵
۱-۱۲ تطبیق با هدف‌های مصوب ۶۵
۲-۱۲ سهم مهندسی و حدود نتیجه ۶۵
۳-۱۲ اولویت‌های مهندسی ۶۶
۴-۱۲ برنامه‌ی ارزیابی و توسعه‌ی علمی ۶۶
۵-۱۲ نتیجه‌ی نهایی ۶۷
منابع و مراجع ۶۹
پیوست الف مرجع مسیرهای واقعی API ۷۱
الف‌ـ۱ سلامت فرایند ۷۱
الف‌ـ۲ حساب و ورود ۷۱
الف‌ـ۳ کاتالوگ مدل ۷۲
الف‌ـ۴ پرونده بیمار ۷۲
الف‌ـ۵ مشکلات بالینی ۷۳
الف‌ـ۶ حساسیت ۷۳
الف‌ـ۷ کاتالوگ و مصرف دارو ۷۴
الف‌ـ۸ اسناد ۷۵
الف‌ـ۹ صوت و یادداشت ۷۵
الف‌ـ۱۰ گفتگوی بدون ماندگاری ۷۶
الف‌ـ۱۱ قواعد دارویی ۷۷
الف‌ـ۱۲ تحلیل دارویی AI ۷۷
الف‌ـ۱۳ گفتگوی ماندگار ۷۷
الف‌ـ۱۴ عامل آزمایشی ۷۸
پیوست ب فرهنگ داده و روابط پایگاه ۷۹
بـ۱ کاتالوگ دارو ۷۹
بـ۲ درخواست کد ورود ۷۹
بـ۳ مشخصات پایه‌ی بیمار ۸۰
بـ۴ حساب برنامه ۸۱

بـ۵ حساسیت‌ها ۸۲
بـ۶ رویداد ثبت‌شده ۸۳
بـ۷ مشکلات بالینی ۸۴
بـ۸ گفتگوی ماندگار ۸۵
بـ۹ متادیتا و متن سند ۸۶
بـ۱۰ سابقه‌ی مصرف دارو ۸۷
بـ۱۱ انتساب دسترسی ۸۸
بـ۱۲ ضبط و رونوشت صوت ۸۹
بـ۱۳ یادداشت بالینی ۹۰
بـ۱۴ پیام و شواهد ۹۱
بـ۱۵ نتیجه‌ی منتخب آزمایش ۹۲
پیوست پ تنظیمات و حدود اجرای محیط ۹۵
پـ۱ برنامه و مرورگر ۹۵
پـ۲ پایگاه داده ۹۵
پـ۳ فایل و OCR ۹۶
پـ۴ گفتار ۹۶
پـ۵ ورود و پیام‌رسان ۹۷
پـ۶ مدل زبانی ۹۸
پیوست ت قرارداد نمونه و گزیده‌ی کد ۱۰۱
تـ۱ ایجاد بیمار ۱۰۱
تـ۲ ارسال پیام ماندگار ۱۰۱
تـ۳ شکل پاسخ شواهد مدل ۱۰۱
تـ۴ ورودی تحلیل دارویی ۱۰۲
تـ۵ ثبت متن بازبینی‌شده ۱۰۲
تـ۶ جلوگیری از اختلاط شواهد دو بیمار ۱۰۲
تـ۷ حفظ سؤال در شکست ارائه‌دهنده ۱۰۲
تـ۸ فرمان‌های مستند محیط آزمایشی ۱۰۳
پیوست ث مرجع آزمون و سناریوهای تکمیلی ۱۰۵
ثـ۱ موارد واقعی اجرای منتخب ۱۰۷

ث‌۲ سناریوهای تکمیلی پیشنهادی ۱۱۰
ث‌۳ طرح ثبت نتایج ارزیابی..... ۱۱۱

فهرست شکل‌ها

شکل ۱-۳ معماری سامانه و مرزهای ذخیره و پردازش ۱۹
شکل ۲-۳ مسیر درخواست از اعتبارسنجی و مجوز تا پاسخ ۲۰
شکل ۱-۴ رابطه‌های اصلی پایگاه داده ۲۵
شکل ۱-۵ ساخت زمینه از رکوردهای مرتبط بیمار ۲۹
شکل ۱-۶ جریان سند از فایل اولیه تا متن و تحلیل اختیاری ۳۳
شکل ۱-۷ توالی گفتگوی ماندگار و حفظ سؤال در شکست مدل ۴۰
شکل ۱-۸ ارائه‌دهنده‌های گفتار و مسیرهای جایگزین ۴۳
شکل ۱-۹ تفکیک تحلیل AI و موتور محدود قواعد از ثبت دارو ۴۹
شکل ۲-۹ اعتبارسنجی شناسه‌ی شواهد و دسترسی به منبع ۵۱
شکل ۱-۱۰ احراز هویت و کنترل انتساب بیمار ۵۶
شکل ۱-۱۱ توپولوژی تعریف‌شده در Compose استقرار ۶۲

فهرست جدول‌ها

جدول ۱ تقسیم مسئولیت مطابق پروپوزال ۱۲
جدول ۲ نیاز و سطح تحقق ۱۶
جدول ۳ زنجیره‌ی واقعی مهاجرت‌ها ۲۶
جدول ۴ منبع و تاریخ رویدادهای تایم‌لاین ۳۱
جدول ۵ قرارداد ارائه‌دهنده‌های متن ۳۷
جدول ۶ نتیجه‌ی اجرای منتخب ۶۰
جدول ۷ تطبیق خروجی با هدف‌های طرح ۶۵
جدول ۸ قراردادهای سلامت فرایند ۷۱
جدول ۹ قراردادهای حساب و ورود ۷۱
جدول ۱۰ قراردادهای کاتالوگ مدل ۷۲
جدول ۱۱ قراردادهای پرونده بیمار ۷۲
جدول ۱۲ قراردادهای مشکلات بالینی ۷۳
جدول ۱۳ قراردادهای حساسیت ۷۳
جدول ۱۴ قراردادهای کاتالوگ و مصرف دارو ۷۴
جدول ۱۵ قراردادهای اسناد ۷۵
جدول ۱۶ قراردادهای صوت و یادداشت ۷۶
جدول ۱۷ قراردادهای گفتگوی بدون ماندگاری ۷۶
جدول ۱۸ قراردادهای قواعد دارویی ۷۷
جدول ۱۹ قراردادهای تحلیل دارویی AI ۷۷
جدول ۲۰ قراردادهای گفتگوی ماندگار ۷۷
جدول ۲۱ قراردادهای عامل آزمایشی ۷۸
جدول ۲۲ فیلدهای medications ۷۹
جدول ۲۳ فیلدهای otp_requests ۸۰
جدول ۲۴ فیلدهای patients ۸۰
جدول ۲۵ مقادیر enum در patients ۸۱
جدول ۲۶ فیلدهای users ۸۲
جدول ۲۷ مقادیر enum در users ۸۲

جدول ۲۸ فیلدهای allergies ۸۳
جدول ۲۹ مقادیر enum در allergies ۸۳
جدول ۳۰ فیلدهای clinical_events ۸۳
جدول ۳۱ فیلدهای conditions ۸۴
جدول ۳۲ مقادیر enum در conditions ۸۵
جدول ۳۳ فیلدهای conversations ۸۵
جدول ۳۴ فیلدهای documents ۸۶
جدول ۳۵ مقادیر enum در documents ۸۷
جدول ۳۶ فیلدهای patient_medications ۸۷
جدول ۳۷ مقادیر enum در patient_medications ۸۸
جدول ۳۸ فیلدهای patient_user_access ۸۹
جدول ۳۹ فیلدهای voice_notes ۸۹
جدول ۴۰ مقادیر enum در voice_notes ۹۰
جدول ۴۱ فیلدهای clinical_notes ۹۱
جدول ۴۲ مقادیر enum در clinical_notes ۹۱
جدول ۴۳ فیلدهای conversation_messages ۹۲
جدول ۴۴ مقادیر enum در conversation_messages ۹۲
جدول ۴۵ فیلدهای lab_results ۹۲
جدول ۴۶ مقادیر enum در lab_results ۹۳
جدول ۴۷ تنظیمات برنامه و مرورگر ۹۵
جدول ۴۸ تنظیمات پایگاه داده ۹۵
جدول ۴۹ تنظیمات فایل و OCR ۹۶
جدول ۵۰ تنظیمات گفتار ۹۶
جدول ۵۱ تنظیمات ورود و پیام‌رسان ۹۷
جدول ۵۲ تنظیمات مدل زبانی ۹۸
جدول ۵۳ تنظیمات خارج از Settings ۹۸
جدول ۵۴ فهرست فایل‌های آزمون ۱۰۵
جدول ۵۵ جزئیات موارد اجراشده ۱۰۷
جدول ۵۶ طرح آزمون برای شکاف‌های مهندسی ۱۱۰

فصل اول مقدمه و معرفی پروژه

۱-۱ مسئله و جایگاه پروژه

پرونده‌ی بیمار مجموعه‌ای از داده‌های هم‌شکل نیست. مشخصات پایه، وضعیت مصرف دارو، حساسیت‌ها، نتیجه‌ی آزمایش، تصویر یک گزارش و یادداشت پزشک هرکدام ساختار و زمان ثبت متفاوتی دارند. مسئله‌ی مهندسی این پروژه فراهم کردن محیطی است که این منابع را به یک بیمار مشخص مرتبط کند و هنگام مرور پرونده، هم اطلاعات اصلی و هم منشأ آن‌ها در دسترس بماند. وجود یک پاسخ روان از مدل زبانی، به‌تنهایی این مسئله را حل نمی‌کند؛ پاسخ باید در کنار داده‌ای دیده شود که پزشک بتواند آن را بررسی کند.

عنوان مصوب در پروپوزال «طراحی و پیاده‌سازی دستیار هوشمند بالینی (Clinical AI Copilot) مبتنی بر تلفیق پرونده بیمار و دانش پزشکی معتبر» است. نسخه‌ی موجود یک نمونه‌ی دانشگاهی از سامانه‌ی تصمیم‌یار است: داده‌ها را ثبت و سازمان‌دهی می‌کند، متن اسناد و صوت را آماده می‌سازد و با مدل آماده پاسخ کمکی تولید می‌کند. بخش «دانش پزشکی معتبر» در عنوان، هدف طرح را بیان می‌کند؛ کد فعلی اتصال عملیاتی به کتابخانه‌ی مقالات یا دانش‌نامه‌ی جامع دارویی را نشان نمی‌دهد. این تمایز برای ارزیابی درست محصول ضروری است.

پروژه در قلمرو مهندسی نرم‌افزار و یکپارچه‌سازی ابزارهای آماده قرار دارد. آموزش مدل پایه، تشخیص خودکار تصاویر پزشکی و تصمیم مستقل درمانی جزء دامنه‌ی پیاده‌سازی نیستند. اطلاعات بیمار به‌صورت رکوردهای ساختاریافته و مستندات نگهداری می‌شود و خروجی هوش مصنوعی در بخش‌هایی جداگانه قرار می‌گیرد. بنابراین موفقیت اصلی را باید در انسجام قراردادهای داده، قابلیت ردیابی، مدیریت شکست و امکان آزمودن گردش کار جست‌وجو کرد، نه در ادعای دقت تشخیصی.

۱-۲ هدف‌ها و مرز نمونه‌ی اولیه

هدف نخست ایجاد هسته‌ای برای مدیریت بیمار و داده‌های وابسته است. هدف دوم تبدیل سند و صوت به متن قابل استفاده، بدون کنار گذاشتن فایل اصلی است. هدف سوم ساخت زمینه‌ی بیمار برای گفتگوی بالینی و نمایش شواهد پرونده همراه پاسخ است. بررسی ایمنی دارو نیز به‌عنوان یک جریان کمکی وجود دارد، اما نسخه‌ی فعلی آن با طرح اولیه تفاوت دارد: مسیر اصلی رابط از تحلیل مدل زبانی استفاده می‌کند و موتور محدود قواعد همچنان در مسیر دیگری باقی مانده است.

نمونه‌ی اولیه یا MVP، مخفف Minimum Viable Product، در اینجا به معنای مجموعه‌ای محدود از گردش کارهای قابل اجرا است. کاربر هدف پزشک است و مرز سامانه تا ارائه‌ی اطلاعات و تحلیل پیشنهادی ادامه دارد. ثبت قطعی دارو توسط کاربر انجام می‌شود؛ تحلیل ایمنی، آن را خودکار تصویب یا رد نمی‌کند. تبدیل گفتار به یادداشت بالینی نیز به درخواست صریح کاربر نیاز دارد. این تصمیم‌ها استقلال ثبت داده از دسترس‌پذیری مدل را حفظ می‌کنند، ولی به معنای اثبات ایمنی بالینی نیستند.

در مخزن رابط تبادل با سامانه‌ی واقعی بیمارستان، ورودی استاندارد پرونده‌ی الکترونیک سلامت یا پیاده‌سازی FHIR وجود ندارد. پرونده‌ها از طریق API و داده‌ی نمونه ساخته می‌شوند. همچنین معیارهایی مانند کاهش زمان ویزیت یا بهبود پیامد بیمار در این نسخه اندازه‌گیری نشده‌اند. این موارد در پروپوزال به‌عنوان انگیزه یا روش ارزیابی مطرح شده‌اند و نباید به نتیجه‌ی انجام‌شده تبدیل شوند.

۱-۳ تفکیک مسئولیت و سهم عرفان

بخش تخصیص کار پروپوزال، Backend و هسته‌ی هوشمند را به عرفان واگذار کرده است. این حوزه شامل طراحی API، پایگاه داده، مدیریت داده‌ی بیمار، پردازش سمت سرور سند و صوت، اتصال سرویس تشخیص گفتار، ساخت زمینه، اتصال مدل آماده، ارجاع و ایمنی نسخه است. تمرکز این گزارش بر همین حوزه و تصمیم‌های قابل مشاهده در کد قرار دارد. شناسه‌ی توابع و فایل‌ها در فصل‌های فنی و پیوست‌ها آورده شده‌اند تا شرح معماری قابل پیگیری باشد.

مسئولیت اصلی Frontend و تجربه‌ی کاربری در پروپوزال با ایمان کربلائی است: داشبورد، نمای بیمار، بارگذاری، نمایش متن، تایم‌لاین، چت و نمایش هشدار و منبع در این حوزه قرار می‌گیرند. معماری کلان، قرارداد API، یکپارچه‌سازی، آزمون سرتاسری، استقرار و نمایش نهایی وظایف مشترک‌اند. شرح رابط در این گزارش برای توضیح مسیر کامل داده لازم است؛ از این شرح نمی‌توان نتیجه گرفت که عرفان تمام رابط را مستقلاً پیاده کرده است.

تاریخچه‌ی Git تغییرات مهم Backend را با نویسنده‌ی ErfanKarbalaei ثبت کرده است؛ از جمله مدل داده در 09a4e6d، زمینه‌ی بیمار در 12045ee، گفتگوی ماندگار در 7fc9c53 و شواهد در .7148efd همین نام در ثبت برخی تغییرات Frontend هم دیده می‌شود. نویسنده‌ی یک commit لزوماً نشان‌دهنده‌ی مالکیت انحصاری تمام کار گروهی نیست؛ بنابراین تخصیص رسمی پروپوزال مبنای بیان مسئولیت است و تاریخچه فقط قرینه‌ی فنی برای مسیر تغییرات محسوب می‌شود.

جدول ۱ تقسیم مسئولیت مطابق پروپوزال

حوزه	مسئول اصلی	مرز مسئولیت
Backend و AI	عرفان کربالئی	API و داده، سند و صوت، Context، اتصال مدل، شواهد و ایمنی
Frontend و UX	ایمان کربلائی	رابط بالینی، نمایش پرونده و خروجی، ضبط و بارگذاری و اتصال UI
معماری و تحویل	هر دو دانشجو	قرارداد API، Integration، E2E، Deployment و Demo

۱-۴ وضعیت بررسی‌شده و معیار خواندن گزارش

مبنای فنی گزارش نسخه‌ی کاری مخزن در زمان بررسی است. آخرین commit مشاهده‌شده 56ed451 است و تغییرات ثبت‌نشده‌ی گفتار به متن، تنظیمات و وابستگی‌های متناظر نیز در تحلیل وارد شده‌اند. به همین دلیل مسیرهای Google و OpenRouter در صوت شرح داده می‌شوند، هرچند همه‌ی آن‌ها در آخرین commit ثبت نشده‌اند. فایل‌های برنامه برای تدوین این گزارش تغییر داده نشده‌اند.

برای هر ادعا سه سطح از هم جدا شده است: وجود پیاده‌سازی در کد، وجود آزمون برای آن رفتار و نتیجه‌ی اجرای قابل مشاهده. وجود تست نشان می‌دهد انتظار رفتاری تعریف شده، اما تا زمانی که آن تست اجرا نشده باشد نتیجه‌ی موفق فعلی محسوب نمی‌شود. گزارش‌های تسک‌ها نیز شواهد تاریخی‌اند و نتایج آن‌ها به اجرای تازه نسبت داده نشده است. همین قاعده درباره‌ی Docker اعمال می‌شود: داشتن پیکربندی، اثبات استقرار عمومی نیست.

فصل‌های بعد از نیازمندی و معماری به پیاده‌سازی داده، زمینه، اسناد، گفتگو، صوت و ایمنی می‌رسند. سپس امنیت، آزمون و استقرار بررسی می‌شوند و فصل پایانی تطبیق هدف‌ها و کارهای آینده را ارائه می‌کند. پیوست‌ها مرجع قراردادهای واقعی، فرهنگ داده و تنظیمات‌اند؛ خواننده برای فهم استدلال اصلی به خواندن تمام جزئیات آن‌ها نیاز ندارد، اما برای بازبینی فنی می‌تواند به آن‌ها رجوع کند.

فصل دوم تعریف مسئله و مبانی طراحی

۲-۱ گردش کار بالینی و نیازهای عملکردی

گردش کار مورد نظر با یافتن یا ایجاد بیمار آغاز می‌شود. پس از ثبت مشخصات، کاربر مشکل بالینی، حساسیت و دارو را وارد می‌کند و مستندات را به همان پرونده می‌افزاید. برنامه باید میان شناسه‌ی بیمار و شناسه‌ی یک منبع فرزند تمایز بگذارد؛ برای مثال داشتن شماره‌ی یک سند نباید مجوز دیدن آن سند باشد. این نیاز در طراحی مسیرهای تودرتو و در بررسی دسترسی مسیرهای مستقل نقش مستقیم دارد.

نمای خلاصه باید اطلاعات لازم برای مرور اولیه را با ترتیب قابل پیش‌بینی عرضه کند. داروهای فعال، حساسیت‌های شدید و نتایج اخیر از نظر کاربردی اولویت دارند. نمای زمانی هدف دیگری دارد: نشان دادن ترتیب رویدادهای ثبت‌شده و امکان رفتن به منبع هر رویداد. مخزن این دو نیاز را با دو پاسخ جدا حل کرده است. overview یک خلاصه‌ی ساختاریافته است؛ timeline رویدادهای مشتق از رکوردها را مرتب می‌کند. هیچ‌یک به‌خودی‌خود یک گزارش تشخیصی تولیدشده با مدل نیست.

برای سند و صوت، پاسخ اولیه‌ی آپلود و نتیجه‌ی پردازش در دو زمان به دست می‌آیند. اگر کاربر مجبور باشد تا پایان OCR یا رونویسی منتظر پاسخ بارگذاری بماند، خطای شبکه می‌تواند وضعیت ذخیره‌ی فایل را مبهم کند. ثبت متادیتا و سپس اجرای کار پس‌زمینه این دو مرحله را جدا می‌کند. رابط می‌تواند pending یا processing را نشان دهد و بعد وضعیت را دوباره بخواند. با این حال این روش درون فرایند برنامه است و دوام یک صف مستقل را ندارد.

گفتگو باید پرونده‌ی بیمار را در لحظه‌ی پرسش دریافت کند و سؤال قبلی کاربر را نیز در صورت نیاز حفظ کند. این دو نوع زمینه یکسان نیستند: سابقه‌ی گفتگو نشان می‌دهد چه گفته شده، در حالی که پرونده بیان می‌کند چه داده‌ای اکنون ثبت است. تفکیک آن‌ها در سرویس گفتگو جلوی تبدیل تاریخچه‌ی پاسخ مدل به منبع اصلی پرونده را می‌گیرد. با وجود این، پاسخ‌های قدیمی ممکن است به داده‌ای مربوط باشند که بعداً تغییر کرده است؛ مسئله‌ی بازتولید دقیق پاسخ همچنان باقی می‌ماند.

۲-۲ نیازهای کیفی و معیار پذیرش

قابلیت ردیابی در این پروژه یعنی بتوان خروجی مشتق را به رکورد یا فایل اولیه مرتبط کرد. فایل اصلی سند و صوت در ذخیره‌ساز باقی می‌ماند؛ متن استخراج‌شده و متن پاک‌سازی‌شده جای آن را نمی‌گیرند. شناسه‌ی منبع در پاسخ مدل نیز باید از رکورد همان بیمار به دست آید. این تعریف عملی‌تر از ادعای کلی «پاسخ معتبر» است، زیرا یک شناسه‌ی واقعی را می‌توان با کد آزمود، در حالی که اعتبار پزشکی پاسخ به ارزیابی معنایی دیگری نیاز دارد.

آزمون‌پذیری با جدا کردن سرویس‌های خالص از مسیرهای HTTP و با قابل جایگزین بودن ارائه‌دهنده‌ها حاصل می‌شود. موتور قواعد دارویی بدون شبکه کار می‌کند و سرویس STT از یک قرارداد کوچک استفاده می‌کند. در آزمون می‌توان درخواست خارجی را با پاسخ کنترل‌شده جایگزین کرد. چنین آزمونی صحت ساخت درخواست و مدیریت خطا را بررسی می‌کند، نه کیفیت تشخیص گفتار یا استدلال پزشکی سرویس واقعی را.

تاب‌آوری در اینجا حذف کامل شکست نیست؛ حفظ معنای داده هنگام شکست است. برای مثال در گفتگوی ماندگار، سؤال کاربر ثبت می‌شود، ولی هنگام شکست ارائه‌دهنده هیچ پیام دستیار ساختگی ذخیره نمی‌شود. شکست پاک‌سازی صوت نباید رونویسی موفق را از بین ببرد. شکست تحلیل ایمنی هم نباید به معنی از دست رفتن امکان ثبت دارو باشد. این قراردادهای محدود و روشن، معیار پذیرش مناسب‌تری از عبارت‌هایی مانند «سیستم همیشه کار می‌کند» هستند.

از نظر محرمانگی، حداقل نیاز آن است که احراز هویت و اجازه‌ی پرونده پیش از پردازش داده بررسی شوند. داشتن JWT معتبر، دسترسی به تمام بیماران ایجاد نمی‌کند. داده‌ی ارسالی به مدل بیرونی نیز از نظر طراحی باید حداقل مقدار لازم باشد. کد فعلی بخشی از متن را کوتاه می‌کند، اما ناشناس‌سازی کامل یا حذف مشخصات شناسایی‌کننده را اجرا نمی‌کند. بنابراین کنترل اندازه‌ی ورودی با کنترل حریم خصوصی متفاوت است.

جدول ۲ نیاز و سطح تحقق

نیاز	پیاده‌سازی موجود	وضعیت ارزیابی
مدیریت بیمار	CRUD و انتساب پزشک	کد و آزمون API موجود؛ اجرای DB در این گزارش انجام نشد
مرور پرونده	overview و Context و timeline	تجمیع قطعی؛ بدون ارزیابی زمان مرور
پردازش سند	متن PDF/OCR و ai_summary	استخراج ساختاریافته‌ی فیلد خودکار نیست
مستندسازی صوت	STT و متن خام و یادداشت بازبینی‌شده	providerها موجود؛ WER/CER اندازه‌گیری نشده
گفتگو و ارجاع	پیام ماندگار و evidence واقعی	اعتبار ID کنترل می‌شود؛ صحت معنایی جداست
ایمنی نسخه	AI-first و موتور قدیمی مستقل	فاقد اعتبارسنجی بالینی
دانش بیرونی	در طرح اختیاری/آینده	PubMed و RxNorm و FDA متصل نشده‌اند

۲-۳ قرارداد API و جدایی لایه‌ها

API، مخفف Application Programming Interface، قرارداد ارتباط رابط و Backend است. در این پروژه درخواست‌های JSON برای داده‌ی ساختاریافته و multipart برای فایل استفاده می‌شوند. Pydantic نوع

و برخی محدودیت‌های ورودی را تعریف می‌کند؛ مدل SQLAlchemy مسئول نگاشت داده به جدول است. داشتن دو نمایش جدا باعث می‌شود مثلاً password_hash یا storage_key الزاماً به پاسخ عمومی وارد نشوند و تغییرات پایگاه داده مستقیماً به تمام مصرف‌کنندگان تحمیل نشود.

طراحی مسیرها بر منابع استوار است. ایجاد سند در مسیر وابسته به بیمار انجام می‌شود و خواندن یا حذف یک سند با شناسه‌ی خود آن صورت می‌گیرد. در مسیر اول، وابستگی patient_from_id هم بیمار را پیدا می‌کند و هم مجوز را می‌سنجد. در مسیر دوم، ابتدا منبع خوانده می‌شود و سپس patient_id آن برای بررسی دسترسی استفاده می‌شود. این انتخاب باید در تمام منابع فرزند رعایت شود، زیرا کنترل مسیر فهرست به‌تنهایی جلوی دسترسی مستقیم با شناسه را نمی‌گیرد.

کد وضعیت HTTP بخشی از قرارداد است. ۴۰۱ برای نبودن یا نامعتبر بودن احراز هویت، ۴۰۳ برای نبود اجازه، ۴۰۴ برای منبع پیدا نشده و ۴۲۲ برای ورودی نامعتبر به کار می‌رود. ۵۰۳ می‌تواند نشان دهد سرویس لازم تنظیم نشده یا در دسترس نیست و ۵۰۲ شکست ارائه‌دهنده‌ی خارجی را بیان کند. همه‌ی خطاهای پیش‌فرض چارچوب الزاماً فارسی نیستند؛ مسیرها در موارد مشخص detail فارسی دارند و رابط برای برخی خطاها پیام عمومی می‌سازد.

۲-۴ پایگاه داده و ذخیره‌سازی شیء

PostgreSQL برای داده‌هایی مناسب این معماری است که به رابطه، قید یکتایی و تراکنش نیاز دارند. نام بیمار، انتساب پزشک و ارتباط آزمایش با سند در قالب جدول قابل کنترل‌اند. فایل چندمگابایتی نیاز متفاوتی دارد: باید بایت‌های اولیه‌ی آن نگهداری و مستقل دانلود شوند. MinIO این وظیفه را با رابط سازگار با S3 انجام می‌دهد. مبنای این جداسازی، تفاوت چرخه‌ی عمر داده و فایل است، نه صرفاً انتخاب دو فناوری رایج.

تراکنش پایگاه داده مجموعه‌ای از تغییرات مربوط را با هم ثبت یا بازگردانی می‌کند. ساخت مدخل دارو و نسخه‌ی بیمار می‌تواند در یک تراکنش باشد، اما عملیات MinIO در همان تراکنش قرار ندارد. اگر آپلود شیء موفق و ثبت متادیتا ناموفق شود، احتمال فایل بدون رکورد وجود دارد. اگر حذف فایل موفق و حذف رکورد ناموفق شود، جهت مسئله برعکس می‌شود. نسخه‌ی فعلی هماهنگ‌سازی پایدار میان این دو سامانه را تضمین نمی‌کند و این موضوع یک محدودیت مشخص معماری است.

قید خارجی و رفتار حذف، معنا دارند. CASCADE برای داده‌ی وابسته به بیمار مناسب است؛ SET NULL برای پیوند اختیاری آزمایش به سند اجازه می‌دهد مقدار آزمایش پس از حذف سند باقی بماند. این دو رفتار یکسان نیستند. مستندات رسمی PostgreSQL توضیح قیدهای ارجاعی را ارائه می‌کند [۴]؛ انتخاب‌های خاص این پروژه از مدل‌ها و مهاجرت‌ها برداشت می‌شود.

۲-۵ مدل زبانی و زمینه‌ی کنترل‌شده

مدل زبانی بزرگ یا LLM، مخفف Large Language Model، در این سامانه یک سرویس آماده‌ی تولید متن است. به‌جای آنکه پزشک همه‌ی پرونده را در هر سؤال تایپ کند، Backend زمینه را از داده‌های مرتبط بیمار می‌سازد. پیام سیستمی حدود نقش را بیان می‌کند و پیام کاربر شامل JSON پرونده و پرسش است. این روش ارتباط پاسخ با داده‌ی بیمار را هدف می‌گیرد، اما تضمین نمی‌کند که مدل همیشه دستورها را رعایت کند.

Grounding یا متکی کردن پاسخ به زمینه، با Retrieval یا بازیابی از منبع خارجی فرق دارد. کد فعلی پرونده را با queryهای مشخص خوانده و در پیام مدل قرار می‌دهد. در این مسیر انتخاب قطعه با شباهت برداری، رتبه‌بندی مقاله یا پایگاه برداری وجود ندارد. RAG، مخفف Retrieval-Augmented Generation، در پروپوزال اختیاری است. بنابراین نبود زیرساخت RAG نقض الزام مستقل نیست؛ ولی دسترسی به دانش پزشکی بیرونی هنوز هدف انجام‌نشده‌ای است که عنوان پروژه نباید آن را پنهان کند.

سند و متن صوت می‌توانند علاوه بر اطلاعات بیمار شامل عبارت‌های دستورگونه باشند. پیام سیستمی از مدل می‌خواهد داده را جعل نکند، اما متن ورودی منبع مورد اعتماد برای دستور دادن نیست. کد، جداکننده‌ی قطعی یا پالایش تخصصی حمله‌ی prompt injection ندارد. طراحی آینده باید با پرونده به‌عنوان داده و با دستور سیستم به‌عنوان سیاست برخورد کند و آزمون‌های خصمانه را نیز به آزمون‌های معمول بیفزاید.

۲-۶ معنای ارزیابی فنی و بالینی

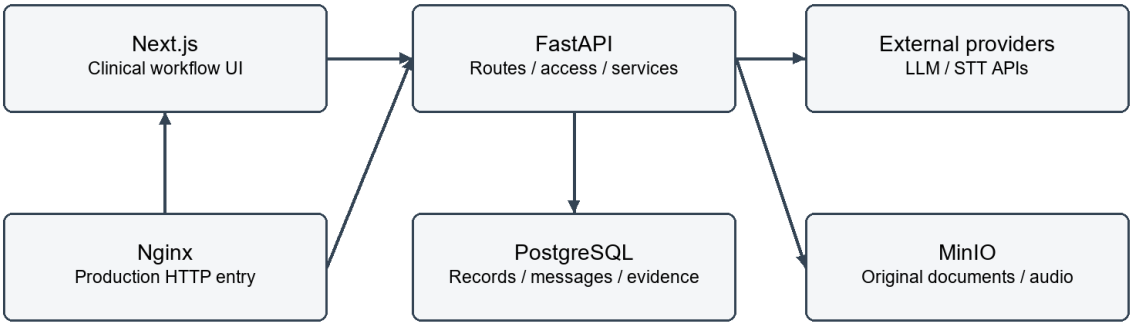
معیارهای پروپوزال از چند جنس‌اند: تطابق فیلد استخراج‌شده، خطای رونویسی، صحت ارجاع، کیفیت پاسخ، حساسیت هشدار، تأخیر و کاربری. نمی‌توان همه‌ی آن‌ها را با شمارش تست‌های API جایگزین کرد. آزمون API که برای ورودی نامعتبر ۴۲۲ می‌بیند، کیفیت پاسخ مدل را نمی‌سنجد. آزمون mocked STT که سه قطعه را به ترتیب کنار هم می‌گذارد، نرخ خطای کلمه روی صدای فارسی را اندازه نمی‌گیرد.

پذیرش مهندسی باید مرحله‌ای باشد: ابتدا صحت ذخیره و دسترسی، سپس صحت قرارداد پردازش، بعد رفتار سرویس واقعی و در پایان ارزیابی انسانی بر داده‌ی مرجع. برای ادعای کاهش خطای دارویی، به سناریوهای مرجع و تحلیل مثبت و منفی کاذب نیاز است. برای ادعای سرعت بهتر ثبت صوت، باید زمان کاربر و زمان تصحیح متن هم اندازه‌گیری شود. گزارش حاضر بخش‌های موجود و روش پیشنهادی برای بخش‌های اندازه‌گیری‌نشده را جدا نگه می‌دارد.

فصل سوم معماری کلان و جریان داده

۳-۱ اجزای اجرا و مسئولیت آن‌ها

معماری موجود شامل Frontend با Next.js، Backend با FastAPI، پایگاه PostgreSQL و ذخیره‌ساز MinIO است. سرویس‌های مدل و STT بیرون از این هسته‌اند و Backend درخواست آن‌ها را هماهنگ می‌کند. Nginx در پیکربندی استقرار نقطه‌ی ورودی مشترک است. شکل ۳-۱ ارتباط اصلی اجزا را نمایش می‌دهد؛ فلش سرویس بیرونی به معنی قابلیت تماس کد است و موفقیت زنده‌ی آن تماس را اثبات نمی‌کند.



شکل ۳-۱ معماری سامانه و مرزهای ذخیره و پردازش

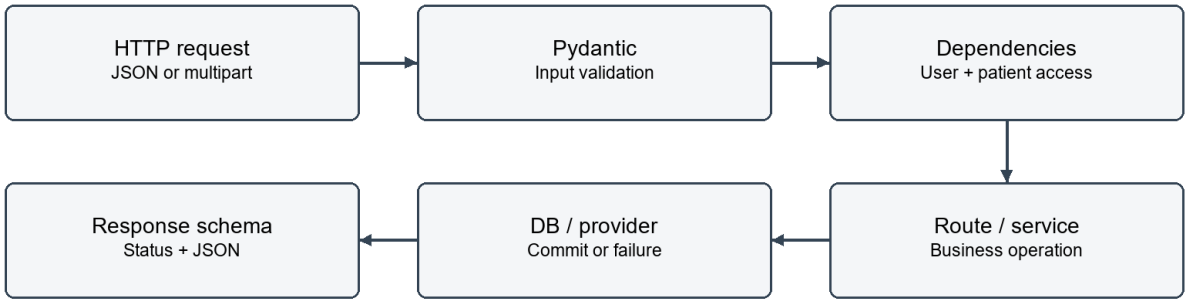
Frontend مسئول نمایش و تعامل کاربر است و ارتباطش با داده از طریق قرارداد API انجام می‌شود. Backend اعتبارسنجی، مجوز، تراکنش و انتخاب ارائه‌دهنده را در اختیار دارد. پایگاه داده منبع اصلی اطلاعات ثبت‌شده و MinIO محل فایل اولیه است. مدل خارجی نقش تولید خروجی مشتق را دارد. با این تفکیک، قطع مدل نباید اصل رکورد بیمار را از بین ببرد و رابط نباید برای دسترسی به پایگاه داده اعتبار مستقلی داشته باشد.

ساختار Backend این مرزها را در پوشه‌ها منعکس می‌کند: api برای مسیر و وابستگی، schemas برای قرارداد، models برای نگاشت رابطه‌ای، services برای عملیات تخصصی و core برای تنظیمات، امنیت و logging. این ساختار به معنی پیاده‌سازی کامل الگوی لایه‌ای با repository مستقل نیست. برخی queryها در route قرار دارند و برخی در service؛ گزارش رفتار موجود را شرح می‌دهد، نه الگویی را که صرفاً می‌توان برای آن پیشنهاد کرد.

۳-۲ چرخه‌ی درخواست و تزریق وابستگی

ورود HTTP به برنامه‌ی ساخته‌شده با create_app می‌رسد. مسیرهای api_router زیر پیشوند API_V1_PREFIX ثبت می‌شوند. وابستگی get_db یک Session برای درخواست ایجاد می‌کند و در پایان آن را می‌بندد. get_current_user توکن را می‌خواند و کاربر فعال را بازیابی می‌کند. در مسیر وابسته به بیمار،

patient_from_id علاوه بر خواندن بیمار، انتساب دسترسی را بررسی می‌کند. شکل ۲-۳ جای بررسی مجوز را پیش از ورود به منطق کاربردی نشان می‌دهد.



شکل ۲-۳ مسیر درخواست از اعتبارسنجی و مجوز تا پاسخ

وابستگی‌ها باعث می‌شوند منطق تکراری در تمام مسیرها بازنویسی نشود. همین نقطه‌ی مشترک در آزمون نیز قابل override است. با این حال override کردن کاربر به admin در fixture عمومی، آزمون را برای بررسی رفتار اصلی مسیر ساده می‌کند ولی امنیت واقعی آن مسیر را نمی‌سنجد. مخزن برای این منظور تست‌های جداگانه‌ی احراز هویت و دسترسی دارد. تحلیل نتیجه باید بداند کدام dependency واقعی و کدام جایگزین شده است.

کارهای پس‌زمینه پس از پاسخ اولیه اجرا می‌شوند و Session مستقل می‌سازند؛ Session درخواست در این زمان ممکن است بسته باشد. شناسه‌ی رکورد، به‌جای شیء Session یا فایل باز، به تابع پردازش فرستاده می‌شود. این تصمیم از استفاده‌ی نابجا از منابع درخواست جلوگیری می‌کند. مستندات FastAPI نیز اجرای کار پس از ارسال پاسخ را توضیح می‌دهد [۱]؛ ویژگی دوام و بازگشت پس از crash در کد فعلی به این مکانیزم افزوده نشده است.

۳-۳ چرخه‌ی عمر داده و خروجی مشتق

داده‌ی بیمار از چند مرحله عبور می‌کند: ثبت اولیه، پردازش، ارائه در زمینه و تولید پاسخ. اصل فایل در مرحله‌ی پردازش عوض نمی‌شود. متن استخراج‌شده برای نمایش و مدل قابل استفاده است؛ ai_summary تحلیل ثانویه است. در صوت نیز transcript، cleaned_note و ClinicalNote سه مفهوم متفاوت دارند. تغییر عنوان یا نوع سند، تغییر فایل نیست و تأیید یک یادداشت جدید نیز بازنویسی transcript محسوب نمی‌شود.

داده‌ی مشتق به زمان و ورودی وابسته است. اگر استخراج دوباره اجرا شود، خلاصه‌ی قبلی سند ممکن است بر متن قبلی تکیه داشته باشد؛ مسیر استخراج در کد فعلی به‌طور صریح خلاصه‌ی AI را باطل نمی‌کند. در مقابل پردازش مجدد صوت cleaned_note را پاک و وضعیت پاک‌سازی را pending می‌کند. این تفاوت نشان

می‌دهد سیاست بی‌اعتبار کردن داده‌ی مشتق در تمام زیرسامانه‌ها یکسان نشده است. افزودن نسخه یا fingerprint ورودی می‌تواند این رابطه را قابل کنترل‌تر کند.

در پاسخ چت، زمینه در هر بار ارسال تازه ساخته می‌شود. این کار احتمال پاسخ بر پرونده‌ی قدیمی را کم می‌کند، اما snapshot ورودی ذخیره نمی‌شود. رکورد پیام شامل متن و شواهد است؛ مدل انتخابی، نسخه‌ی prompt و hash زمینه در آن ثبت نمی‌شوند. اگر بعداً بخواهیم علت پاسخ را با همان ورودی بازتولید کنیم، تاریخچه‌ی پیام به‌تنهایی کافی نیست. این محدودیت با ارزش داشتن داده‌ی زنده تعارض ندارد؛ دو نیاز متفاوت‌اند که باید جدا طراحی شوند.

۳-۴ مرز Frontend و Backend

فایل frontend/src/lib/api.ts یک client نوع‌دار دارد. درخواست‌ها cache: no-store دارند و بدنه‌ی FormData بدون تعیین دستی Content-Type ارسال می‌شود تا مرورگر boundary را بسازد. خطای fetch به ApiError با status صفر تبدیل می‌شود. برای ۴۲۲ پیام عمومی اعتبارسنجی و برای بسیاری از وضعیت‌های دیگر detail پاسخ استفاده می‌شود. این لایه رابط را از تکرار قراردادهای شبکه در هر component بی‌نیاز می‌کند.

در مرورگر، token از localStorage خوانده و در Authorization قرار می‌گیرد. در Server Component، cookie access_token به Bearer تبدیل می‌شود. این دو مسیر به یک Backend متکی‌اند، اما خصوصیات امنیتی یکسانی ندارند. cookie توسط JavaScript نوشته شده است و HttpOnly نیست. این جزئیات در فصل امنیت بررسی می‌شود. استفاده‌ی یکسان از URL API برای fetch سمت سرور نیز علت نیاز پیکربندی استقرار به نشانی کامل است.

برای دانلود سند، helper فقط URL مسیر Backend را برمی‌گرداند. component شواهد یک پیوند عادی با target جدید دارد. پیوند عادی مرورگر Bearer موجود در localStorage را مانند fetch اضافه نمی‌کند و Backend فقط Bearer می‌پذیرد. از این رو صحت مسیر دانلود در آزمون Backend، به‌تنهایی صحت کلیک رابط را اثبات نمی‌کند. این شکاف مشاهده‌شده در یکپارچه‌سازی باید در آزمون مرورگر بازبینی شود؛ در این کار اصلاح کد برنامه انجام نشده است.

۳-۵ مرزهای اعتماد و نقاط شکست

سه مرز مهم وجود دارد: مرورگر به API، API به ذخیره‌ساز و API به ارائه‌دهنده‌ی خارجی. ورودی مرورگر نیازمند نوع و مجوز است. فایل ذخیره‌شده می‌تواند محتوای خراب یا ناسازگار با MIME اعلامی داشته باشد. پاسخ خارجی نیز ممکن است خطای HTTP، متن ناقص یا JSON نامعتبر باشد. هر مرز کنترل مخصوص خود را می‌خواهد؛ قید پایگاه داده نمی‌تواند صحت JSON مدل را تضمین کند و prompt نمی‌تواند جای ACL را بگیرد.

محدودیت حجم آپلود پس از خواندن کامل فایل اعمال می‌شود. این کنترل جلوی ثبت فایل بزرگ را می‌گیرد، اما مصرف حافظه‌ی اولیه را محدود نمی‌کند. در استقرار Nginx حد ۲۵ مگابایت دارد که برای اسناد ۲۰ مگابایتی مناسب است، ولی از سقف ۵۰ مگابایتی صوت کوچک‌تر است. در نتیجه یک صوت مجاز از نگاه Backend ممکن است پیش از رسیدن به آن رد شود. این اختلاف، مسئله‌ی قرارداد میان لایه‌هاست و باید در برنامه‌ی استقرار حل شود.

سرویس‌های بیرونی با timeout محدود شده‌اند و برخی خطاهای حمل‌ونقل در لایه‌ی AI دوباره امتحان می‌شوند. timeout هر تماس برابر بودجه‌ی کل گردش کار نیست. چند تلاش، انتظار بین تلاش‌ها و چند فراخوانی عامل می‌توانند زمان نهایی را بیشتر کنند. در تحلیل ظرفیت باید طول هر مرحله و تعداد درخواست‌ها ثبت شود؛ هیچ مقدار P95 از این تنظیمات قابل استنتاج نیست.

فصل چهارم پیاده‌سازی Backend و مدل داده

۴-۱ ساخت برنامه و سازمان مسیرها

تابع create_app در backend/app/main.py تنظیمات را دریافت می‌کند، logging را آماده می‌سازد و نمونه‌ی FastAPI را با نام، شرح و نسخه ایجاد می‌کند. سپس CORS و api_router اضافه می‌شوند. مسیر /health خارج از پیشوند نسخه نیز وجود دارد و پاسخ status=ok می‌دهد. این مسیر وجود فرایند پاسخ‌گو را نشان می‌دهد، اما اتصال به PostgreSQL، MinIO یا AI را بررسی نمی‌کند.

فایل api/v1/api.py شانزده router را کنار هم قرار می‌دهد؛ منابع بالینی، اسناد، صوت، گفتگو، catalog مدل، امنیت و عامل در آن ثبت شده‌اند. تجمیع مسیرها در یک فایل امکان بررسی سطح API را فراهم می‌کند. توضیح‌های بعضی فایل‌ها متعلق به مرحله‌ی قدیمی توسعه‌اند؛ برای مثال نام یک ارائه‌دهنده یا عبارت «فاز آینده» ممکن است در comment باقی مانده باشد. رفتار کنونی از مسیر فراخوانی واقعی تعیین می‌شود.

میان مسیر خواندن داده و مسیر تحلیل تفاوت وجود دارد. GET اطلاعات ذخیره‌شده را بازمی‌گرداند؛ POST analyze یا ai-safety-check یک فراخوانی خارجی انجام می‌دهد. اجرای مجدد استخراج با ۲۰۲ پذیرفته می‌شود، ولی تحلیل AI سند همگام است. دانستن این تفاوت برای طراحی loading و timeout رابط لازم است و نمی‌توان همه‌ی عملیات پردازش را یک کار غیرهمگام فرض کرد.

۴-۲ schema و اعتبارسنجی

PatientCreate از PatientBase استفاده می‌کند. نام و نام خانوادگی طول یک تا صد دارند؛ کد ملی اختیاری است و در صورت وجود باید ده رقم باشد؛ تاریخ تولد PastDate است. این قید کد ملی، ساختار ظاهری را بررسی می‌کند و الگوریتم رقم کنترل یا هویت واقعی را اثبات نمی‌کند. phone طول محدود دارد و address متن است. Gender از enum می‌آید. همین دقت در بیان قیدها جلوی نسبت دادن اعتبارسنجی قوی‌تر از کد را می‌گیرد.

در PATCH فیلدها اختیاری‌اند و model_dump(exclude_unset=True) فقط مقادیر ارسال‌شده را اعمال می‌کند. فرستادن null با نفرستادن یک فیلد تفاوت دارد؛ برخی فیلدهای nullable قابل پاک شدن‌اند، اما null برای ستون غیرnullable ممکن است تا مرحله‌ی پایگاه داده برسد. schemaهای Update باید از نظر این تفاوت در آزمون بررسی شوند. طول حداقل یک نیز همیشه معادل غیرسفید بودن متن نیست، مگر strip و validator مناسب اعمال شده باشد.

پاسخ‌ها با from_attributes از ORM ساخته می‌شوند. این تبدیل محدوده‌ی داده‌ی عمومی را تعریف می‌کند و تاریخ و enum را به نمایش سازگار با JSON می‌برد. وجود relationship بارگذاری‌نشده می‌تواند

هنگام ساخت پاسخ query دیگری ایجاد کند؛ برای داده‌ی دارو در Context از selectinload استفاده شده است. بنابراین schema فقط مسئله‌ی ظاهر JSON نیست و با زمان خواندن داده نیز ارتباط دارد.

۴-۳ بیمار و مدیریت دسترسی هنگام ایجاد

ایجاد بیمار با ساخت Patient و commit انجام می‌شود. IntegrityError به خطای ۴۰۹ با پیام کد ملی تکراری تبدیل می‌شود. برای کاربر doctor، پس از ثبت بیمار یک PatientUserAccess ساخته و دوباره commit می‌شود. ادمین نیاز به انتساب ندارد. این دو commit باعث می‌شود خرابی مرحله‌ی انتساب بتواند بیمار ثبت‌شده‌ای بر جا بگذارد که پزشک سازنده به آن دسترسی ندارد. اتمیک کردن هر دو مرحله، بهبود قابل مشخصی است.

فهرست بیماران q، limit و offset می‌پذیرد. limit پیش‌فرض بیست و حداکثر صد است. جست‌وجو روی نام، نام خانوادگی و کد ملی با ilike انجام می‌شود؛ تعداد کل از subquery شمارش و items با ترتیب id خوانده می‌شوند. برای doctor join با جدول انتساب انجام می‌گیرد. این روش دسترسی فهرست را محدود می‌کند، ولی offset در داده‌ی بزرگ و تغییرات هم‌زمان می‌تواند هزینه و بی‌ثباتی بیشتری از صفحه‌بندی مبتنی بر cursor داشته باشد.

حذف بیمار، db.delete و commit دارد. روابط ORM و FKها داده‌های وابسته‌ی پایگاه را پاک می‌کنند. این cascade فایل‌های MinIO را حذف نمی‌کند، زیرا ذخیره‌ساز بیرون از پایگاه داده است و این route فراخوانی حذف شیء ندارد. بنابراین حذف پرونده در لایه‌ی رابطه‌ای، پاک‌سازی کامل تمام بایت‌های مرتبط محسوب نمی‌شود. سیاست نگهداری و حذف فایل باید مستقل و قابل آزمون طراحی شود.

۴-۴ دارو و داده‌ی بالینی

Medication مدخل مشترک نام و generic_name است و PatientMedication وضعیت مصرف، dosage، frequency، تاریخ شروع و پایان و notes را به بیمار مرتبط می‌کند. این تفکیک اجازه می‌دهد یک نام دارو به چند بیمار متصل شود و تاریخچه‌ی مصرف از هویت کاتالوگ جدا بماند. در نسخه‌ی موجود نام، جای شناسه‌ی استاندارد دارویی را نگرفته است؛ RxCUI یا اتصال RxNorm در مدل نیست.

get_or_create_medication نام را strip می‌کند و با lower در پایگاه جست‌وجو می‌کند. در صورت نبود مدخل، flush انجام می‌دهد و commit را به caller واگذار می‌کند تا همراه نسخه‌ی بیمار ثبت شود. یکتایی ستون name در پایگاه با جست‌وجوی case-insensitive کاملاً یکسان نیست؛ همچنین دو درخواست هم‌زمان می‌توانند هر دو نبود رکورد را ببینند. برای محیط پرترافیک باید قید سازگار و رسیدگی به race بررسی شود.

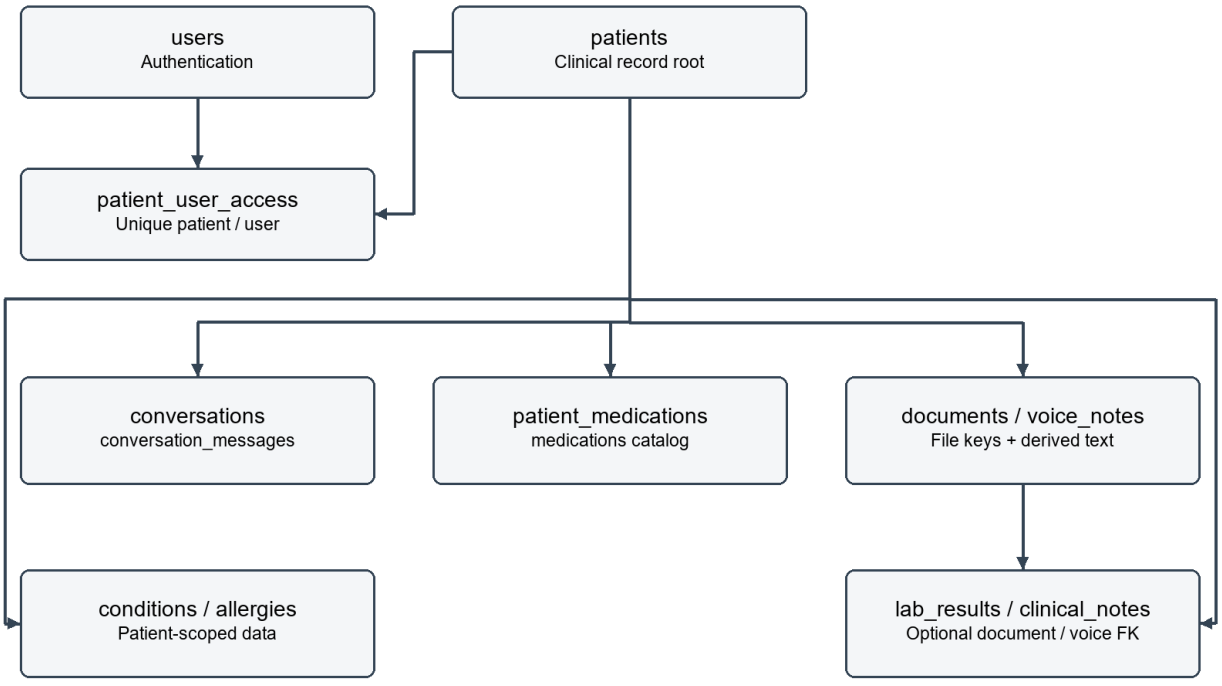
Condition یک مشکل بالینی متنی با status، onset_date، resolved_date و notes است. کاتالوگ بیماری یا کد تشخیصی استاندارد برای آن تعریف نشده است. Allergy نیز allergen، reaction و severity دارد و شدت‌های severe، moderate، mild و unknown را نگهداری می‌کند. فهرست مشکلات، فعال‌ها را

مقدم می‌گذارد و فهرست حساسیت، شدیدترها را. این ترتیب، تصمیم نمایش است و از آن نمی‌توان رتبه‌بندی قطعی خطر بیمار را نتیجه گرفت.

LabResult یک مدل عمومی برای مقدار منتخب است: test_name، value متنی، unit، reference_range، flag و result_date. نگهداری value به‌صورت متن نتایج کیفی را ممکن می‌کند، ولی محاسبه‌ی عددی یا مقایسه‌ی خودکار با بازه را بدون تبدیل اضافه فراهم نمی‌کند. وجود مدل به معنی وجود استخراج خودکار Hb یا WBC نیست. router فعلی مسیر CRUD مستقل آزمایش و یادداشت عمومی را ثبت نکرده است؛ یادداشت از صوت مسیر مشخص دارد و آزمایش در داده‌ی نمونه و Context دیده می‌شود.

۴-۵ روابط و رفتار حذف

شکل ۴-۱ رابطه‌های اصلی را نمایش می‌دهد. بیمار ریشه‌ی بیشتر داده‌ی بالینی است. ارتباط نسخه با Medication و ارتباط پیام با Conversation در زنجیره‌های جدا قرار دارند. PatientUserAccess پیوند user و patient را با قید یکتایی زوج برقرار می‌کند. این نمودار از FKهای مدل‌ها ساخته شده است و سرویس فرضی یا جدول برداری ندارد.



شکل ۴-۱ رابطه‌های اصلی پایگاه داده

ClinicalNote می‌تواند به VoiceNote با SET NULL مرتبط باشد. حذف ضبط، یادداشت تأییدشده را از بین نمی‌برد، ولی لینک آن را خالی می‌کند. LabResult نیز برای document_id رفتار SET NULL دارد. در مقابل پیام‌های یک گفتگو با حذف Conversation پاک می‌شوند. برای کاتالوگ دارو حذف cascade از نسخه وجود ندارد؛ مدخل ارجاع‌شده را نمی‌توان مانند فرزند بیمار حذف کرد. این تصمیم‌ها باید با سیاست نگهداری داده هماهنگ باشند.

ClinicalEvent به منبع با source_type و source_id اشاره می‌کند، اما این اشاره FK واقعی به چند جدول مختلف نیست. باز بودن نوع رویداد انعطاف دارد؛ هزینه‌ی آن نبود کنترل ارجاعی پایگاه برای source_id است. تایم‌لاین اصلی برای جلوگیری از اتکا به این جدول ناقص، از رکوردهای واقعی ساخته می‌شود. جدول رویداد همچنان برای برخی عملیات پر می‌شود و نباید آن را دفتر ممیزی کامل یا منبع انحصاری تایم‌لاین دانست.

۴-۶ Session و migration

Engine با pool_pre_ping و echo=False ساخته می‌شود. pre_ping پیش از استفاده، اتصال موجود را می‌آزماید و احتمال استفاده از اتصال قطع‌شده را کاهش می‌دهد؛ این ویژگی تضمین بازیابی هر تراکنش نیمه‌تمام نیست. SessionLocal با autoflush=False، autocommit=False و expire_on_commit=False تنظیم شده است. get_db با finally Session را می‌بندد. مدیریت commit و rollback در اختیار مسیر یا سرویس قرار دارد [۲].

هفت revision در Alembic زنجیره‌ی تغییر schema را ثبت می‌کنند: هسته‌ی بالینی، تحلیل سند، پاک‌سازی صوت، گفتگو، شواهد پیام، کاربر و دسترسی، و .OTP فایل‌های مهاجرت در فصل استقرار به‌عنوان گام جدا اجرا می‌شوند؛ create_all در fixture تست جای آزمودن واقعی migration را نمی‌گیرد. برای انتشار باید نسخه‌ی schema موجود و head مهاجرت یکسان باشند، و سازگاری downgrade فقط روی داده‌ی آزمایشی بررسی شود.

جدول ۳ زنجیره‌ی واقعی مهاجرت‌ها

Revision	پیشین	تغییر
d3d54628e9bd	آغاز	create_core_clinical_entities
b309d1be6900	d3d54628e9bd	add_document_ai_analysis_columns
fe6bf890d790	b309d1be6900	add_voice_note_cleanup_columns
03378aed5376	fe6bf890d790	add_persistent_conversations
ab22d66552c2	03378aed5376	add_message_evidence
9a1c78a56f1a	ab22d66552c2	add_users_and_patient_access
49c86fd969f2	9a1c78a56f1a	add_otp_login

۴-۷ خطا و یکپارچگی عملیات

خطر مهم در عملیات چندمرحله‌ای، موفقیت ناقص است. upload سند پیش از ساخت رکورد، شیء را ذخیره می‌کند؛ حذف سند نیز فایل را پیش از commit پایگاه حذف می‌کند. هر دو مسیر نیازمند رسیدگی جبرانی‌اند. در پردازش پس‌زمینه، SQLAlchemyError به rollback و log تبدیل می‌شود تا تغییر هم‌زمان

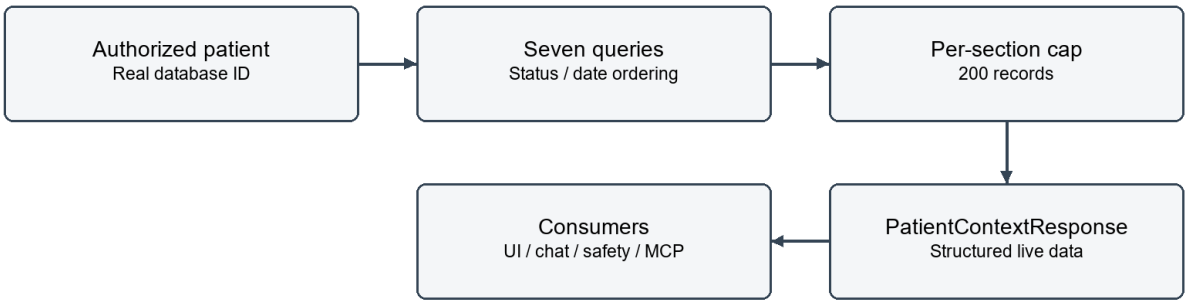
یا حذف رکورد باعث crash بی‌مهار نشود. با این حال صرف گرفتن exception به معنی اصلاح تمام وضعیت‌های نیمه‌کاره نیست.

برای توسعه‌ی بعدی می‌توان وضعیت کار، شناسه‌ی درخواست و نتیجه‌ی هر مرحله را در یک جدول عملیاتی نگهداری کرد. پردازش تکراری باید idempotent باشد، یعنی تکرار آن به داده‌ی ناسازگار یا نسخه‌ی تکراری منجر نشود. در نسخه‌ی فعلی idempotency key عمومی برای ارسال پیام، آپلود یا ایجاد یادداشت تعریف نشده است. بنابراین retry مرورگر پس از قطع پاسخ باید با احتیاط طراحی و آزمایش شود.

فصل پنجم پرونده بیمار زمینه و تایم‌لاین

۵-۱ تجمیع داده بدون تولید اطلاعات تازه

build_patient_context در services/patient_context.py هفت گروه داده‌ی بالینی را همراه مشخصات بیمار می‌خواند: مشکلات، داروها، حساسیت‌ها، آزمایش‌ها، اسناد، صوت و یادداشت‌ها. خروجی PatientContextResponse شامل generated_at است. این زمان، لحظه‌ی ساخت پاسخ را نشان می‌دهد؛ تاریخ ویزیت یا زمان رخداد همه‌ی اطلاعات نیست. در این سرویس تماس AI یا جست‌وجوی برداری وجود ندارد. شکل ۵-۱ مسیر تجمیع را نشان می‌دهد.



شکل ۵-۱ ساخت زمینه از رکوردهای مرتبط بیمار

هر query با patient_id محدود می‌شود. چنین فیلتر مشترکی لازمه‌ی جلوگیری از آمیخته شدن داده‌ی دو پرونده است، اما جای بررسی مجوز کاربر را نمی‌گیرد. بررسی مجوز پیش از ورود در route انجام می‌شود. مصرف‌کننده‌ی داخلی مانند MCP نیز از همین builder استفاده می‌کند، ولی اگر مستقیم اجرا شود احراز هویت وب را همراه ندارد. بنابراین patient-scoped بودن query و user-authorized بودن فراخوانی باید دو شرط جدا تلقی شوند.

ساخت زنده‌ی Context از انحراف یک نسخه‌ی کپی‌شده نسبت به پایگاه اصلی جلوگیری می‌کند. هزینه‌ی آن چند query در هر بار پرسش است. در نسخه‌ی فعلی لایه‌ی cache برای Context و invalidation متناظر دیده نمی‌شود. در مقیاس نمونه‌ی دانشگاهی این سادگی قابل فهم است؛ در داده‌ی بزرگ باید زمان queryها، حجم پاسخ و تعداد تماس‌های هم‌زمان اندازه‌گیری شود. انتخاب cache بدون سیاست تغییر داده می‌تواند همان مسئله‌ی کهنگی را بازگرداند.

۵-۲ ترتیب و سقف اطلاعات

در هر گروه سقف RECENT_LIMIT_ برابر دویست رکورد است. این سقف دفاعی اندازه‌ی فهرست را محدود می‌کند، اما انتخاب معنایی مرتبط‌ترین اطلاعات با سؤال نیست. برای مشکلات، active مقدم است و

سپس تاریخ شروع نزولی و id نزولی می‌آید. برای دارو، active، stopped و سایر وضعیت‌ها به ترتیب قرار می‌گیرند. حساسیت severe بر moderate و mild مقدم است. این ترتیب موجب می‌شود داده‌ی مهم‌تر از نظر وضعیت پیش از پر شدن سقف وارد شود.

آزمایش‌ها با result_date نزولی و تاریخ null در انتها خوانده می‌شوند. سند، صوت و یادداشت با created_at و id نزولی مرتب می‌شوند. selectinload برای مدخل دارو از query جدا برای هر نسخه جلوگیری می‌کند. استفاده از ترتیب دوم روی id نیز زمانی که تاریخ‌ها برابرند خروجی را قابل پیش‌بینی می‌سازد. این جزئیات برای تست و تجربه‌ی کاربر اهمیت دارند، زیرا تغییر بی‌دلیل ترتیب می‌تواند به‌صورت تغییر پرونده برداشت شود.

عنوان «زمینه‌ی کامل» در توضیح API باید با سقف دویست خوانده شود. اگر بیمار بیش از این مقدار رکورد در یک گروه داشته باشد، همه‌ی تاریخچه وارد پاسخ نمی‌شود. کد فیلد مشخصی برای اعلام تعداد حذف‌شده از Context ندارد. مدل نمی‌تواند از نبود یک رکورد در ورودی نتیجه بگیرد آن اتفاق هرگز رخ نداده است. بهتر است در توسعه‌ی بعدی علاوه بر محدوده‌ی زمانی، اطلاعات truncation نیز در قرارداد زمینه ثبت شود.

۵-۳ overview در برابر Context

get_patient_overview پاسخ کوچک‌تری برای نمای مرور بیمار دارد. مشکلات با active مقدم و حساسیت‌ها با severe مقدم‌اند؛ داروها فقط active هستند. نتایج آزمایش و یادداشت بالینی به پنج مورد اخیر محدود می‌شوند. این پاسخ با Context که سابقه‌ی دارو را نیز حمل می‌کند یکسان نیست. استفاده از overview برای پرسشی درباره‌ی داروی متوقف‌شده می‌تواند اطلاعات لازم را حذف کند؛ به همین دلیل مسیر AI از builder مستقل Context استفاده می‌کند.

واژه‌ی Summary در رابط ممکن است به این نمای ساختاریافته اشاره کند. در نسخه‌ی فعلی این نما یک خلاصه‌ی روایی مستقل با مدل تولید نمی‌کند. ai_summary فقط روی سند دیده می‌شود و cleaned_note روی صوت است. این تفکیک جلوی آن را می‌گیرد که خواننده تصور کند هر بار باز شدن نمای بیمار، یک خلاصه‌ی بالینی هوشمند محاسبه می‌شود. نبود این قابلیت نیز به معنای نبود نمای خلاصه‌ی کاربردی نیست؛ نوع خلاصه متفاوت است.

۵-۴ تایم‌لاین مشتق از رکورد

build_timeline برای مشکل، دارو، سند، صوت، یادداشت و آزمایش رویداد می‌سازد. برای Condition یک رویداد شروع و در صورت وجود resolved_date یک رویداد رفع می‌آید. برای PatientMedication شروع وجود دارد و پایان زمانی اضافه می‌شود که end_date موجود و status غیرactive باشد. سند و صوت با تاریخ ایجاد و آزمایش با result_date معرفی می‌شوند. حساسیت در حلقه‌های این تابع رویداد مستقل ندارد.

helper داخلی add، event_type، date، title، description، source_type و source_id را به TimelineItem تبدیل می‌کند. تاریخ اصلی اگر موجود نباشد با created_at جایگزین می‌شود. date بدون ساعت در نیمه‌شب UTC قرار می‌گیرد و datetime بدون timezone، UTC فرض می‌شود. این قاعده برای مرتب‌سازی سازگار است، اما نمی‌تواند زمان واقعی رویداد مجهول را کشف کند. در نمایش باید معنای «تاریخ ثبت» از «تاریخ رخداد» قابل تشخیص باشد.

مرتب‌سازی نهایی روی سه‌تایی date، source_type و source_id با reverse=True انجام می‌شود. تساوی زمان‌ها به ترتیب قطعی منابع می‌انجامد، نه به ترتیب بالینی اثبات‌شده. این روش در برابر حذف رکورد نیز روشن است: رویداد مشتق آن در پاسخ بعدی وجود نخواهد داشت. رویداد دستی ClinicalEvent که منبعش در گروه‌های تابع نیست، صرفاً به دلیل بودن در آن جدول وارد این تایم‌لاین نمی‌شود.

جدول ۴ منبع و تاریخ رویدادهای تایم‌لاین

منبع	رویداد	تاریخ اصلی و fallback
Condition	condition_onset / condition_resolved	onset_date یا resolved_date؛ نبود تاریخ شروع ← created_at
PatientMedication	medication_started / medication_stopped	start_date؛ پایان فقط با end_date و وضعیت غیرactive
Document	document_uploaded	created_at
VoiceNote	voice_note_recorded	created_at
ClinicalNote	note_created	created_at
LabResult	lab_result	result_date؛ در نبود ← created_at

۵-۵ کیفیت داده و تفسیر نبود اطلاعات

Context وضعیت فعال و غیرفعال دارو، شدت حساسیت و تاریخ‌ها را حمل می‌کند، ولی اعتبار پزشکی آن‌ها را خودکار بازبینی نمی‌کند. اگر یک وضعیت اشتباه ثبت شده باشد، مدل همان داده را می‌بیند. اگر value آزمایش متن غیرقابل تبدیل باشد، builder آن را اصلاح نمی‌کند. این رفتار برای حفظ منبع مناسب است، اما ضرورت بازبینی داده‌ی ورودی را از بین نمی‌برد. تولید پاسخ دقیق‌تر به داده‌ی معتبر و کافی نیاز دارد.

نبود آلرژی ثبت‌شده با تأیید «فاقد آلرژی» متفاوت است. فهرست خالی فقط می‌گوید رکوردی در محدوده‌ی خوانده‌شده وجود ندارد. چنین تمایزی در داده‌ی نمونه و آزمون Context خالی اهمیت دارد. مدل باید نبود اطلاعات را بیان کند؛ prompt این رفتار را درخواست می‌کند، ولی schema به‌تنهایی معنای منفی بالینی را تعیین نمی‌کند. افزودن نشانگرهای صریح «پرسیده نشده» و «بررسی و رد شده» می‌تواند ابهام را کاهش دهد.

متن خام اسناد و صوت هم در Context قرار می‌گیرد. ایجاد ClinicalNote از transcript به تأیید پزشک نیاز دارد، اما transcript خام از قبل می‌تواند به مدل برسد. بنابراین عبارت «متن فقط پس از تأیید وارد پرونده

می‌شود» اگر درباره‌ی تمام مصرف‌های داده به کار رود دقیق نیست. تأیید، مرحله‌ی تبدیل به یادداشت بالینی است؛ دسترسی مدل به خروجی خام مسیر جدا دارد و باید برچسب وضعیت بازبینی را در نظر بگیرد.

۵-۶ آزمون‌های زمینه و مسیر آینده

test_patient_context_api.py و test_patient_overview_api.py انتظار حضور گروه‌ها، ترتیب و داده‌ی اخیر را تعریف می‌کنند. test_patient_context_api.py همچنین تایم‌لاین و fallback تاریخ را بررسی می‌کند. این آزمون‌ها به PostgreSQL fixture متکی‌اند و در اجرای منتخب این گزارش اجرا نشده‌اند. شواهد آن‌ها وجود قرارداد آزموده‌شدنی است؛ نتیجه‌ی فعلی آن‌ها باید با اجرای مستقل روی پایگاه آزمایشی گزارش شود.

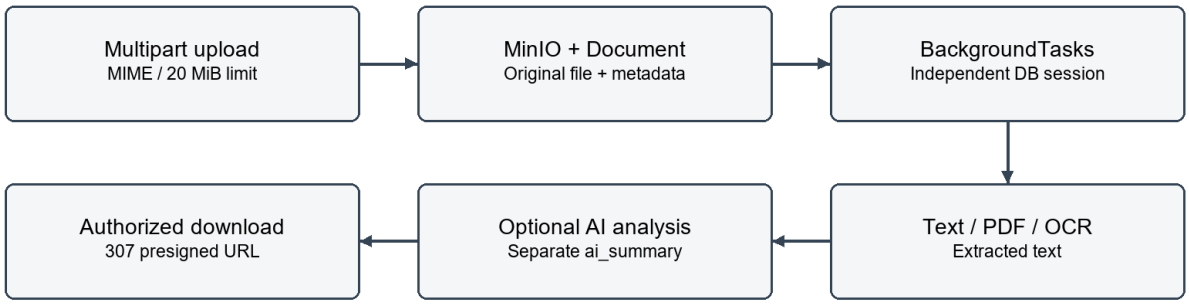
بهبود مناسب Context، انتخاب کنترل‌شده‌ی داده بر اساس سؤال و حفظ اطلاعات حذف‌شده است. می‌توان پرسش درباره‌ی دارو را به سابقه‌ی مصرف و حساسیت محدود کرد، اما این انتخاب باید قابل توضیح باشد و اطلاعات حساس لازم را کنار نگذارد. روش انتخاب باید در کنار حالت «پرونده‌ی کامل‌تر» ارزیابی شود. صرف کاهش تعداد token کافی نیست؛ باید نرخ حذف شواهد لازم و اثر آن بر پاسخ اندازه‌گیری شود.

فصل ششم بارگذاری پردازش و تحلیل اسناد

۶-۱ قرارداد بارگذاری و ثبت متادیتا

مسیر POST /api/v1/patients/{patient_id}/documents فایل را با فیلد file و title و doc_type اختیاری دریافت می‌کند. بیمار از وابستگی مجوزدار می‌آید. نام فایل حداکثر سیصد نویسه و حجم حداکثر بیست MiB است. نوع‌های مجاز PDF، PNG، JPEG، WebP و متن ساده‌اند. فایل خالی و نوع خارج از فهرست به ۴۲۲ تبدیل می‌شوند. MIME از مشخصات UploadFile خوانده می‌شود؛ بررسی امضای واقعی بایت فایل به‌عنوان کنترل جدا پیاده نشده است.

پس از read کامل، storage.upload_file بایت‌ها را به MinIO می‌فرستد. رکورد Document عنوان، نوع، کلید، نام اولیه، MIME و file_size_bytes را نگهداری می‌کند. flush شناسه را فراهم می‌سازد و یک ClinicalEvent برای بارگذاری ایجاد می‌شود. سپس commit و refresh انجام می‌گیرد و process_document با شناسه در BackgroundTasks قرار می‌گیرد. شکل ۶-۱ نقاط ذخیره و تولید داده‌ی مشتق را نشان می‌دهد.



شکل ۶-۱ جریان سند از فایل اولیه تا متن و تحلیل اختیاری

۶-۲ کلید شیء و مسئولیت ذخیره‌ساز

build_storage_key اجزای مسیر ورودی را حذف می‌کند، نویسه‌های نامطمئن را با زیرخط جایگزین می‌کند، نقطه‌های پی‌درپی را جمع می‌کند و نام را تا صد و پنجاه نویسه محدود می‌سازد. مسیر نهایی شامل patients، شناسه‌ی بیمار، پوشه و UUID است. استفاده از شناسه‌ی یکتا احتمال برخورد نام را کاهش می‌دهد و مانع می‌شود نام تکراری فایل قبلی را بازنویسی کند. وجود شناسه‌ی بیمار در مسیر، مجوز نیست و حفاظت باید در مسیر دسترسی اعمال شود.

get_client نمونه‌ی MinIO را به‌صورت lazy می‌سازد. در upload، bucket در صورت نبود ایجاد می‌شود. get_object_bytes پاسخ را در finally می‌بندد و اتصال را آزاد می‌کند. خطاهای S3Error به

StorageError تبدیل می‌شوند تا route با جزئیات SDK درگیر نباشد. با این حال همه‌ی خطاهای transport الزاماً S3Error نیستند؛ نتیجه‌ی اجرای تست ذخیره‌سازی نشان داد قطع اتصال می‌تواند در fixture به exception دیگری برسد.

presigned_download_url به‌طور پیش‌فرض یک ساعت اعتبار دارد. این نشانی یک مجوز موقت مبتنی بر امضا است و تا پایان اعتبار می‌تواند توسط دارنده‌ی آن استفاده شود. خروجی مدل نباید این URL را تولید کند و URL در evidence پایدار ذخیره نمی‌شود. این انتخاب از کهنه شدن لینک‌های ذخیره‌شده جلوگیری می‌کند، ولی سیاست اعتبار و امکان لغو پس از تغییر دسترسی همچنان نیازمند بررسی عملیاتی است.

۶-۳ استخراج متن و وضعیت کار

process_document Session مستقل می‌سازد و Document را با id می‌خواند. نبود رکورد با log و بازگشت مدیریت می‌شود. ابتدا extraction_status به processing می‌رود و commit می‌شود. فایل از ذخیره‌ساز خوانده و extract_text اجرا می‌شود. متن strip‌شده یا None و وضعیت completed ثبت می‌شوند. completed در اینجا پایان اجرای ابزار است و لزوماً به معنی متن غیرخالی یا استخراج صحیح تمام محتوای سند نیست.

در شکست عمومی، متن پاک و وضعیت failed ثبت می‌شود؛ در SQLAlchemyError rollback انجام می‌گیرد. خطای روش استخراج در log دیده می‌شود، ولی Document فیلد error_message مانند VoiceNote ندارد. روش ExtractionResult نیز در log ثبت می‌شود و ستون مستقل method در مدل Document دیده نمی‌شود. بنابراین اطلاعات منشأ موجود شامل فایل و ارتباط رکورد است، نه یک گزارش پایدار کامل از نسخه‌ی OCR و روش هر صفحه.

مسیر extract برای اجرای مجدد ۲۰۲ می‌دهد و اگر وضعیت processing باشد ۴۰۹ برمی‌گرداند. کنترل فقط روی وضعیت جاری است و enqueue هم‌زمان پیش از تغییر آن به processing هنوز می‌تواند به بررسی رقابت نیاز داشته باشد. رابط باید وضعیت را دوباره بخواند؛ این پاسخ به معنی پایان استخراج نیست. برای صف پایدار، لازم است شناسه‌ی job و انتقال وضعیت در یک قرارداد اتمیک تعریف شود.

۶-۴ PDF متنی و OCR

برای text/plain، UTF-8 با errors=replace استفاده می‌شود. بایت نامعتبر باعث توقف decoding نمی‌شود، اما ممکن است نویسه‌ی جایگزین در متن ایجاد کند. PDF با pypdf خوانده و فقط پنجاه صفحه‌ی نخست وارد پردازش می‌شود. متن صفحات به هم متصل می‌گردد و اگر طول مجموع آن حداقل پنجاه نویسه باشد روش pdf_text انتخاب می‌شود. آستانه بر مجموع سند است؛ برای هر صفحه‌ی کم‌متن به‌طور مستقل OCR اجرا نمی‌شود.

در PDF ترکیبی، ممکن است صفحه‌ی اول متن کافی داشته باشد و صفحات اسکن‌شده‌ی بعدی بدون متن بمانند. این محدودیت از شرط مجموع متن نتیجه می‌شود. اگر مجموع کوتاه باشد، تصاویر جاسازی‌شده‌ی صفحات برای OCR استفاده می‌شوند. این مسیر با رندر تمام صفحه به تصویر فرق دارد؛ چیدمان چند تصویر، متن برداری یا سند غیرمعمول می‌تواند پوشش متفاوتی داشته باشد. مستندات pypdf نیز تمایز استخراج لایه‌ی متن و OCR را توضیح می‌دهد [۸].

Tesseract در اولین نیاز بررسی و نتیجه‌ی آماده بودن cache می‌شود. اگر هر دو fas و eng نصب باشند زبان fas+eng انتخاب می‌شود و در غیر این صورت یکی از زبان‌های موجود استفاده می‌شود. تصویر با Pillow خوانده می‌شود و pytesseract متن می‌دهد. نبود binary با ExtractionUnavailable بیان می‌شود. نصب بسته‌ی Python به‌تنهایی OCR را فعال نمی‌کند؛ executable و بسته‌ی زبان هم لازم‌اند.

کیفیت OCR به خوانایی فایل و ترتیب خروجی وابسته است. کد فعلی validator واحدهای آزمایشگاهی، confidence هر واژه یا تصحیح عدد با فایل اولیه ندارد. در تست تصویر فارسی تنها تولید حداقل متن معنادار از نظر طول بررسی می‌شود؛ این آزمون دقت نویسه یا تطابق محتوای پزشکی را نمی‌سنجد. حفظ فایل اولیه امکان بازبینی را می‌دهد، اما جای سنجش خطای استخراج را نمی‌گیرد.

۶-۵ تحلیل AI سند

تحلیل در /api/v1/documents/{document_id}/analyze POST با خواندن سند و بررسی دسترسی آغاز می‌شود. شرط اجرای آن completed بودن استخراج و غیرخالی بودن extracted_text است؛ در غیر این صورت ۴۰۹ داده می‌شود. analyze_document با ارائه‌دهنده‌ی مشترک AI یک پیام از عنوان، نوع و متن سند می‌سازد. درخواست، خلاصه‌ای فارسی با یافته‌های موجود و محدودیت‌ها می‌خواهد و از جعل یافته، عدد، تشخیص یا داروی تازه منع می‌کند.

خروجی متن در ai_summary ذخیره می‌شود و analysis_status به completed یا failed می‌رود. اصل فایل و extracted_text عوض نمی‌شوند. اگر AI تنظیم نشده باشد ۵۰۳ برمی‌گردد؛ اگر AIError رخ دهد mark_analysis_failed خلاصه را خالی و وضعیت را failed می‌کند و route خطای ۵۰۲ می‌دهد. این خطا درباره‌ی تحلیل است و نباید به‌عنوان خرابی فایل اصلی نمایش داده شود.

این تحلیل استخراج ساختاریافته‌ی آزمایش نیست. نتیجه‌ی آن متن آزاد است و کد، فیلدهای LabResult را از خروجی به‌طور خودکار ایجاد نمی‌کند. برای نمونه، مدل می‌تواند درباره‌ی مقدار موجود در متن خلاصه بنویسد، اما این عمل معادل ثبت مقدار معتبر در ستون آزمایش نیست. پروپوزال استخراج Fieldهای منتخب را هدف گرفته بود؛ در پیاده‌سازی فعلی این قسمت باید محدود به استخراج متن و تحلیل سطح سند توصیف شود.

۶-۶ دانلود حذف و نقاط یکپارچه‌سازی

GET download ابتدا get_document_or_404_ را اجرا می‌کند؛ این helper مجوز بیمار را بررسی می‌کند. سپس URL امضاشده ساخته و پاسخ ۳۰۷ داده می‌شود. آزمون دسترسی نیز ترتیب مجوز پیش از تولید URL را بررسی می‌کند. مسیر PATCH عنوان و نوع را تغییر می‌دهد و DELETE شیء و رکورد و ClinicalEvent متناظر را حذف می‌کند. موفقیت حذف در دو سامانه تراکنش واحد ندارد.

در رابط بارگذاری، فهرست و وضعیت متن و تحلیل نمایش داده می‌شوند. بررسی End-to-End واقعی باید علاوه بر HTTP و پایگاه، کلیک دانلود، cookie و Bearer و hostname ذخیره را پوشش دهد. پیوند مستقیم موجود در component شواهد یکی از نقاطی است که به آزمون مرورگر نیاز دارد. گزارش فعلی از وجود endpoint محافظت‌شده نتیجه نمی‌گیرد که تمام راه‌های باز کردن فایل در رابط موفق‌اند.

۶-۷ شواهد آزمون و توسعه‌ی واقع‌بینانه

در اجرای منتخب هفت تست extraction موفق شدند: متن ساده، PDF با لایه‌ی متن، نبود OCR، OCR تصویر انگلیسی و فارسی و نوع نامجاز در میان آن‌هاست. این نتیجه نشان می‌دهد مسیرهای بررسی‌شده در محیط موجود کار کرده‌اند. آزمون‌های API اسناد و تحلیل سند در این اجرای محدود انتخاب نشده‌اند و نتیجه‌ی موفق فعلی برای آن‌ها ادعا نمی‌شود.

برای مرحله‌ی بعد می‌توان مجموعه‌ای از اسناد مصنوعی با متن مرجع ساخت: سند کاملاً متنی، کاملاً اسکن‌شده، ترکیبی، جدول چندستونی، متن فارسی و لاتین و سند بیش از سقف صفحه. باید هم پوشش صفحات و هم خطای محتوا اندازه‌گیری شود. استخراج فیلد نیازمند schema، منبع هر مقدار، وضعیت بازبینی و آزمون روی واحد و عدد است. افزودن نام چند فیلد به prompt، جای این زنجیره‌ی اعتبارسنجی را نمی‌گیرد.

فصل هفتم مدل زبانی و گفتگوی بالینی

۷-۱ قرارداد ارائه‌دهنده و پیکربندی

LLMProvider یک Protocol با complete است. ورودی‌های آن system، user و history اختیاری و خروجی آن str است. سه پیاده‌سازی موجود عبارت‌اند از OpenAICompatibleProvider، GeminiProvider و .OpenRouterProvider درخواست با httpx ساخته می‌شود و SDK مدل شرط اجرا نیست. قرارداد کوچک باعث می‌شود تحلیل سند، پاک‌سازی صوت، گفتگو و ایمنی ساختاریافته از همان لایه استفاده کنند، هرچند prompt و قرارداد خروجی‌شان متفاوت است.

build_provider شناسه‌ی provider و model را از درخواست یا تنظیمات می‌گیرد. شناسه‌ی ارائه‌دهنده باید در PROVIDER_IDS باشد. model طول حداکثر دویست دارد و URL یا رشته‌ی دارای فاصله پذیرفته نمی‌شود. کلید و base URL از کاربر دریافت نمی‌شوند. در OpenRouter کلید اختصاصی OPENROUTER_API_KEY مقدم و AI_API_KEY جایگزین است؛ سایر مسیرهای متن از AI_API_KEY استفاده می‌کنند. نبود کلید AIUnavailableError ایجاد می‌کند.

get_provider نمونه‌ی پیش‌فرض محیط را cache می‌کند و انتخاب صریح درخواست از build_provider می‌گذرد. get_settings نیز cache دارد؛ تغییر فایل محیط در فرایند فعال لزوماً تنظیمات را تازه نمی‌کند. reset_provider_cache برای آزمون وجود دارد. در استفاده‌ی عادی باید اثر restart و متغیر build-time رابط از تنظیمات runtime سرور جدا توضیح داده شود.

جدول ۵ قرارداد ارائه‌دهنده‌های متن

ارائه‌دهنده	قالب تماس	کلید و محدودیت
OpenAICompatibleProvider	chat/completions و messages	AI_API_KEY؛ base URL از سرور
GeminiProvider	generateContent و system_instruction	AI_API_KEY؛ roles به مدل نگاشت می‌شوند
OpenRouterProvider	chat/completions و X-Title	OPENROUTER_API_KEY سپس AI_API_KEY؛ نام مدل تضمین availability نیست

۷-۲ درخواست OpenAI-compatible و OpenRouter

OpenAICompatibleProvider فهرست messages را با system، history و پیام user می‌سازد و به /chat/completions می‌فرستد. model در JSON و کلید در Authorization قرار می‌گیرد. پاسخ از choices[0].message.content خوانده می‌شود. statusهای ۴۰۱ و ۴۰۳ به رد کلید، ۴۲۹ به محدودیت نرخ

و سایر وضعیت‌های غیر۲۰۰ به AIError تبدیل می‌شوند. نبود ساختار مورد انتظار نیز به خطای تمیز تبدیل می‌گردد.

OpenRouterProvider همین پیاده‌سازی را ارث می‌برد و base URL و X-Title را تعیین می‌کند. این اتصال به aggregator به معنی تأیید وجود یا کیفیت هر model id نیست. شناسه‌ی پیش‌فرض در کد deepseek/deepseek-r1:free است؛ این نام، داده‌ی پیکربندی مخزن است و دسترس‌پذیری روز آن در این بررسی با درخواست واقعی آزموده نشده است. مستندات رسمی OpenRouter قرارداد عمومی API را شرح می‌دهد [۶].

مدل انتخابی در تمام گردش کارها از رابط اعمال نمی‌شود. chat و conversation فیلد provider و model می‌پذیرند، اما تحلیل سند، پاک‌سازی و endpoint اختصاصی ایمنی از get_provider پیش‌فرض استفاده می‌کنند. حالت agent هم از تنظیم پیش‌فرض می‌گیرد. بنابراین انتخاب یک مدل در selector گفتگو نباید در گزارش به‌عنوان تغییر مدل تمام سامانه معرفی شود.

۷-۳ Gemini و تفاوت قالب پیام

GeminiProvider از models/{model}:generateContent در v1beta استفاده می‌کند. system در system_instruction قرار می‌گیرد و history با roleهای Gemini تبدیل می‌شود؛ assistant به model نگاشت می‌گردد. محتوا به‌صورت parts دارای text ارسال و پاسخ از candidates[0].content.parts به هم متصل می‌شود. کلید در x-goog-api-key قرار دارد. مستندات Google روش تولید متن را توضیح می‌دهد [۵]؛ قالب خاص این client از کد برداشت شده است.

برای ۴۰۳ HTML یک تشخیص جدا وجود دارد تا مسدود شدن در سطح شبکه با خطای کلید یکسان تلقی نشود. در ۴۰۰ با متن «API key not valid» رد کلید تشخیص داده می‌شود. این شاخه‌ها برای بیان علت در log مفیدند، اما معیار جامع تشخیص همه‌ی محدودیت‌های شبکه نیستند. API به کاربر پیام فارسی عمومی می‌دهد و علت اصلی در سمت سرور ثبت می‌شود.

نام gemini-3.6-flash در تنظیم و کاتالوگ ثابت پروژه دیده می‌شود. گزارش وجود این مقدار در کد را ثبت می‌کند و آن را مدل رسمی تأییدشده یا نتیجه‌ی یک فراخوانی موفق نمی‌نامد. فهرست Gemini در catalog ایستا است و از endpoint کشف مدل گرفته نمی‌شود. اگر نامی در ارائه‌دهنده موجود نباشد، انتخاب در UI یا وجود کلید مانع خطای تماس نمی‌شود.

۷-۴ تکرار تماس و خطای شبکه

_post_with_retries برای خطاهای transport حداکثر سه تلاش دارد و میان آن‌ها یک و دو ثانیه منتظر می‌ماند. پاسخ HTTP خطا به‌خودی‌خود در این helper retry نمی‌شود؛ مثلاً ۴۲۹ از مسیر تفسیر پاسخ به

AIError می‌رود. timeout از AI_TIMEOUT_SECONDS می‌آید. چند تلاش می‌تواند زمان نهایی را بیش از timeout یک تماس کند. بودجه‌ی کل و deadline سراسری برای گردش کار تعریف نشده است.

هر provider برای complete یک httpx.Client می‌سازد و در finally آن را می‌بندد. AI_PROXY به این client منتقل می‌شود. log درخواست حضور کلید را به‌صورت boolean بیان می‌کند، ولی _log_response تا هزار نویسه‌ی متن پاسخ را در سطح info ثبت می‌کند. این متن ممکن است اطلاعات بالینی تولیدشده داشته باشد. پس عدم چاپ کلید به معنی عدم ثبت اطلاعات بیمار در log نیست؛ پاک‌سازی و سیاست نگهداری log لازم است.

۷-۵ ساخت ورودی مدل و محدودیت زمینه

build_user_message یک JSON با ensure_ascii=False و سپس سؤال پزشک می‌سازد. متن extracted_text، transcript و content یادداشت هرکدام تا دو هزار نویسه کوتاه می‌شوند. سقف به هر فیلد مربوط است و محدودیت token کل پیام نیست. هفت گروه دویست‌تایی همراه metadata و متن‌های مشتق دیگر می‌توانند ورودی بزرگی بسازند. ai_summary و cleaned_note در این helper به همان روش محدود نمی‌شوند.

در قطع متن، علامت حذف اضافه می‌شود ولی انتخاب قسمت مرتبط با سؤال انجام نمی‌شود. اگر بخش لازم در پایان یک سند باشد ممکن است وارد مدل نشود. از سوی دیگر JSON پرونده مشخصات پایه را نیز حمل می‌کند و ناشناس‌سازی پیش از ارسال در این مسیر دیده نمی‌شود. کنترل زمینه‌ی آینده باید اندازه، ارتباط و محرمانگی را هم‌زمان پوشش دهد و حد صرفاً نویسه‌ای را معادل محافظت کامل تلقی نکند.

۷-۶ گفتگوی بدون ماندگاری

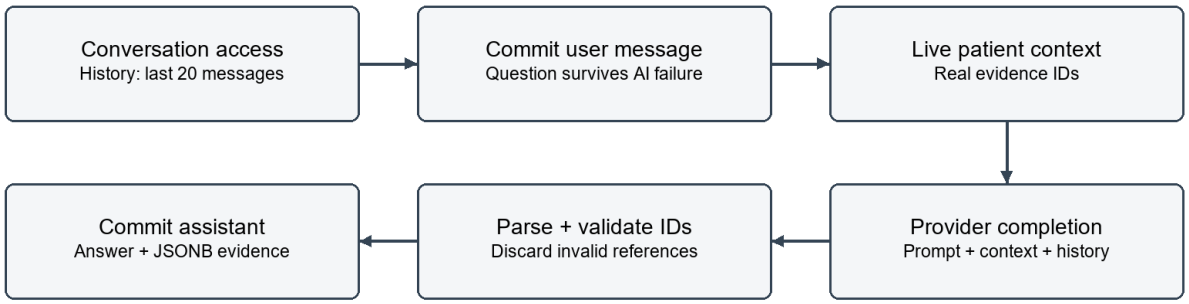
POST /api/v1/patients/{patient_id}/chat سؤال، تاریخچه و فلگ medication_safety را می‌گیرد. سؤال طول یک تا دو هزار و هر پیام تاریخچه تا چهار هزار نویسه دارد؛ فهرست تاریخچه حداکثر بیست ورودی می‌پذیرد. route از انتهای تاریخچه به اندازه‌ی AI_MAX_HISTORY_TURNS استفاده می‌کند که پیش‌فرض شش است. با وجود نام turns، برش روی تعداد پیام انجام می‌شود و الزاماً شش زوج سؤال و پاسخ نیست.

Context پس از مجوز ساخته می‌شود و prompt معمول یا ایمنی انتخاب می‌گردد. نتیجه ChatResponse شامل patient_id، question، answer و generated_at است. این route پیام را در پایگاه نگهداری نمی‌کند و evidence ساختاریافته‌ی مسیر ماندگار را نیز اجرا نمی‌کند. هدف آن قرارداد ساده‌ی اولیه و سازگاری است؛ شرح Citation باید به مسیر درست نسبت داده شود.

۷-۷ گفتگوی ماندگار و ترتیب commit

Conversation به بیمار و ConversationMessage به گفتگو وابسته‌اند. عنوان پیش‌فرض «گفتگوی جدید» است و نخستین پیام در نبود عنوان صریح به عنوان کوتاه تا پنجاه نویسه تبدیل می‌شود. این عنوان‌سازی قطعی است و مدل دیگری فراخوانی نمی‌کند. فهرست گفتگو بر updated_at و id نزولی و پیام‌ها بر id صعودی ارائه می‌شوند. شواهد پیام دستیار در JSONB ذخیره می‌گردند.

send_conversation_message ابتدا مجوز گفتگو را می‌سنجد و بیست پیام اخیر پیش از پیام جاری را خوانده، ترتیب زمانی آن‌ها را بازمی‌گرداند. سپس پیام user و در صورت لزوم عنوان را commit می‌کند. بعد provider انتخاب می‌شود و Context تازه، index شواهد و annotation ساخته می‌شوند. تماس مدل انجام می‌شود، پاسخ و شناسه‌ها parse و validate می‌شوند و پیام assistant همراه شواهد commit می‌شود. شکل ۷-۱ این دو مرز ثبت را مشخص می‌کند.



شکل ۷-۱ توالی گفتگوی ماندگار و حفظ سؤال در شکست مدل

اگر انتخاب provider نامعتبر یا سرویس تنظیم‌نشده باشد نیز پیام کاربر پیش‌تر ثبت شده است. در شکست AI پاسخ ۵۰۲ می‌آید و پیام دستیار ساخته نمی‌شود. retry کاربر می‌تواند پیام تازه‌ی تکراری بسازد؛ شناسه‌ی درخواست برای جلوگیری از آن وجود ندارد. به‌علاوه فراخوانی‌های هم‌زمان ممکن است پاسخ‌ها را با ترتیب پایان ذخیره کنند. ترتیب id در نتیجه، ترتیب کامل و قطعی زوج‌های مکالمه تحت رقابت نیست و نیاز به آزمون دارد.

۷-۸ کاتالوگ و رابط گفتگو

GET /api/v1/ai/providers/ احراز هویت می‌خواهد و فقط شناسه و نام ارائه‌دهنده و مدل را عرضه می‌کند. Gemini از فهرست ثابت و OpenRouter از /models خوانده می‌شود. نتیجه‌ی کشف تا سیصد ثانیه cache می‌گردد و شکست کشف مدل خیالی تولید نمی‌کند. با این حال حضور کاتالوگ عمومی OpenRouter به‌تنهایی به معنی آماده بودن کلید برای completion نیست و build_provider دوباره کلید را می‌سنجد.

component گفتگوی بالینی selector مدل و حالت ایمنی و حالت آزمایشی agent دارد. انتخاب‌ها در localStorage نگهداری می‌شوند و در ارسال پیام ماندگار provider و model می‌آیند. متن پاسخ و منابع در

componentهای مجزا نمایش داده می‌شوند. گزارش این جزئیات را برای تشریح اتصال API به گردش کار ذکر می‌کند؛ طراحی اصلی Frontend طبق پروپوزال در حوزه‌ی مسئولیت ایمان قرار دارد و یکپارچه‌سازی وظیفه‌ی مشترک است.

۹-۷ دامنه‌ی تضمین و ارزیابی لازم

قواعد prompt از جعل داده و تصمیم مستقل منع می‌کنند، اما اعتبارسنجی فعلی روی متن آزاد نمی‌تواند هر ادعای پزشکی را اثبات یا رد کند. در مسیر evidence اعتبار شناسه کنترل می‌شود و در ایمنی JSON نوع فیلدها؛ این دو کنترل مکمل‌اند ولی معیار صحت پزشکی نیستند. برای ارزیابی باید پاسخ‌ها بر اساس شواهد پرونده و دانش مرجع بررسی شوند و نرخ ادعای بدون پشتوانه به‌صورت مستقل گزارش گردد.

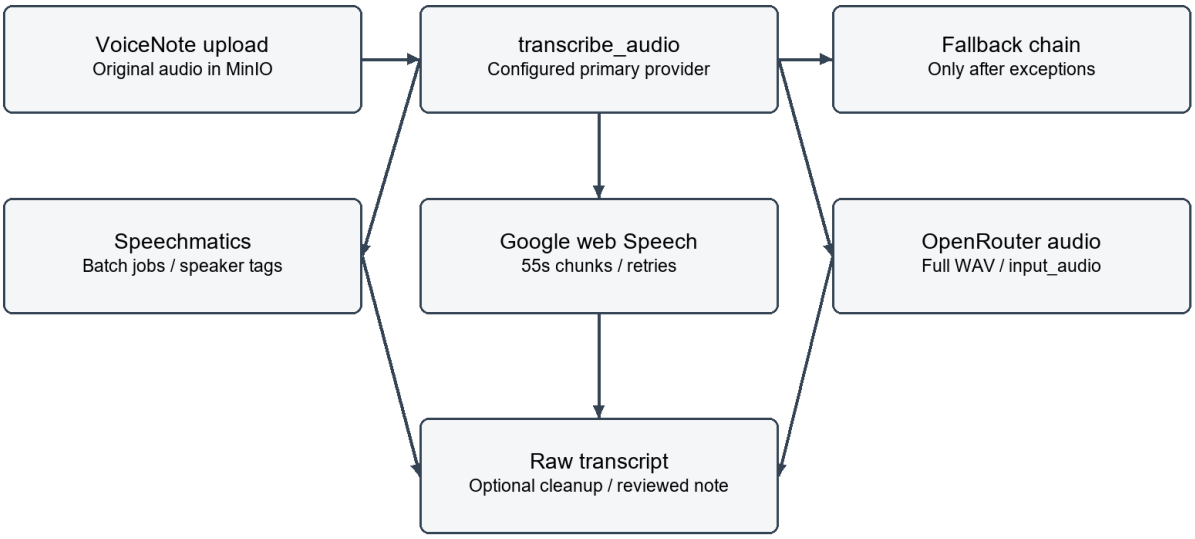
test_openrouter_provider.py در اجرای منتخب یازده آزمون موفق داشت و ساخت درخواست، خطا و parse پاسخ را با MockTransport بررسی کرد. این نتیجه موفقیت قرارداد کد است، نه موفقیت سرویس زنده یا صحت پاسخ بالینی. تست انتخاب catalog و APIهای گفتگو در مخزن وجود دارد ولی در این اجرا انتخاب نشده است. گزارش‌های تسک قبلی نیز شکست واقعی شبکه‌ی Gemini را ذکر کرده‌اند و نباید به پاسخ موفق تبدیل شوند.

فصل هشتم صوت و تبدیل گفتار به متن

۸-۱ ثبت صوت و وضعیت‌های مستقل

آپلود صوت در مسیر وابسته به بیمار با عنوان، conversation_type و duration_seconds اختیاری انجام می‌شود. MIME از پارامترهای بعد از نقطه‌ویرگول جدا می‌شود تا خروجی MediaRecorder مانند audio/webm دارای codecs پذیرفته شود. فهرست مجاز شامل WebM، OGG، MP3، MP4/M4A، WAV و FLAC و نیز video/webm است. سقف Backend پنجاه MiB و نام فایل سیصد نویسه است. duration_seconds از فرم دریافت می‌شود و اندازه‌گیری معتبر مدت در این route انجام نمی‌شود.

اصل صوت در پوشه‌ی voice از ذخیره‌ساز قرار می‌گیرد. VoiceNote شامل زبان، نوع مکالمه، وضعیت رونویسی، transcript، وضعیت پاک‌سازی، cleaned_note، error_message و processed_at است. پردازش پس‌زمینه ابتدا processing و سپس completed یا failed را ثبت می‌کند. در اجرای دوباره، cleaned_note پاک و cleanup_status pending می‌شود؛ خروجی مشتق قبلی دیگر متناظر با ورودی تازه فرض نمی‌شود.



شکل ۸-۱ ارائه‌دهنده‌های گفتار و مسیرهای جایگزین

۸-۲ قرارداد STT و انتخاب ارائه‌دهنده

SpeechToTextProvider دارای transcribe با data، file_name، mime_type، language و diarize است و متن برمی‌گرداند. facade عمومی transcribe_audio نام provider را از STT_PROVIDER می‌گیرد. پیش‌فرض کد speechmatics است. مسیرهای google و openrouter در

فایل کاری فعلی وجود دارند و زنجیره‌ی STT_FALLBACK_PROVIDERS می‌تواند پس از مسیر اصلی اجرا شود. این طراحی انتخاب سرویس را از route صوت جدا می‌کند.

نتیجه‌ی قرارداد فقط متن است. شناسه‌ی provider موفق، زمان‌بندی واژه‌ها، confidence و جزئیات job در مدل VoiceNote ذخیره نمی‌شوند. بنابراین وقتی fallback موفق شود، متن نهایی در پایگاه لزوماً نشان نمی‌دهد کدام مسیر آن را ساخته است. ثبت metadata ارائه‌دهنده در آینده برای ارزیابی و ردیابی اهمیت دارد. همچنین تمام providerها تضمین یکسانی درباره‌ی گوینده یا متن خالی ندارند.

۸-۳ Speechmatics و کار batch

SpeechmaticsProvider ابتدا job_config با type=transcription، language و operating_point=enhanced می‌سازد. اگر diarize درخواست شود، diarization=max افزوده می‌شود. فایل و JSON تنظیمات به jobs/ POST می‌روند. پاسخ ۲۰۱ باید id کار را داشته باشد. سپس GET jobs/{id}/ در فاصله‌ی پیش‌فرض سه ثانیه تا deadline پیش‌فرض سیصد ثانیه تکرار می‌شود. done پایان موفق و error، rejected یا cant_run شکست است.

متن از jobs/{id}/transcript با format=json-v2 گرفته می‌شود. format_transcript_json نخستین alternative هر بخش را می‌خواند و هنگام تغییر speaker خط جدید با برچسب می‌سازد. برچسب S1 یا S2 به‌تنهایی معلوم نمی‌کند فرد پزشک است یا بیمار. نسبت دادن نقش نیازمند اطلاعات یا بازبینی بیشتر است. ورودی doctor_patient سبب درخواست diarize می‌شود؛ physician_note آن را فعال نمی‌کند.

این مسیر یک job غیرهمگام در سرویس خارجی دارد، ولی worker Backend تا نتیجه منتظر می‌ماند. صف پایدار مستقل برای پیگیری job پس از restart موجود نیست و شناسه‌ی job هم در VoiceNote ثبت نمی‌شود. پاسخ خالی می‌تواند به transcript=None با وضعیت completed تبدیل شود. completed یعنی کار اجرا شده است؛ نیاز به بازبینی کفایت متن همچنان وجود دارد.

۸-۴ Google web Speech و آماده‌سازی فایل

GoogleSpeechProvider از SpeechRecognition و recognize_google استفاده می‌کند؛ این مسیر Google Cloud Speech-to-Text با حساب سرویس نیست. زبان fa به fa-IR نگاشت می‌شود. ffprobe مدت فایل را می‌خواند و ffmpeg آن را به WAV تک‌کاناله با نرخ شانزده هزار نمونه در ثانیه تبدیل می‌کند. این تبدیل شکل ورودی را یکنواخت می‌کند؛ ادعای افزایش دقت از آن بدون آزمایش قابل نتیجه‌گیری نیست.

اگر مدت نامعلوم یا کمتر از سقف باشد یک فایل WAV ساخته می‌شود. برای مدت بیشتر، بازه‌ها از صفر با گام STT_GOOGLE_MAX_CHUNK_SECONDS، پیش‌فرض پنجاه‌وپنج ثانیه، ایجاد می‌شوند. در آزمون صوت ۱۲۰ ثانیه‌ای سه شروع صفر، ۵۵ و ۱۱۰ انتظار می‌رود. قطعه‌ها هم‌پوشانی ندارند و متن خروجی با

فاصله به هم متصل می‌شود. بریدن واژه روی مرز قطعه و از دست رفتن زمینه‌ی جمله باید در ارزیابی واقعی بررسی شود.

برای هر قطعه UnknownValueError باعث عبور از آن قطعه می‌شود. خطای درخواست تا تعداد STT_GOOGLE_RETRIES، پیش‌فرض سه، دوباره امتحان می‌شود. اگر هیچ متن تولید نشود TranscriptionError فارسی ایجاد می‌شود. اگر بعضی قطعه‌ها غیرقابل تشخیص باشند و بقیه متن بدهند، نتیجه‌ی جزئی می‌تواند موفق محسوب شود و fallback فعال نشود. در خروجی ساختاریافته پرچم «برخی قطعه‌ها حذف شدند» وجود ندارد.

مسیر Google تفکیک گوینده ندارد. با diarize=True فقط warning ثبت و متن ساده برگردانده می‌شود. برای proxy، urllib opener به‌صورت عمومی نصب و در finally با None بازگردانده می‌شود. این تغییر در سطح فرایند است و لزوماً opener قبلی را حفظ نمی‌کند؛ هم‌زمانی درخواست‌ها نیز می‌تواند اثر بپذیرد. برای کاربرد جدی بهتر است transport خصوصی و قابل آزمون برای هر client وجود داشته باشد.

۸-۵ OpenRouter برای ورودی صوت

OpenRouterSTTProvider کلید اختصاصی یا AI_API_KEY و مدل STT_OPENROUTER_MODEL می‌خواهد. فایل کامل با ffmpeg به WAV شانزده کیلوهرتز mono تبدیل و به Base64 تبدیل می‌شود. پیام user دو بخش text و input_audio دارد. prompt می‌خواهد گفتار دقیقاً رونویسی شود و فقط متن بازگردد. این مسیر از complete متنی جداست تا قرارداد LLMProvider تغییر نکند.

تبدیل تمام صوت به یک payload باعث می‌شود اندازه‌ی Base64 بیشتر از بایت اولیه باشد و همه‌ی مدل‌های کاتالوگ قابلیت audio نداشته باشند. کد chunking خاصی برای این provider ندارد. ظرفیت مدل منتخب، حد درخواست و زمان conversion باید مستقل سنجیده شوند. نام STT_OPENROUTER_MODEL در تنظیمات خالی است و تنظیم شدن AI_MODEL به‌تنهایی جای آن را پر نمی‌کند.

این provider مقدار proxy و timeout را در constructor می‌خواند، اما تماس از client_ عمومی STT انجام می‌شود؛ آن client با timeout شصت ثانیه ساخته شده و AI_PROXY را دریافت نمی‌کند. بنابراین وجود self.proxy اثبات اعمال proxy نیست. این شکاف از مسیر واقعی فراخوانی مشاهده شده و در گزارش به‌عنوان محدودیت کنونی ثبت می‌شود. مسیر OpenRouter صوت نیز diarization ندارد و با درخواست آن warning می‌دهد.

۸-۶ زنجیره‌ی جایگزین و معنای موفقیت

در نبود STT_FALLBACK_PROVIDERS، خطای provider مستقیم عبور می‌کند. در وجود فهرست، provider اصلی و سپس نام‌های جایگزین به ترتیب امتحان می‌شوند. SttUnavailableError، TranscriptionError و حتی exception عمومی ثبت و مسیر بعدی انتخاب می‌شود. نخستین نتیجه‌ی موفق بازگردانده می‌شود و شکست همه‌ی مسیرها یک خطای ترکیبی فارسی دارد. نام تکراری در تنظیمات حذف نمی‌شود و می‌تواند تماس دوباره بسازد.

fallback راهی برای ادامه‌ی سرویس است، نه تضمین حفظ توانایی‌های مسیر اصلی. اگر Speechmatics با تفکیک گوینده شکست بخورد و Google موفق شود، متن بدون تفکیک خواهد بود. همچنین مسیر اصلی می‌تواند متن خالی بازگرداند و facade آن را موفق بداند؛ fallback بر exception فعال می‌شود، نه بر ارزیابی کیفیت محتوا. بهتر است قرارداد آینده علاوه بر text، قابلیت‌ها و کامل بودن نتیجه را برگرداند.

۸-۷ فایل موقت و نکته‌ی Windows

descriptor _mkstemp_path ساخته‌شده را بلافاصله می‌بندد، زیرا Windows حذف فایل باز را محدود می‌کند. فایل منبع و WAVها در finally با cleanup_ پاک می‌شوند و OSError در پاک‌سازی best-effort گرفته می‌شود. مسیر نام فایل با suffix اصلی ساخته می‌شود و فرمان ffmpeg به‌صورت فهرست آرگومان اجرا می‌گردد؛ shell interpolation برای نام فایل استفاده نشده است.

در Google، فایل قطعه پس از تبدیل موفق به فهرست chunks اضافه می‌شود. اگر conversion پیش از افزودن آن شکست بخورد، finally فقط source و chunks ثبت‌شده را پاک می‌کند و فایل ساخته‌شده‌ی تازه ممکن است باقی بماند. آزمون موجود پاک‌سازی مسیر موفق را بررسی می‌کند؛ این شاخه‌ی شکست به آزمون جدا نیاز دارد. subprocessها timeout صریح ندارند و ffprobe یا ffmpeg معیوب می‌تواند زمان کار را از کنترل معمول STT خارج کند.

Dockerfile فعلی Backend، Tesseract را نصب می‌کند ولی ffmpeg و ffprobe را ندارد. اضافه شدن SpeechRecognition در requirements به معنای وجود ابزارهای سیستم در container نیست. بنابراین مسیرهای جدید Google و OpenRouter صوت در تصویر فعلی به آماده‌سازی اضافی نیاز دارند. این محدودیت از تفاوت وابستگی Python و executable ناشی می‌شود و پیش از ادعای قابلیت استقرار باید رفع و آزموده شود.

۸-۸ پاک‌سازی اختیاری و تأیید یادداشت

پس از ثبت رونویسی، attempt_cleanup از AI پیش‌فرض استفاده می‌کند. prompt فقط قالب‌بندی و روشن کردن همان گفته‌ها را می‌خواهد و افزودن تشخیص، دارو، علامت یا اندازه‌گیری را ممنوع می‌کند. transcript تا بیست هزار نویسه محدود می‌شود. نبود provider پاک‌سازی را رد می‌کند و وضعیت می‌تواند

pending بماند؛ شکست دیگر cleaned_note را خالی و cleanup_status را failed می‌کند. متن خام نباید جایگزین شود.

کد try اصلی در orchestration کل پردازش را در بر می‌گیرد. بیشتر خطاهای پاک‌سازی در attempt_cleanup گرفته می‌شوند، اما خطای ساخت provider غیر از AIUnavailableError پیش از try داخلی ممکن است به caller برسد و رفتار failure کلی ایجاد کند. بنابراین قرارداد best-effort باید روی همه‌ی شاخه‌ها آزموده شود، نه صرفاً خطای .complete بررسی این نکته نمونه‌ای از تفاوت نیت comment و پوشش واقعی exception است.

مسیر /voice-notes/{voice_note_id}/note عنوان و content بازبینی‌شده را از کاربر می‌گیرد و ClinicalNote با source=voice و voice_note_id می‌سازد. این مرحله عمل صریح پزشک است. route شرطی برای completed بودن transcript یا برابر بودن متن با آن ندارد؛ می‌تواند متن ارسال‌شده‌ی کاربر را برای ضبط موجود ثبت کند. تأیید انسانی در قرارداد UI معنا دارد و باید با شناسه و زمان تأیید در سیاست آینده دقیق‌تر ثبت شود.

۹-۸ آزمون و سنجش کیفیت

در اجرای منتخب یازده آزمون STT پایه و بیست‌ویک آزمون Google/OpenRouter موفق شدند. آزمون‌ها conversion، recognize و HTTP خارجی را در مرز ماژول جایگزین می‌کنند. chunking، retry، متن غیرقابل تشخیص، قالب input_audio، نبود کلید و مدل، پاک‌سازی موفق و عبور fallback از این راه بررسی شده‌اند. این نتیجه به صدای واقعی، کیفیت فارسی، دقت نسبت دادن گوینده یا دسترس‌پذیری سرویس تعمیم داده نمی‌شود.

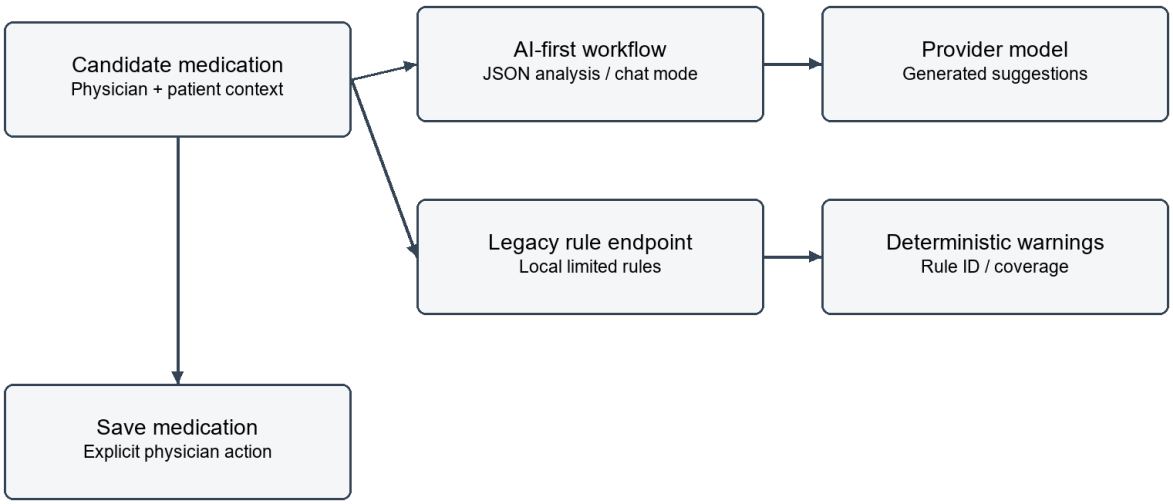
برای محاسبه‌ی WER باید تعداد جانشینی، حذف و درج بر تعداد واژه‌های متن مرجع تقسیم شود. CER همان منطق را روی نویسه دارد. در فارسی قواعد عادی‌سازی فاصله، نیم‌فاصله، شکل حروف و اعداد باید پیش از محاسبه مشخص شوند؛ وگرنه تفاوت نوشتاری با خطای گفتار مخلوط می‌شود. پروپوزال این ارزیابی را پیش‌بینی کرده، اما مجموعه‌ی مرجع و نتیجه‌ی عددی آن در شواهد بررسی‌شده وجود ندارد.

فصل نهم ایمنی دارو شواهد و عامل آزمایشی

۹-۱ تغییر مسیر ایمنی نسبت به طرح اولیه

پروپوزال برای Prescription Safety بر هشدارهای ساخته‌شده با قواعد و نقش توضیح‌دهنده‌ی LLM تأکید دارد. این اصل ابتدا در services/medication_safety.py و مسیر /medications/check پیاده شده است. سپس سرویس ai_medication_safety و حالت ایمنی چت افزوده شده‌اند که از قواعد قدیمی استفاده نمی‌کنند. رابط افزودن دارو از endpoint AI و چت از prompt تخصصی استفاده می‌کند. بنابراین مسیر فعلی را باید AI-first نامید و تفاوت آن با اصل اولیه را آشکار کرد.

موتور قدیمی حذف نشده و router آن همچنان ثبت است. وجود هم‌زمان دو مسیر، حفاظت ترکیبی خودکار ایجاد نمی‌کند. در درخواست AI، نتیجه‌ی قواعد به prompt افزوده نمی‌شود و خروجی AI نیز برای هشدار قطعی معتبرسازی بالینی نمی‌شود. شکل ۹-۱ دو مسیر مستقل را نشان می‌دهد. فلش جداگانه‌ی ثبت دارو بیان می‌کند که هیچ‌یک از تحلیل‌ها شرط اجباری ذخیره نیست.



شکل ۹-۱ تفکیک تحلیل AI و موتور محدود قواعد از ثبت دارو

ارزش مسیر AI در امکان بیان ملاحظات متنوع و اطلاعات ناقص است، اما هزینه‌ی آن وابستگی به مدل، تغییرپذیری پاسخ و نبود پوشش قابل شمارش قواعد است. موتور محدود قواعد تکرارپذیرتر است، ولی نام ناشناخته و شرایط خارج از دانش محلی را نمی‌پوشاند. هیچ‌کدام از این ویژگی‌ها به‌تنهایی برتری بالینی را ثابت نمی‌کند. انتخاب مسیر باید با سناریوهای مرجع و توجه به خطای منفی کاذب ارزیابی شود.

۹-۲ قرارداد تحلیل ساختاریافته

AiMedicationSafetyCheckRequest یک تا پنج MedicationCandidate با name و dose اختیاری و question اختیاری دارد. Context کامل builder و سؤال ساخته‌شده‌ی build_safety_question به مدل پیش‌فرض می‌روند. پاسخ شامل overall_assessment، interactions، patient_factors، missing_information و uncertainty است و route شناسه‌ی بیمار، echo کاندیدها، generated_at و disclaimer ثابت را اضافه می‌کند. داده‌ی این تحلیل در جدول مستقلی ذخیره نمی‌شود.

parse_analysis ابتدا JSON میان نخستین و آخرین آکولاد را پیدا می‌کند و fence را کنار می‌گذارد. overall_assessment باید رشته‌ی غیرخالی باشد و uncertainty رشته یا .null فیلدهای فهرستی باید list باشند و اعضای غیررشته‌ای حذف می‌شوند. نبود برخی فهرست‌ها به فهرست خالی تبدیل می‌شود. متن آزاد یا نوع نامعتبر به AIError و سپس ۵۰۲ می‌انجامد. بنابراین raw output شکسته به‌عنوان تحلیل ساختاریافته عرضه نمی‌شود.

این کنترل‌ها شکل داده را تضمین می‌کنند، نه صحت محتوا. prompt از مدل می‌خواهد تداخل احتمالی و شناخته‌شده را جدا و اطلاعات ناقص را بیان کند. parser وجود پیشوند درست هر interaction یا پشتوانه‌ی داروشناختی آن را راستی‌آزمایی نمی‌کند. schema شدت، rule_id یا شناسه‌ی استاندارد دارو ندارد. نداشتن هشدار در این خروجی نباید به معنی تجویز بی‌خطر تلقی شود.

۹-۳ حالت ایمنی در چت

فلگ medication_safety در چت معمول و ماندگار prompt را تغییر می‌دهد. تشخیص واژه‌ی دارو یا انتخاب خودکار حالت در کد وجود ندارد؛ کاربر toggle را فعال می‌کند. در مسیر ماندگار، قواعد evidence با prompt ایمنی ترکیب می‌شوند و پاسخ JSON شامل answer و evidence انتظار می‌رود. endpoint اختصاصی تحلیل، قرارداد متفاوتی با فهرست تعامل و عوامل بیمار دارد. یکسان بودن موضوع، یکسان بودن schema را ایجاب نمی‌کند.

تحلیل شکست‌خورده ثبت دارو را مسدود نمی‌کند و interface می‌تواند خطا را جدا نشان دهد. این جداسازی از وابستگی عملکرد پایه به سرویس بیرونی جلوگیری می‌کند. با این حال نباید به کاربر علامت موفق ایمنی بدهد وقتی فقط ذخیره موفق شده است. نتیجه‌ی تحلیل، وضعیت اجرا و عمل کاربر باید در ارائه‌ی آینده روشن و برای ممیزی قابل بازیابی باشند.

۹-۴ موتور قواعد محدود

medication_rules.json کلاس دارو، alias، interaction و cross_reactivity را تعریف می‌کند. load_rules_kb فایل را یک بار خوانده و dataclassهای ثابت می‌سازد. normalize_drug_name حروف عربی ی و ک را به فارسی تبدیل می‌کند، فاصله و نیم‌فاصله و نشانه‌ها را عادی می‌سازد و اعداد دوز و بعضی

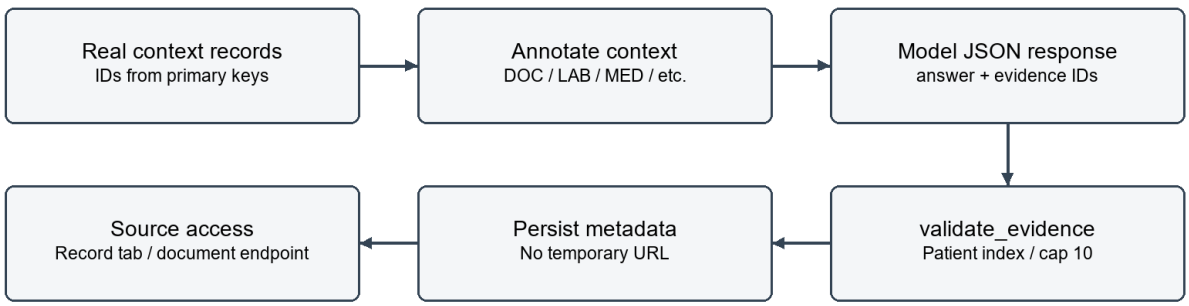
واحدها را کنار می‌گذارد. match_classes با alias به کلاس نگاشت می‌کند؛ این روش شناسه‌ی استاندارد دارویی یا حل جامع نام تجاری نیست.

check_medication_safety کاندید را با داروهای active و allergenهای پرونده می‌سنجد. هشدارها از سه نوع drug_interaction، allergy_conflict و duplicate_therapy هستند. هر هشدار rule_id، severity، title، detail و involved دارد و بر شدت مرتب می‌شود. برای کلاس دارای duplicate_allowed هشدار تکرار درمان ایجاد نمی‌شود. نامی که به کلاس شناخته‌شده نگاشت نشود، هشدار تازه نمی‌سازد و candidate_covered=false وضعیت پوشش را بیان می‌کند.

مسیر /medications/check می‌تواند کاندید را با medication_id یا medication_name بگیرد و در صورت explain، مدل فقط هشدارهای قطعی تولیدشده را توضیح می‌دهد. شکست توضیح به explanation=null می‌انجامد و هشدارها باقی می‌مانند. داده‌ی محلی قواعد را نباید دانش‌نامه‌ی تأییدشده‌ی پزشکی نامید؛ وجود rule_id به معنای نسخه‌پذیری محلی است و نه اعتبار بالینی همه‌ی متن قواعد.

۹-۵ شواهد پرونده و شناسه‌ی پایدار

build_evidence_index برای هر رکورد موجود در Context یک ID با پیشوند گروه و کلید اصلی می‌سازد. MED، ALLERGY، COND، LAB، DOC، NOTE و VOICE پیشوندهای واقعی‌اند. annotate_context این شناسه را کنار رکورد به پیام مدل اضافه می‌کند. مدل اجازه دارد فقط همین شناسه‌ها را استفاده کند و نباید مقاله یا URL بسازد. شکل ۹-۲ مسیر اعتبارسنجی و پیوند منبع را نشان می‌دهد.



شکل ۹-۲ اعتبارسنجی شناسه‌ی شواهد و دسترسی به منبع

validate_evidence شناسه‌های تکراری را کنار می‌گذارد و فقط موارد موجود در index همان بیمار را نگه می‌دارد؛ سقف پاسخ ده شاهد است. metadata از رکورد اصلی ساخته می‌شود و document_id تنها در Document یا رابطه‌ی واقعی آزمایش با سند وجود دارد. نشانی دانلود موقت در شواهد ذخیره نمی‌شود. این روش مانع تبدیل ID جعلی یا متعلق به بیمار دیگر به منبع معتبر در فهرست ساختاریافته می‌شود.

اعتبار ID با صحت استناد فرق دارد. مدل ممکن است MED واقعی را ذکر کند ولی از آن نتیجه‌ی پزشکی پشتیبانی‌نشده بگیرد. validate_evidence تطابق معنایی جمله و منبع را بررسی نمی‌کند و محتوای answer

را نیز برای شناسه‌های جعلی داخل متن بازنویسی نمی‌کند. بنابراین فهرست شواهد کنترل‌شده است، اما کل پاسخ با verifier معنایی تأیید نشده است. این مرز باید در هر ارزیابی Citation Accuracy رعایت شود.

۹-۶ parse پاسخ و نمایش منبع

parse_evidence_answer JSON با answer رشته‌ای و evidence فهرستی را می‌پذیرد. اگر پاسخ نثر یا JSON نامعتبر باشد، متن خام به‌عنوان answer و evidence خالی برمی‌گردد. این fallback پاسخ قابل خواندن را حفظ می‌کند، ولی عدم رعایت قرارداد JSON را به خطای قطعی تبدیل نمی‌کند. این رفتار با parser تحلیل دارو که در خطای ساختار ۵۰۲ می‌دهد متفاوت است و باید به مقصد مناسب نسبت داده شود.

component EvidenceList نوع منبع و شرح خوانا را نمایش می‌دهد؛ منابع دارای document_id به دانلود اصل سند و سایر منابع به تب پرونده هدایت می‌شوند. label دارو در UI «داروی فعال» است، در حالی که Context می‌تواند داروی stopped یا completed هم داشته باشد. این برچسب برای تمام شواهد دارویی دقیق نیست. در توسعه‌ی بعدی نمایش status در metadata و label وابسته به آن از برداشت اشتباه جلوگیری می‌کند.

شواهد پرونده فقط واقعیت ثبت‌شده درباره‌ی بیمار را پشتیبانی می‌کنند. رکورد مصرف یک دارو، مرجع اثبات تداخل عمومی آن دارو نیست. کد prompt همین تمایز را بیان می‌کند. PubMed، FDA و RxNorm در طرح به‌عنوان منابع ممکن آمده‌اند، اما سرویس بازیابی خارجی یا اتصال عملیاتی آن‌ها در جریان فعلی وجود ندارد. فهرست منابع علمی گزارش نیز نباید با منابعی که سامانه هنگام پاسخ خوانده مخلوط شود.

۹-۷ MCP محلی و عامل

MCP، مخفف Model Context Protocol، در این مخزن یک رابط ابزار محلی و فقط‌خواندنی است. app.mcp_server شش ابزار برای profile، Context، labs، medications، metadata اسناد و timeline دارد. هر ابزار Session خود را می‌سازد و داده‌ی بیمار را از مدل‌ها و builder موجود می‌خواند. ورودی، پارامترهای عددی مشخص است؛ SQL، نام جدول یا storage_key دلخواه از مدل گرفته نمی‌شود.

MCPStdioClient یک subprocess با همان Python و python -m app.mcp_server راه‌اندازی می‌کند. ارتباط stdio روی event loop پس‌زمینه و با thread انجام می‌شود تا route همگام از client غیرهمگام استفاده کند. ساخت session سی ثانیه و call ابزار شصت ثانیه timeout دارد. session cache می‌شود و در خرابی reset انجام می‌گیرد. این مسیر یک آزمایش معماری است و تحمل خطا و lifecycle کامل سرویس عملیاتی برای آن اثبات نشده است.

run_agent نخست از مدل برنامه‌ی ابزار را به JSON می‌خواهد. نام‌ها با TOOL_LABELS محدود و تکراری‌ها حذف می‌شوند و حداکثر سه ابزار اجرا می‌شود. شناسه‌ی بیمار را Backend از patient مجوزدار تزریق می‌کند؛ مدل فقط نام ابزار می‌گوید. نتیجه‌ی هر ابزار تا چهار هزار نویسه محدود و در فراخوانی دوم به

مدل داده می‌شود. مسیر answer مستقیم برای سؤال عمومی نیز وجود دارد و کنترل مستقل برای عمومی بودن سؤال اعمال نمی‌شود.

۹-۸ امنیت و محدودیت آزمایش عامل

endpoint عامل از patient_from_id استفاده می‌کند، ولی MCP مستقیم هویت کاربر یا ACL وب را دریافت نمی‌کند. بنابراین فقط وقتی subprocess در مرز اعتماد Backend باقی بماند می‌توان به بررسی دسترسی endpoint تکیه کرد. ارائه‌ی مستقیم ابزار به کاربر یا شبکه به سازوکار مجوز جدا نیاز دارد. read-only بودن مانع خواندن غیرمجاز داده‌ی محرمانه نمی‌شود.

ابزار get_patient_documents metadata محدود می‌دهد و متن و storage_key را حذف می‌کند، اما get_patient_context کل Context با متن استخراج‌شده را بازمی‌گرداند. پس جمله‌ی «MCP هیچ متن سندی را عرضه نمی‌کند» درباره‌ی کل سطح ابزار درست نیست. باید دامنه‌ی هر ابزار را مستقل بررسی کرد. شکل ابزار و نتیجه نیز در تست_mcp_tools تعریف شده، ولی آزمون زنده‌ی subprocess در این اجرای گزارش انجام نشده است.

عامل stateless است و پاسخ آن در Conversation ذخیره نمی‌شود. شکست MCP یا AI به خطای تمیز می‌انجامد و مسیر چت معمول جدا باقی می‌ماند. مسیر agent ابزار نسخه‌نویسی یا تغییر پرونده ندارد. این جدایی ارزش نمونه‌ی آزمایشی را نشان می‌دهد، اما نباید به‌عنوان عامل بالینی پایدار یا مستقل معرفی شود. ثبت ابزار استفاده‌شده شفافیت فنی ایجاد می‌کند و جای ارزیابی معنایی پاسخ را نمی‌گیرد.

فصل دهم احراز هویت امنیت و حریم خصوصی

۱۰-۱ حساب کاربری و توکن

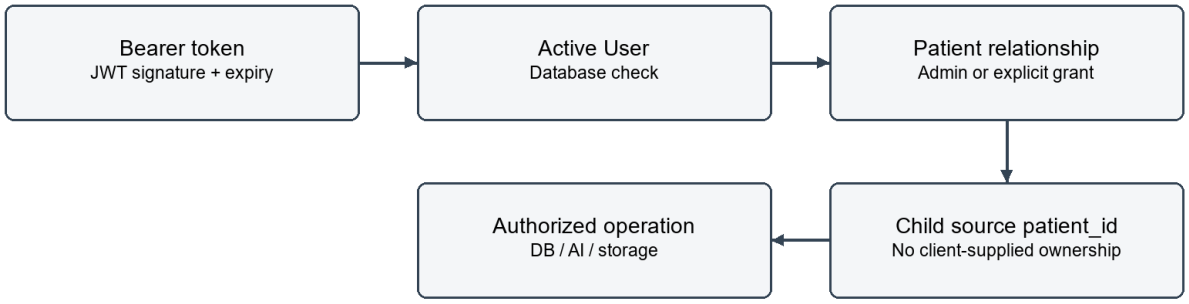
ثبت‌نام ایمیل و گذرواژه حساب doctor می‌سازد و درخواست نقش admin رد می‌شود. ایمیل به lowercase تبدیل و password با bcrypt ذخیره می‌شود. پاسخ UserResponse hash را عرضه نمی‌کند. login خطای ایمیل ناشناخته و گذرواژه‌ی غلط را با پیام یکسان بیان می‌کند و کاربر غیرفعال نیز توکن نمی‌گیرد. نقش admin از مسیر seed توسعه تعریف می‌شود و یک فرایند تأیید رسمی صلاحیت پزشکی در ثبت‌نام وجود ندارد.

JWT، مخفف JSON Web Token، شامل sub برابر id کاربر و exp است. algorithm پیش‌فرض HS256 و عمر پیش‌فرض چهارصدوهشتاد دقیقه است. decode فقط الگوریتم تنظیم‌شده را می‌پذیرد و subject را به عدد تبدیل می‌کند. get_current_user سپس User را از پایگاه می‌خواند و is_active را می‌سنجد. توکن فقط ادعای امضاشده است؛ فعال بودن حساب و مجوز پرونده در پایگاه دوباره کنترل می‌شوند.

JWT_SECRET_KEY در Settings یک مقدار توسعه‌ی ناامن دارد و باید پیش از هر استقرار غیرآزمایشی جایگزین شود. گزارش مقدار واقعی محیط یا کلید را بازنشر نمی‌کند. وجود متغیر محیطی به معنی مدیریت حرفه‌ای secret، rotation یا جلوگیری از همه‌ی نشت‌ها نیست. refresh token، فهرست ابطال token و نشست دستگاه در پیاده‌سازی فعلی دیده نمی‌شود؛ خروج از رابط توکن محلی را پاک می‌کند، ولی لزوماً نسخه‌ی کپی‌شده‌ی آن را باطل نمی‌کند.

۱۰-۲ اجازه‌ی پرونده و منابع فرزند

user_has_patient_access برای admin true می‌دهد و برای doctor وجود زوج PatientUserAccess را می‌سنجد. patient_from_id بعد از یافتن بیمار ensure_patient_access را اجرا می‌کند. در منبع مستقل مانند Document یا Conversation، helper رکورد را می‌خواند و patient_id واقعی آن را برای کنترل استفاده می‌کند. این الگو مانع تکیه بر patient_id ادعایی مرورگر برای مجوز یک سند می‌شود. شکل ۱۰-۱ جریان مجوز را نمایش می‌دهد.



شکل ۱۰-۱ احراز هویت و کنترل انتساب بیمار

برگرداندن ۴۰۴ برای منبع ناموجود و ۴۰۳ برای منبع موجود ولی غیرمجاز می‌تواند تفاوت وجود شناسه را نشان دهد. این رفتار فعلی است و سیاست پنهان‌سازی وجود منابع در همه‌ی routeها یکسان‌سازی نشده است. همچنین نقش doctor دارای دسترسی، سطح read-only و write جدا ندارد؛ قرارداد ACL فعلی انتساب پرونده را کنترل می‌کند، نه تمام سطح‌های مجوز سازمانی.

test_patient_access_control.py دسترسی doctor به پرونده‌ی خود و دیگری، bypass ادمین و کنترل مسیرهای مستقل را بررسی می‌کند. تست دانلود ترتیب مجوز پیش از presign را می‌سنجد و تست agent اجازه را پیش از MCP بررسی می‌کند. این آزمون‌ها به پایگاه آزمایشی وابسته‌اند و در اجرای محدود تازه اجرا نشده‌اند. وجود آن‌ها شواهد طراحی آزمون است و نتایج تاریخی در docs باید با تاریخ خود خوانده شوند.

۱۰-۳ ورود یک‌بارمصرف

ورود شماره‌ی تلفن در auth/request-otp/ و auth/verify-otp/ تعریف شده است. normalize_phone شکل‌های معمول شماره‌ی ایرانی را به E.164 تبدیل می‌کند. issue_otp با secrets.randbelow کد شش‌رقمی می‌سازد و hash SHA-256 از phone و code نگه می‌دارد. زمان انقضا ۱۲۰ ثانیه و attempts_left پنج است. verify جدیدترین درخواست را می‌خواند و پس از مقایسه‌ی hash با hmac.compare_digest، کد را مصرف می‌کند.

این hash مانع نگهداری متن ساده‌ی کد در جدول است، اما فضای شش‌رقمی محدود است و hash بدون secret در برابر مهاجم دارای پایگاه، مانع قوی حدس آفلاین نیست. طرح محدودیت درخواست، حداکثر سه کد در ده دقیقه را هدف می‌گیرد. شرط فعلی recent[0].created_at را با window_start می‌سنجد؛ اگر سه رکورد موجود باشند، تازه بودن جدیدترین شرط رد است حتی اگر دو رکورد دیگر خارج بازه باشند. بنابراین رفتار دقیق می‌تواند سخت‌گیرانه‌تر از شمارش سه درخواست در پنجره باشد.

کد از طریق Uniom به Soroush Plus ارسال می‌شود؛ این مسیر در پیاده‌سازی پیام‌رسان است و نباید SMS عادی نامیده شود. نبود تنظیم پیام‌رسان به ۵۰۳ می‌انجامد و رکورد جدید برای جلوگیری از محدود

شدن بی‌دلیل حذف می‌شود. خطای ارسال UniomError به ۵۰۲ تبدیل می‌شود، اما route شماره‌ی مقصد را در log خطا ثبت می‌کند. ادعای comment مبنی بر «شماره هرگز log نمی‌شود» برای تمام مسیرهای برنامه برقرار نیست.

verify-otp کاربر متناظر را پیدا یا می‌سازد و همان JWT سیستم را می‌دهد. مصرف کد و ساخت حساب مراحل پایگاه‌اند و مدیریت هم‌زمانی دو درخواست مصرف باید به‌طور جدا بررسی شود. lock یا update شرطی اتمیک برای جلوگیری از race در verify دیده نمی‌شود. تست‌های معمول انقضا و مصرف دوباره مفیدند، اما هم‌زمانی نیازمند سناریوی مستقل است.

۱۰-۴ نگهداری توکن در رابط

auth.ts توکن را در localStorage و cookie access_token نگه می‌دارد. /=cookie path، عمر هشت ساعت و SameSite=Lax دارد، ولی HttpOnly و Secure در رشته‌ی ساخته‌شده نیستند. این انتخاب برای انتقال توکن به Server Component ساده است؛ هزینه‌ی آن قابل خواندن بودن توکن با JavaScript است. در صورت آسیب‌پذیری XSS، مهاجم می‌تواند آن را بردارد. توصیه‌ی امنیتی مربوط به این نوع ذخیره را باید با منبع تخصصی و مدل تهدید بررسی کرد [۱۰].

Backend header-only است و cookie را مستقیم نمی‌خواند. درخواست API از مرورگر Bearer اضافه می‌کند و درخواست سروری آن را از cookie منتقل می‌کند. این تفاوت علت شکاف پیوند مستقیم دانلود است. استفاده‌ی cookie برای SSR به‌تنهایی یک راهکار کامل session محسوب نمی‌شود. توسعه‌ی بعدی می‌تواند واسط سروری با cookie امن و proxy دانلود مجوزدار را بررسی کند و هم‌زمان CSRF و خروج و انقضا را آزمون کند.

۱۰-۵ فایل داده و log

کلید ذخیره UUID دارد، ولی فایل‌ها در حالت معمول با MinIO نگهداری می‌شوند و رمزگذاری در سطح برنامه برای متن یا فایل در کد بررسی‌شده وجود ندارد. MINIO_SECURE انتخاب transport است و به معنی رمزگذاری پایگاه یا دیسک نیست. فایل موقت صوت نیز روی دیسک ایجاد می‌شود؛ پاک‌سازی معمول انجام می‌گیرد، اما خرابی فرایند می‌تواند آن را باقی بگذارد. retention و پاک‌سازی پس از crash باید جزء طراحی عملیاتی باشند.

AI context مشخصات بیمار را با متن و رکوردها به سرویس بیرونی می‌فرستد. کد رضایت، ناشناس‌سازی خودکار یا قرارداد منطقه‌ی پردازش را پیاده نکرده است. برای داده‌ی واقعی باید حداقل‌سازی داده، ضرورت ارسال و شرایط نگهداری سرویس بررسی شوند. این گزارش داده‌ی بیمار واقعی را نمونه قرار نمی‌دهد و مثال‌های پیوست مصنوعی‌اند. داشتن نمونه‌ی آموزشی مجوز کاربرد عملیاتی در بیمارستان نیست.

در logging، بخشی از پاسخ AI و در خطای چت بخشی از سؤال و در خطای OTP شماره دیده می‌شود. ClinicalEvent هم همه‌ی خواندن‌ها، تغییرها و هویت انجام‌دهنده را ثبت نمی‌کند. بنابراین لاگ موجود یک audit trail جامع بالینی نیست. افزودن ثبت حداقل عملیات حساس با redaction و سیاست دسترسی، از چاپ متن آزاد در log مفیدتر است و باید با نیاز عیب‌یابی متوازن شود.

۱۰-۶ کنترل‌های موجود و کارهای باقی‌مانده

CORS مبداهای مشخص را می‌پذیرد و preflight برای مبدا نامجاز رد می‌شود. این کنترل رفتار مرورگر را محدود می‌کند، نه client مستقل را؛ ابزار خارج مرورگر می‌تواند درخواست مستقیم بفرستد و همچنان به احراز هویت نیاز دارد. بررسی MIME و نام فایل نیز مفید است، ولی ضدبدافزار، اسکن محتوای فعال و sandbox parser مستقل نیست. هر کنترل باید در محدوده‌ی واقعی‌اش توصیف شود.

اولویت امنیتی آینده شامل secretهای غیرتوسعه، TLS واقعی، cookie امن، کنترل دانلود رابط، حذف داده‌ی حساس از log، محدودیت نرخ برای login، سیاست نگهداری فایل و آزمون race است. این موارد از شواهد کد و مسیرهای مشاهده‌شده نتیجه می‌شوند؛ هیچ ادعای انطباق قانونی یا گواهی امنیتی از وجود bcrypt، JWT و ACL استنتاج نمی‌شود. تأیید بالینی و مقرراتی فرایندی مستقل از آزمون نرم‌افزار است.

فصل یازدهم آزمون یکپارچه‌سازی و استقرار

۱۱-۱ ساختار آزمون‌ها

مخزن تست‌های service، schema، ORM و API و چند script سرتاسری دارد. conftest یک TestClient بدون پایگاه و fixtureهای پایگاه واقعی تعریف می‌کند. نام پایگاه تست با پسوند test_ از تنظیمات گرفته می‌شود. db_engine می‌تواند آن پایگاه را بسازد، metadata.create_all اجرا کند و در پایان drop_all انجام دهد. این فرایند روی پایگاه تست است، اما پیش از اجرای کامل باید اختصاصی بودن آن تأیید شود؛ اجرای بی‌بررسی روی محیط مشترک مناسب نیست.

db_session با تراکنش اتصال و create_savepoint کار می‌کند تا commitهای route در تست قابل بازگردانی باشند. api_client کاربر admin می‌سازد و get_db و get_current_user را override می‌کند. تست‌های اختصاصی auth و access client دیگری با وابستگی واقعی می‌سازند. تفکیک این دو گروه لازم است؛ سبز بودن تست CRUD با admin جای بررسی مجوز doctor را نمی‌گیرد.

در تست providerها، MockTransport پاسخ شبکه را کنترل می‌کند. در تست STT جدید، ffprobe، ffmpeg و recognize در مرز ماژول stub می‌شوند. این روش تست را از هزینه و ناپایداری سرویس بیرونی جدا می‌کند و امکان بررسی شاخه‌ی ۴۲۹ یا پاسخ نامعتبر را می‌دهد. در مقابل کیفیت سرویس واقعی و سازگاری فایل کامل باید با آزمون دیگری سنجیده شوند.

۱۱-۲ اجرای تازه برای این گزارش

برای بررسی عملی، هشت فایل موجود شامل health، CORS، logging، storage، extraction، STT پایه، STT جدید و OpenRouter provider انتخاب شدند. ابزار pytest در زیرپوشه‌ی گزارش نصب شد تا requirements و محیط برنامه تغییر نکند. اجرای انتخاب‌شده روی Python 3.12.2 و pytest 8.4.2 شصت‌ویک مورد جمع‌آوری کرد. نتیجه‌ی واقعی ۵۹ passed و ۲ error در setup بود؛ هیچ skipped یا شکست assertion در این مجموعه ثبت نشد.

دو خطا به test_storage_service.py مربوط‌اند. fixture به MinIO در localhost:9000 وصل شد و اتصال رد گردید. خطا از جنس آماده نبودن زیرساخت بود و به موفقیت یا skip تبدیل نشده است. هفت آزمون extraction، یازده STT پایه، بیست‌ویک STT جدید و یازده OpenRouter همراه تست‌های health و CORS و logging موفق شدند. مدت مشاهده‌شده‌ی این اجرای محدود ۳۰/۴۲ ثانیه است و معیار تأخیر سرویس کاربردی محسوب نمی‌شود.

جدول ۶ نتیجه‌ی اجرای منتخب

فایل آزمون	موفق	خطا	کنارگذاشته
test_health	3	0	0
test_cors	4	0	0
test_logging	2	0	0
test_storage_service	0	2	0
test_extraction_service	7	0	0
test_stt_service	11	0	0
test_stt_google_and_openrouter	21	0	0
test_openrouter_provider	11	0	0

این اجرا درخواست واقعی به AI یا STT ایجاد نکرده و Docker یا پایگاه داده را راه‌اندازی نکرده است. تست‌های API وابسته به PostgreSQL و تست‌های E2E کامل در آن انتخاب نشده‌اند. به همین دلیل نتیجه‌ی پنجاه‌ونه آزمون به کل suite تعمیم داده نمی‌شود. گزارش اعتبارسنجی فرمان، نسخه‌ها و محدوده‌ی اجرا را ثبت می‌کند تا نتیجه قابل پیگیری باشد.

۱۱-۳ نتایج تاریخی و تفسیر آن‌ها

docs/task-19-final-integration-e2e-report.md اجرای تاریخی ۳۶ مرحله‌ی smoke، آزمون دسترسی و بررسی migration را گزارش کرده است. همان سند تصریح می‌کند که تماس موفق واقعی Gemini به دلیل محدودیت شبکه روی محیط آن تسک تأیید نشده است. برخی عنوان‌های اسناد هنوز می‌گویند commit انجام نشده، در حالی که Git commit متناظر را دارد. تاریخچه‌ی نسخه‌ی فعلی نسبت به توضیح قدیمی سند برای وضعیت commit مقدم است.

مستند Task 14 نتیجه‌ی تست با provider جایگزین و مسیر خطای زنده‌ی AI را از هم جدا می‌کند. از این گزارش نمی‌توان کیفیت پزشکی تحلیل یا موفقیت حال حاضر Google را نتیجه گرفت. این اسناد برای فهم هدف تست و خطاهای شناخته‌شده مفیدند، اما بازاجرای مستقل نیستند. در ارزیابی نهایی بهتر است هر نتیجه با نسخه‌ی کد، تنظیم غیرمحرمانه و زمان اجرا همراه باشد.

۱۱-۴ طرح ارزیابی و معیارهای غایب

در شواهد بررسی‌شده WER، CER، P50، P95، SUS یا حساسیت بالینی با مجموعه‌ی مرجع گزارش نشده است. برای سنجش پاسخ AI باید دو حالت بدون زمینه و همراه زمینه بر سؤال‌های یکسان مقایسه شوند. ارزیاب باید علاوه بر مرتبط بودن پاسخ، ادعای unsupported، حفظ مقدار عددی، بیان اطلاعات ناقص و نسبت درست منبع را ثبت کند. این پیشنهاد، نتیجه‌ی انجام‌شده نیست و به‌عنوان طرح ارزیابی آینده ارائه می‌شود.

برای ایمنی دارو، true positive، false positive، true negative و false negative باید از سناریوی مرجع مستقل به دست آیند. حساسیت از TP تقسیم بر TP+FN و ویژگی از TN تقسیم بر TN+FP محاسبه می‌شود؛ کم بودن خطای منفی کاذب اهمیت ویژه دارد. نداشتن warning برای نام خارج پوشش نباید true negative شمرده شود. در مسیر AI، نتیجه‌ی عدم قطعیت نیز باید جدا از «ایمن» برچسب بخورد.

تأخیر باید برای مسیرهای جدا اندازه‌گیری شود: CRUD، upload تا ثبت، upload تا استخراج، سؤال تا پاسخ و صوت تا .transcript میانگین ساده می‌تواند دنباله‌ی بلند شبکه را پنهان کند؛ صدک‌ها و درصد timeout اطلاعات بیشتری می‌دهند. کنترل تعداد و طول درخواست، وضعیت cache و مدل لازم است. زمان اجرای pytest یا مقدار timeout در settings اندازه‌گیری latency واقعی نیست.

۱۱-۵ زیرساخت توسعه

docker-compose.yml ریشه فقط PostgreSQL و MinIO را اجرا می‌کند و Backend و Frontend در توسعه محلی‌اند. PostgreSQL از تصویر postgres:17-alpine و MinIO از latest استفاده می‌کند. هر دو volume دارند و healthcheck تعریف شده است. در این فایل پورت‌های پایگاه و ذخیره‌ساز به میزبان منتشر می‌شوند. این توپولوژی برای توسعه با فایل production متفاوت است و نباید شرح یکی جای دیگری بنشیند.

infra/README.md همان مرحله‌ی توسعه را شرح می‌دهد و می‌گوید container برنامه هنوز وجود ندارد. Dockerfileهای جدید و deploy/docker-compose.prod.yml بعداً به مخزن افزوده شده‌اند. این ناهماهنگی مستندات از مواردی است که بررسی کد آن را اصلاح می‌کند. گزارش استقرار، هر فایل را با دامنه‌ی خودش معرفی می‌کند و README قدیمی را وضعیت نهایی محصول نمی‌نامد.

۱۱-۶ تصویر Backend و Frontend

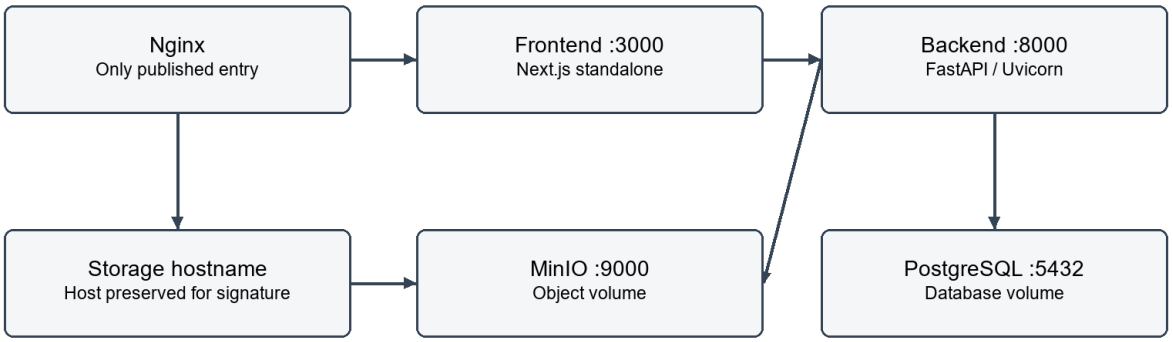
Backend بر python:3.12-slim بنا شده و Tesseract و زبان فارسی را نصب می‌کند. requirements پیش از کد کپی می‌شود تا cache لایه‌ی dependency استفاده شود. app، migrationها و scriptها وارد تصویر می‌شوند. کاربر appuser غیرroot با uid هزار برنامه را اجرا می‌کند و uvicorn روی 0.0.0.0:8000 قرار دارد. healthcheck از urllib درخواست health/ می‌زند. وجود این تنظیمات به معنی آزمون موفق build در اجرای حاضر نیست.

Frontend سه مرحله دارد: deps با npm ci از lockfile، builder با next build و runner با خروجی standalone. تصویر پایه node:24-alpine و runtime با کاربر node است. NEXT_PUBLIC_API_URL هنگام build در bundle قرار می‌گیرد و تغییر env پس از build لزوماً client را عوض نمی‌کند. مستندات Next.js درباره‌ی self-hosting و تنظیمات runtime توضیح می‌دهد [۳]؛ جزئیات تصویر حاضر از Dockerfile است.

نسخه‌های وابستگی Backend عمدتاً بازه‌ای‌اند و MinIO latest استفاده می‌شود. این انتخاب نگهداری را آسان‌تر می‌کند، ولی build دو زمان مختلف الزاماً بایت یکسان ندارد. برای تکرارپذیری می‌توان lock یا pin کنترل‌شده و ثبت digest تصویر را اضافه کرد. هر ارتقا نیز باید با تست سازگاری و migration بررسی شود، نه فقط نصب موفق .package

۱۱-۷ توپولوژی production و startup

پیکربندی production پنج سرویس nginx، frontend، backend، postgres و minio دارد. تنها nginx پورت ۸۰ و ۴۴۳ را publish می‌کند؛ سایر سرویس‌ها expose داخلی دارند. Backend منتظر healthy شدن پایگاه و MinIO است، ولی Frontend فقط Backend service_started را می‌خواهد. Nginx نیز dependency آغاز سرویس دارد. شکل ۱-۱۱ شبکه و volumeها را نمایش می‌دهد.



شکل ۱-۱۱ توپولوژی تعریف‌شده در Compose استقرار

healthcheck پایگاه با pg_isready و MinIO با mc ready است. Backend health فقط پاسخ HTTP را بررسی می‌کند. شرط started برابر آماده بودن تمام endpointها نیست و صحت migration را تضمین نمی‌کند. مستندات Docker تفاوت ترتیب راه‌اندازی و healthcheck را توضیح می‌دهد [۹]. در این مخزن migration عمداً خودکار نیست و باید به‌عنوان مرحله‌ی جدا اجرا شود.

۱۱-۸ Nginx و نشانی امضاشده

در Nginx، /api/ و /health به Backend و سایر مسیرهای برنامه به Frontend می‌روند. میزبان storage.local.test و wildcard متناظر به MinIO هدایت می‌شوند. Host بدون تغییر مهم است، زیرا امضای S3 به آن وابسته است. Backend با MINIO_ENDPOINT برابر hostname عمومی و extra_hosts نوع host-gateway به همان مسیر می‌رسد. اگر URL با minio:9000 ساخته شود مرورگر بیرون شبکه آن را نمی‌تواند resolve کند.

فایل پیکربندی فعلی فقط 80 listen دارد و comment نیز TLS را مرحله‌ی بعد می‌داند. publish کردن ۴۴۳ و mount کردن پوشه‌ی certs، TLS فعال ایجاد نمی‌کند. hostnameها نیز placeholder محیط

محلی‌اند و باید برای دامنه‌ی واقعی تغییر کنند. timeout proxy برای API سیصد ثانیه است؛ چند فراخوانی عامل یا retry ممکن است بودجه‌ی بیشتری بخواهد و باید در آزمون طولانی بررسی شود.

client_max_body_size در هر دو server برابر 25m است. از نگاه Backend صوت تا پنجاه MiB مجاز است، پس این اختلاف باید پیش از استقرار نهایی حل شود. volumeهای postgres_data و minio_data دوام داده پس از recreate container را هدف می‌گیرند، اما backup، restore و رمزگذاری را خودکار فراهم نمی‌کنند. وجود volume را نباید تضمین بازیابی پس از خرابی دیسک دانست.

۱۱-۹ رویه‌ی راه‌اندازی و محدودیت اثبات

رویه‌ی مستند شامل تنظیم محیط، build تصاویر، شروع data services، اجرای alembic upgrade head و بالا آوردن سرویس برنامه است. در محیط development می‌توان Backend و Frontend را محلی اجرا کرد؛ در production فایل Compose جدا استفاده می‌شود. مثال فرمان‌ها در پیوست آمده و مقادیر secret حذف شده‌اند. seed صرفاً برای محیط آزمایشی و داده‌ی مصنوعی مناسب است و نباید بدون بررسی وارد محیط واقعی شود.

در این کار container ساخته یا روی سرور عمومی منتشر نشده است. دسترس‌پذیری دامنه، DNS، TLS، نسخه‌ی ابزارهای صوت در تصویر و backup تأیید نشده‌اند. این محدودیت نتیجه‌ی گزارش را مشخص می‌کند: پیکربندی استقرار وجود دارد و قابل تحلیل است، اما آمادگی کامل بهره‌برداری یا موفقیت live deployment ادعا نمی‌شود. آزمون فنی محدود انجام‌شده، جای آزمون عملیاتی کل stack را نمی‌گیرد.

فصل دوازدهم خلاصه نتیجه‌گیری و کارهای آینده

۱۲-۱ تطبیق با هدف‌های مصوب

هسته‌ی ثبت بیمار و داده‌ی بالینی، پیوند اسناد و صوت، Context و timeline، اتصال مدل آماده و گفتگوی ماندگار در کد وجود دارد. قابلیت شواهد، IDهای واقعی پرونده را کنترل می‌کند و داده‌ی مشتق با فایل اولیه جدا نگهداری می‌شود. این مجموعه بخش مهمی از اهداف Backend و AI پروپوزال را پوشش می‌دهد. APIها و migrationها نیز مبنایی برای توسعه و آزمون تکرارپذیر فراهم می‌کنند.

چند هدف در سطح محدودتر اجرا شده‌اند: استخراج سند به متن و تحلیل آزاد می‌رسد، نه فیلد بالینی معتبر؛ Citation به رکورد پرونده محدود است، نه مقاله‌ی پزشکی؛ خلاصه‌ی نمای بیمار ساختاریافته است، نه گزارش روایی مستقل؛ و STT چند ارائه‌دهنده دارد اما کیفیت روی داده‌ی مرجع سنجیده نشده است. ایمنی فعلی AI-first با اصل قطعیِ طرح اولیه تفاوت دارد. بیان این تفاوت‌ها به خواننده اجازه می‌دهد محصول را بر پایه‌ی آنچه دارد ارزیابی کند.

جدول ۷ تطبیق خروجی با هدف‌های طرح

هدف	وضعیت نسخه‌ی موجود	کار باقی‌مانده
هسته‌ی داده و API	موجود با مدل و migration	تقویت تراکنش و سیاست حذف فایل
اسناد و مقادیر منتخب	متن و تحلیل سند موجود	فیلدهای معتبر و provenance جزئی
Context و timeline	موجود و زنده	انتخاب مرتبط و اعلام truncation
مدل آماده و گفتگو	سه adapter متن و گفتگو	ارزیابی معنایی و بازتولید پاسخ
Citation	رکورد پرونده	پشتیبانی معنایی و بازیابی بیرونی
Speech-to-Text	سه مسیر provider	ارزیابی واقعی و metadata نتیجه
Prescription Safety	AI-first و قواعد مستقل	تطبیق ایمنی با طرح و مرجع معتبر
استقرار	Docker و Nginx پیکربندی شده	TLS و اجرای stack و backup
ارزیابی کاربر و بالینی	نتیجه‌ی تأییدشده موجود نیست	پروتکل داده‌ی مرجع و مرور انسانی

۱۲-۲ سهم مهندسی و حدود نتیجه

سهم اصلی حوزه‌ی عرفان در اتصال هسته‌ی داده، API و سرویس‌های هوشمند است. تفکیک قرارداد provider، ساخت Context مشترک، حفظ متن خام، ذخیره‌ی سؤال پیش از تماس AI و کنترل ID شواهد

نمونه‌هایی از تصمیم‌های قابل مشاهده‌اند. این تصمیم‌ها مسیر توسعه و رسیدگی به خطا را روشن‌تر می‌کنند. نتیجه‌ی آن‌ها یک نمونه‌ی نرم‌افزاری قابل بررسی است، نه یک سیستم پزشکی تأییدشده.

رابط بالینی امکان مصرف این قابلیت‌ها را فراهم می‌کند و شرح آن در گزارش برای کامل شدن مسیر کاربردی ضروری است. مسئولیت اصلی رابط مطابق پروپوزال با ایمان و معماری و یکپارچه‌سازی با هر دو دانشجو است. تاریخچه‌ی Git قرینه‌ی توسعه است و برای نسبت دادن تجربه‌ی شخصی یا مالکیت انحصاری کافی نیست. از این رو گزارش خاطره، دشواری شخصی یا ارزیابی کاربری ساختگی ارائه نمی‌کند.

نتیجه‌ی تازه‌ی آزمون منتخب نشان داد قراردادهای بررسی‌شده در شصت‌ویک مورد، پنجاه‌ونه موفق و دو مورد گرفتار نبود MinIO بودند. این نتیجه محدوده‌ی فنی دارد. هیچ داده‌ای برای ادعای کاهش خطای درمان، صحت تشخیص، بهبود پیامد بیمار یا زمان مرور پزشک در این بررسی فراهم نشده است. ارزش گزارش در جدا کردن این لایه‌ها و قابل پیگیری کردن شواهد پیاده‌سازی است.

۱۲-۳ اولویت‌های مهندسی

اولویت اول بستن شکاف‌های مسیر موجود است: دانلود مجوزدار در رابط، سازگاری سقف صوت با Nginx، نصب ابزارهای صوت در تصویر، اعمال واقعی proxy در STT و پاک‌سازی فایل موقت در تمام شکست‌ها. این کارها به قابلیت‌های تعریف‌شده مربوط‌اند و پیش از اضافه کردن سرویس جدید، قابلیت اعتماد نمونه را بهبود می‌دهند. هر اصلاح باید با تستی همراه باشد که خطای پیشین را واقعاً بازسازی می‌کند.

اولویت دوم مدیریت وضعیت و منشأ است. جدول کار پایدار، شناسه‌ی درخواست، retry کنترل‌شده و نسخه‌ی ورودی برای خروجی مشتق، مشکلات crash و تکرار را بهتر پوشش می‌دهند. برای چت باید provider، model، نسخه‌ی prompt و fingerprint زمینه ثبت شود؛ در صورت نیاز به snapshot، سیاست نگهداری و محرمانگی آن تعیین گردد. اطلاعات source نباید به URL موقتی یا log حساس وابسته بماند.

اولویت سوم امنیت عملیاتی است: TLS، secretهای معتبر، session امن، کنترل نرخ و بازبینی .log حذف بیمار باید به سیاست روشن فایل‌ها و backup متصل شود. مجوزهای سازمانی، مشاهده و ویرایش و ممیزی باید در صورت گسترش به چند مرکز جدا شوند. این اقدامات شرط توسعه‌ی واقعی‌اند، ولی هیچ‌یک به‌تنهایی گواهی انطباق یا تأیید پزشکی ایجاد نمی‌کند.

۱۲-۴ برنامه‌ی ارزیابی و توسعه‌ی علمی

برای اسناد، مجموعه‌ی مصنوعی دارای متن و فیلد مرجع باید ساخته شود و coverage هر صفحه، CER و صحت مقادیر جدا سنجیده شوند. برای صوت، صدای کنترل‌شده‌ی تک‌گوینده و چندگوینده با transcript مرجع و قواعد عادی‌سازی نیاز است. نتیجه‌ی Google و Speechmatics و OpenRouter باید با تنظیم یکسان و همراه اطلاعات provider ثبت شود. آزمون latency نیز با کیفیت متن جمع شود تا راهکار سریع ولی ناقص برتر شمرده نشود.

برای گفتگو، پرسش‌ها باید شامل داده‌ی کافی، داده‌ی غایب، داده‌ی متناقض و متن سند دستورگونه باشند. ارزیاب باید مشخص کند کدام ادعا به کدام رکورد متکی است و آیا شناسه‌ی معتبر واقعاً از جمله حمایت می‌کند. مقایسه‌ی بدون Context و همراه Context باید روی مجموعه‌ی ثابت و با روش امتیازدهی مشخص انجام شود. وجود prompt ایمنی یا Citation، نتیجه‌ی این آزمایش را از پیش تعیین نمی‌کند.

گسترش بازیابی پزشکی زمانی معنادار است که منبع، نسخه، تاریخ و شرایط استفاده‌ی آن تعیین شده باشد. می‌توان رابطی محدود برای یک منبع رسمی طراحی و کیفیت انتخاب و استناد را مستقل ارزیابی کرد. RAG دائمی یا پایگاه برداری تنها وقتی افزوده شود که ارزش آن نسبت به روش ساده نشان داده شود. در ایمنی دارو نیز بازگرداندن قواعد معتبر یا ترکیب کنترل‌شده‌ی قواعد و توضیح AI باید همراه پوشش و سنجش منفی کاذب باشد.

۱۲-۵ نتیجه‌ی نهایی

Clinical AI Copilot موجود، مسیرهای اصلی ثبت و مرور پرونده و استفاده‌ی کنترل‌شده از ابزارهای آماده را در یک نمونه‌ی تحت وب به هم متصل می‌کند. Backend زمینه و دسترسی و ماندگاری را مدیریت می‌کند و داده‌ی مشتق را از اصل فایل جدا نگه می‌دارد. گزارش حاضر نشان می‌دهد این مبنا قابل توسعه است، اما برای تحلیل بالینی معتبر، بازیابی پزشکی بیرونی و بهره‌برداری عملیاتی به ارزیابی و اصلاحات مشخص دیگری نیاز دارد.

منابع و مراجع

شیوه‌نامه‌ی ارائه‌شده روش استناد خاصی تعیین نکرده است؛ در این گزارش منابع با شماره‌ی یکتا معرفی می‌شوند. ارجاع‌های شماره‌دار مربوط به مبانی فنی‌اند و مسیر فایل و شناسه‌ی تابع، شواهد پیاده‌سازی پروژه را بیان می‌کنند. تاریخ دسترسی منابع وب ۱۴ اکتبر ۲۰۲۶ است. شواهد محلی مطابق نسخه‌ی بررسی‌شده‌اند و نام فایل به معنی نتیجه‌ی موفق آزمون نیست.

[۱] FastAPI. Background Tasks. مستندات رسمی؛ مبنا برای اجرای کار پس از ارسال پاسخ و تفاوت آن با صف مستقل.

https://fastapi.tiangolo.com/tutorial/background-tasks/

[۲] SQLAlchemy. Session Basics، مستندات نسخه‌ی ۲. مبنا برای مدیریت Session و تراکنش؛ انتخاب تنظیمات خاص از کد پروژه برداشت شده است.

https://docs.sqlalchemy.org/en/20/orm/session_basics.html

[۳] Next.js. Self-hosting. مستندات رسمی استقرار برنامه و خروجی runtime.

https://nextjs.org/docs/app/guides/self-hosting

[۴] PostgreSQL. Constraints، مستندات نسخه‌ی ۱۷. مبنا برای قیود یکتایی و خارجی و رفتار حذف.

https://www.postgresql.org/docs/17/ddl-constraints.html

[۵] Google AI for Developers. Gemini API text generation. مرجع قالب تولید متن؛ نام مدل‌های موجود در تنظیمات پروژه، تأیید دسترس‌پذیری روز مدل نیست.

https://ai.google.dev/gemini-api/docs/text-generation

[۶] OpenRouter. API overview. مرجع قرارداد درخواست و پاسخ API؛ نمونه‌ی کد پروژه مستقیماً بررسی شده است.

https://openrouter.ai/docs/api/reference/overview

[۷] Speechmatics. Batch speech-to-text quickstart. مرجع گردش job و دریافت transcript؛ کیفیت فارسی در این گزارش اندازه‌گیری نشده است.

https://docs.speechmatics.com/speech-to-text/batch/quickstart

[۸] pypdf. Extract Text from a PDF. مرجع تفاوت لایه‌ی متن، سند اسکن‌شده و OCR.

https://pypdf.readthedocs.io/en/stable/user/extract-text.html

[۹] Docker. Control startup and shutdown order in Compose. مرجع depends_on و سلامت سرویس؛ وجود پیکربندی با استقرار موفق فرق دارد.

https://docs.docker.com/compose/how-tos/startup-order/

[۱۰] OWASP. HTML5 Security Cheat Sheet. مرجع بحث دسترسی JavaScript به localStorage، خطر XSS و استفاده از cookie دارای HttpOnly.

https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html

[۱۱] پروپوزال پروژه، AI_Clinical_Copilot_Proposal_Task_Split_RTL_WORD_FIXED_V3.docx، بخش‌های ۲٫۲، ۱۰، ۱۱ و ۱۷. دانشگاه تهران، دانشکدگان فارابی، استاد راهنما دکتر کاظم فولادی قلعه؛ سال تحصیلی ۱۴۰۴–۱۴۰۵.

[۱۲] دانشگاه تهران، ساختار و قالب گزارش پروژه‌ی کارشناسی، BSc-project-report-format.pdf، سه صفحه؛ و قالب جلد BSc-project-report-cover-fa-1401.pdf، یک صفحه.

[۱۳] مخزن Clinical AI Copilot، نسخه‌ی کاری با HEAD برابر 56ed451 و تغییرات موجود هنگام بررسی. شواهد Backend در app، tests و alembic؛ شواهد رابط در frontend/src؛ استقرار در deploy و Dockerfileها.

[۱۴] گزارش‌های تسک‌های پروژه در docs، به‌ویژه Task 14، 16، 17، 18 و 19. این موارد مستندات داخلی و تاریخی‌اند و با اجرای تازه‌ی این گزارش تفکیک شده‌اند.

پیوست الف مرجع مسیرهای واقعی API

این پیوست از مسیرهای ثبت‌شده‌ی برنامه و قرارداد OpenAPI ساخته شده است. مسیر /health بدون نسخه و سایر منابع زیر پیشوند نسخه قرار دارند. جدول‌ها فعل، مسیر و نوع ورودی و پاسخ موفق را ثبت می‌کنند. وجود مسیر یا schema به معنی اجرای موفق آن در محیط فعلی نیست. پارامترهای path در آکولاد و پارامترهای query در شرح قرارداد مشخص‌اند. مقدار secret در این مرجع وارد نشده است.

الف‌۱ سلامت فرایند

این گروه فقط liveness را عرضه می‌کند. پاسخ status=ok تأیید ارتباط پایگاه و ذخیره‌ساز نیست. OpenAPI برای این مسیر نیز قرارداد پاسخ دارد.

جدول ۸ قراردادهای سلامت فرایند

فعل و مسیر	ورودی و پارامتر	پاسخ موفق
GET /api/v1/health	بدنه ندارد	200: HealthResponse
GET /health	بدنه ندارد	200: HealthResponse

الف‌۲ حساب و ورود

مسیرهای ثبت‌نام، ورود و درخواست و بررسی OTP ورودی عمومی دارند؛ me به Bearer نیاز دارد. پاسخ token فقط access token است. کد OTP و password واقعی در مثال‌ها یا این جدول درج نشده‌اند.

جدول ۹ قراردادهای حساب و ورود

فعل و مسیر	ورودی و پارامتر	پاسخ موفق
POST /api/v1/auth/register	application/json: UserRegister	201: UserResponse
POST /api/v1/auth/login	application/json: UserLogin	200: TokenResponse
GET /api/v1/auth/me	بدنه ندارد	200: UserResponse
POST /api/v1/auth/request-otp	application/json: OtpRequestPayload	202: object
POST /api/v1/auth/verify-otp	application/json: OtpVerifyPayload	200: TokenResponse

الف‌۳ کاتالوگ مدل

پاسخ این مسیر فقط شناسه و نام provider و مدل را دارد. کاتالوگ Gemini ایستا و OpenRouter از سرویس models گرفته می‌شود. نبود یا شکست کشف، مدل ساختگی تولید نمی‌کند؛ درستی نام و آماده بودن credential برای completion باید جدا بررسی شود.

جدول ۱۰ قراردادهای کاتالوگ مدل

فعل و مسیر	ورودی و پارامتر	پاسخ موفق
GET /api/v1/ai/providers	بدنه ندارد	200: object

الف‌۴ پرونده بیمار

CRUD مشخصات از overview، context و timeline جداست. فهرست بر اساس نقش محدود می‌شود و q و limit و offset دارد. overview فقط داروهای active و پنج آزمایش و یادداشت اخیر را می‌دهد، ولی Context گروه‌های بیشتری با سقف مستقل دارد.

جدول ۱۱ قراردادهای پرونده بیمار

فعل و مسیر	ورودی و پارامتر	پاسخ موفق
POST /api/v1/patients	application/json: PatientCreate	201: PatientResponse
GET /api/v1/patients	بدنه ندارد q (query): null / string limit (query): integer offset (query): integer	200: PatientListResponse
GET /api/v1/patients/{patient_id}	بدنه ندارد patient_id (path): integer	200: PatientResponse
PATCH /api/v1/patients/{patient_id}	application/json: PatientUpdate patient_id (path): integer	200: PatientResponse
DELETE /api/v1/patients/{patient_id}	بدنه ندارد patient_id (path): integer	204: بدون بدنه‌ی JSON تعریف‌شده
GET /api/v1/patients/{patient_id}/context	بدنه ندارد patient_id (path): integer	200: PatientContextResponse

فعل و مسیر	ورودی و پارامتر	پاسخ موفق
GET /api/v1/patients/{patient_id}/timeline	بدنه ندارد patient_id (path): integer	200: PatientTimelineResponse
GET /api/v1/patients/{patient_id}/overview	بدنه ندارد patient_id (path): integer	200: PatientOverviewResponse

الف‌ـ۵ مشکلات بالینی

این گروه نام آزاد مشکل، وضعیت و تاریخ شروع و رفع را نگه می‌دارد. ترتیب فهرست active-first است. مسیرهای مستقل ویرایش و حذف، patient_id رکورد واقعی را برای کنترل مجوز استفاده می‌کنند و تشخیص خودکار در این عملیات انجام نمی‌شود.

جدول ۱۲ قراردادهای مشکلات بالینی

فعل و مسیر	ورودی و پارامتر	پاسخ موفق
POST /api/v1/patients/{patient_id}/conditions	application/json: ConditionCreate patient_id (path): integer	201: ConditionResponse
GET /api/v1/patients/{patient_id}/conditions	بدنه ندارد patient_id (path): integer	200: list[ConditionResponse]
GET /api/v1/conditions/{condition_id}	بدنه ندارد condition_id (path): integer	200: ConditionResponse
PATCH /api/v1/conditions/{condition_id}	application/json: ConditionUpdate condition_id (path): integer	200: ConditionResponse
DELETE /api/v1/conditions/{condition_id}	بدنه ندارد condition_id (path): integer	204: بدون بدنه‌ی JSON تعریف‌شده

الف‌ـ۶ حساسیت

نام عامل، واکنش و شدت حساسیت با schema بالینی ثبت می‌شوند. فهرست شدت بیشتر را مقدم می‌گذارد؛ این ترتیب از enum ذخیره‌شده می‌آید. نبود مورد ثبت‌شده، تأیید نداشتن حساسیت نیست و معنای آن باید در Context حفظ شود.

جدول ۱۳ قراردادهای حساسیت

فعل و مسیر	ورودی و پارامتر	پاسخ موفق
POST /api/v1/patients/{patient_id}/allergies	application/json: AllergyCreate patient_id (path): integer	201: AllergyResponse

فعل و مسیر	ورودی و پارامتر	پاسخ موفق
GET /api/v1/patients/{patient_id}/allergies	بدنه ندارد patient_id (path): integer	200: list[AllergyResponse]
GET /api/v1/allergies/{allergy_id}	بدنه ندارد allergy_id (path): integer	200: AllergyResponse
PATCH /api/v1/allergies/{allergy_id}	application/json: AllergyUpdate allergy_id (path): integer	200: AllergyResponse
DELETE /api/v1/allergies/{allergy_id}	بدنه ندارد allergy_id (path): integer	204: بدون بدنه‌ی JSON تعریف‌شده

الف‌۷ کاتالوگ و مصرف دارو

کاتالوگ Medication با مصرف PatientMedication متفاوت است. تابع get_or_create_medication مدخل نام را پیدا یا flush می‌کند و تراکنش نسخه در caller کامل می‌شود. status و تاریخ‌ها درباره‌ی سابقه‌ی بیمارند، نه مشخصات عمومی دارو.

جدول ۱۴ قراردادهای کاتالوگ و مصرف دارو

فعل و مسیر	ورودی و پارامتر	پاسخ موفق
POST /api/v1/patients/{patient_id}/medications	application/json: PatientMedicationCreate patient_id (path): integer	201: PatientMedicationResponse
GET /api/v1/patients/{patient_id}/medications	بدنه ندارد patient_id (path): integer status (query): MedicationStatus / null	200: list[PatientMedicationResponse]
GET /api/v1/patient-medications/{patient_medication_id}	بدنه ندارد patient_medication_id (path): integer	200: PatientMedicationResponse
PATCH /api/v1/patient-medications/{patient_medication_id}	application/json: PatientMedicationUpdate patient_medication_id (path): integer	200: PatientMedicationResponse
DELETE /api/v1/patient-medications/{patient_medication_id}	بدنه ندارد patient_medication_id (path): integer	204: بدون بدنه‌ی JSON تعریف‌شده

الف‌۸ اسناد

آپلود multipart است، extract کار پس‌زمینه می‌پذیرد و analyze همگام به AI تماس می‌زند. download پاسخ redirect دارد و کنترل مجوز پیش از presign اجرا می‌شود. مدل پاسخ متادیتا را از فایل اولیه جدا می‌کند.

جدول ۱۵ قراردادهای اسناد

فعل و مسیر	ورودی و پارامتر	پاسخ موفق
POST /api/v1/patients/{patient_id}/documents	multipart/form-data: Body_upload_document_api_v1_patients__patient_id__documents_post patient_id (path): integer	201: DocumentResponse
GET /api/v1/patients/{patient_id}/documents	بدنه ندارد patient_id (path): integer	200: list[DocumentResponse]
GET /api/v1/documents/{document_id}	بدنه ندارد document_id (path): integer	200: DocumentResponse
PATCH /api/v1/documents/{document_id}	application/json: DocumentUpdate document_id (path): integer	200: DocumentResponse
DELETE /api/v1/documents/{document_id}	بدنه ندارد document_id (path): integer	204: بدون بدنه‌ی JSON تعریف‌شده
POST /api/v1/documents/{document_id}/extract	بدنه ندارد document_id (path): integer	202: DocumentResponse
POST /api/v1/documents/{document_id}/analyze	بدنه ندارد document_id (path): integer	200: DocumentResponse
GET /api/v1/documents/{document_id}/download	بدنه ندارد document_id (path): integer	307: بدون بدنه‌ی JSON تعریف‌شده

الف‌۹ صوت و یادداشت

بارگذاری و رونویسی با یادداشت تأییدشده متفاوت‌اند. درخواست note متن بازبینی‌شده را می‌سازد و download به اصل صوت مربوط است. وضعیت transcription و cleanup باید مستقل نمایش داده شود.

جدول ۱۶ قراردادهای صوت و یادداشت

فعل و مسیر	ورودی و پارامتر	پاسخ موفق
POST /api/v1/patients/{patient_id}/voice-notes	multipart/form-data: Body_upload_voice_note_api_v1_patients__patient_id__voice_notes_post patient_id (path): integer	201: VoiceNoteResponse
GET /api/v1/patients/{patient_id}/voice-notes	بدنه ندارد patient_id (path): integer	200: list[VoiceNoteResponse]
GET /api/v1/voice-notes/{voice_note_id}	بدنه ندارد voice_note_id (path): integer	200: VoiceNoteResponse
PATCH /api/v1/voice-notes/{voice_note_id}	application/json: VoiceNoteUpdate voice_note_id (path): integer	200: VoiceNoteResponse
DELETE /api/v1/voice-notes/{voice_note_id}	بدنه ندارد voice_note_id (path): integer	204: بدون بدنه‌ی JSON تعریف‌شده
GET /api/v1/voice-notes/{voice_note_id}/download	بدنه ندارد voice_note_id (path): integer	307: بدون بدنه‌ی JSON تعریف‌شده
POST /api/v1/voice-notes/{voice_note_id}/transcribe	بدنه ندارد voice_note_id (path): integer	202: VoiceNoteResponse
POST /api/v1/voice-notes/{voice_note_id}/note	application/json: CreateNoteFromTranscript voice_note_id (path): integer	201: ClinicalNoteResponse

الف‌ـ۱۰ گفتگوی بدون ماندگاری

مسیر اولیه‌ی گفتگو سؤال و تاریخچه‌ی client را می‌گیرد و پاسخ را ذخیره نمی‌کند. فلگ ایمنی فقط prompt را عوض می‌کند. provider و model شناسه‌اند؛ URL و کلید خارجی از client پذیرفته نمی‌شوند و شواهد ماندگار در این مسیر اجرا نمی‌شود.

جدول ۱۷ قراردادهای گفتگوی بدون ماندگاری

فعل و مسیر	ورودی و پارامتر	پاسخ موفق
POST /api/v1/patients/{patient_id}/chat	application/json: ChatRequest patient_id (path): integer	200: ChatResponse

الف‌۱۱ قواعد دارویی

endpoint قدیمی قواعد محلی را با داروهای active و حساسیت مقایسه می‌کند. candidate_covered وضعیت نگاشت نام را نشان می‌دهد و explain فقط توضیح اختیاری هشدارهای موجود است. این مسیر در تحلیل AI-first فراخوانی نمی‌شود.

جدول ۱۸ قراردادهای قواعد دارویی

فعل و مسیر	ورودی و پارامتر	پاسخ موفق
POST /api/v1/patients/{patient_id}/medications/check	application/json: MedicationSafetyCheckRequest patient_id (path): integer	200: MedicationSafetyCheckResponse

الف‌۱۲ تحلیل دارویی AI

این endpoint کاندیدها را با Context به مدل پیش‌فرض می‌فرستد و JSON را از نظر نوع و فیلدهای اصلی کنترل می‌کند. هشدار قطعی rule_id ندارد و نتیجه‌ی آن در پایگاه نگهداری نمی‌شود. شکست تحلیل مانع ثبت دارو نیست.

جدول ۱۹ قراردادهای تحلیل دارویی AI

فعل و مسیر	ورودی و پارامتر	پاسخ موفق
POST /api/v1/patients/{patient_id}/medications/ai-safety-check	application/json: AiMedicationSafetyCheckRequest patient_id (path): integer	200: AiMedicationSafetyCheckResponse

الف‌۱۳ گفتگوی ماندگار

این گروه گفتگو و پیام‌ها را در پایگاه نگه می‌دارد. پیام user پیش از تماس مدل commit می‌شود و assistant تنها پس از پاسخ ساخته می‌شود. history و Context جدا هستند و شناسه‌های evidence با index همان بیمار کنترل می‌شوند.

جدول ۲۰ قراردادهای گفتگوی ماندگار

فعل و مسیر	ورودی و پارامتر	پاسخ موفق
POST /api/v1/patients/{patient_id}/conversations	application/json: ConversationCreate patient_id (path): integer	201: ConversationResponse
GET /api/v1/patients/{patient_id}/conversations	بدنه ندارد patient_id (path): integer	200: list[ConversationResponse]

فعل و مسیر	ورودی و پارامتر	پاسخ موفق
GET /api/v1/conversations/{conversation_id}	بدنه ندارد conversation_id (path): integer	200: ConversationDetailResponse
DELETE /api/v1/conversations/{conversation_id}	بدنه ندارد conversation_id (path): integer	204: بدون بدنه‌ی JSON تعریف‌شده
POST /api/v1/conversations/{conversation_id}/messages	application/json: ConversationMessageCreate conversation_id (path): integer	200: SendMessageResponse

الف‌۱۴ عامل آزمایشی

این مسیر stateless و آزمایشی است. پس از احراز هویت و اجازه‌ی بیمار، عامل از ابزارهای فقط‌خواندنی استفاده می‌کند. ساختار tools_used شفافیت فنی می‌دهد و نتیجه‌ی آن در گفتگو ذخیره نمی‌شود.

جدول ۲۱ قراردادهای عامل آزمایشی

فعل و مسیر	ورودی و پارامتر	پاسخ موفق
POST /api/v1/patients/{patient_id}/agent/chat	application/json: AgentChatRequest patient_id (path): integer	200: AgentChatResponse

مسیرهای این مرجع فقط API ثبت‌شده‌اند؛ مسیر CRUD مستقل برای LabResult و یادداشت عمومی در آن وجود ندارد. routes خودکار docs/ و openapi.json/ فهرست محصول بالینی محسوب نشده‌اند. تعداد عملیات فهرست‌شده: ۵۶.

پیوست ب فرهنگ داده و روابط پایگاه

جدول‌های زیر مستقیماً از metadata مدل‌های SQLAlchemy استخراج شده‌اند. نوع نمایشی ORM، nullable، PK و FK از مدل برداشت شده‌اند. نوع عمومی DATETIME با نوع بومی PostgreSQL متفاوت است؛ ستون DateTime دارای timezone در PostgreSQL به timestamp with time zone نگاشت می‌شود و enum بومی نام نوع خود را دارد. مقادیر enum در جدول جدا آمده‌اند. هیچ مقدار واقعی بیمار یا حساب وجود ندارد. created_at و updated_at از TimestampMixin می‌آیند و Nullable با رشته‌ی خالی یکسان نیست.

بـ۱ کاتالوگ دارو

`table: medications`

نام یکتا و generic_name اختیاری برای استفاده‌ی مشترک میان بیماران وجود دارد. این فهرست محدود اطلاعات دارو است؛ اطلاعات جامع تداخل، دوز معتبر و کد RxNorm در این مدل تعریف نشده‌اند. مدخل کاتالوگ با سابقه‌ی مصرف بیمار متفاوت است.

جدول ۲۲ فیلدهای medications

نام فیلد	نوع ORM	قید و رابطه	تفسیر
id	INTEGER	PK NOT NULL	کلید اصلی رکورد؛ برای رابطه و ردیابی استفاده می‌شود.
name	VARCHAR(200)	UNIQUE NOT NULL	نام متنی منبع؛ در کاتالوگ دارو قید یکتا دارد.
generic_name	VARCHAR(200)	NULL	نام ژنریک اختیاری دارو؛ کد استاندارد محسوب نمی‌شود.
created_at	DATETIME	NOT NULL	زمان ایجاد با timezone؛ fallback بسیاری از تاریخ‌ها.
updated_at	DATETIME	NOT NULL	زمان آخرین تغییر؛ در فهرست گفتگو ترتیب فعالیت می‌دهد.

بـ۲ درخواست کد ورود

`table: otp_requests`

کد به‌صورت hash نگهداری می‌شود، expires_at انقضا و attempts_left تعداد فرصت باقی‌مانده را ثبت می‌کند. consumed_at مصرف موفق را مشخص می‌کند. مقادیر کد واقعی در گزارش نمایش داده نمی‌شوند و تحلیل هم‌زمانی مصرف باید جدا آزموده شود.

جدول ۲۳ فیلدهای otp_requests

نام فیلد	نوع ORM	قید و رابطه	تفسیر
id	INTEGER	PK NOT NULL	کلید اصلی رکورد؛ برای رابطه و ردیابی استفاده می‌شود.
phone	VARCHAR(20)	INDEX NOT NULL	شماره‌ی تماس؛ در User برای ورود OTP نیز استفاده می‌شود.
code_hash	VARCHAR(64)	NOT NULL	hash کد OTP؛ متن ساده‌ی کد در جدول نیست.
expires_at	DATETIME	NOT NULL	زمان انقضای کد ورود.
consumed_at	DATETIME	NULL	زمان مصرف موفق؛ مانع استفاده‌ی دوباره در مسیر عادی.
attempts_left	INTEGER	NOT NULL	فرصت‌های بررسی باقیمانده؛ قبل از تطابق کاهش می‌یابد.
created_at	DATETIME	NOT NULL	زمان ایجاد با timezone؛ fallback بسیاری از تاریخ‌ها.
updated_at	DATETIME	NOT NULL	زمان آخرین تغییر؛ در فهرست گفتگو ترتیب فعالیت می‌دهد.

ب‌ـ۳ مشخصات پایه‌ی بیمار

`table: patients`

این جدول ریشه‌ی روابط بالینی است و اطلاعات هویتی را نگه می‌دارد. داده‌ی تخصصی در جدول‌های وابسته قرار دارد. national_code اختیاری و یکتاست و نبود آن با نبود بیمار تفاوت دارد. حذف رکورد با قواعد CASCADE در پایگاه روی فرزندان اثر می‌گذارد؛ این رفتار شامل حذف بایت‌های MinIO نیست.

جدول ۲۴ فیلدهای patients

نام فیلد	نوع ORM	قید و رابطه	تفسیر
id	INTEGER	PK NOT NULL	کلید اصلی رکورد؛ برای رابطه و ردیابی استفاده می‌شود.

نام فیلد	نوع ORM	قید و رابطه	تفسیر
first_name	VARCHAR(100)	NOT NULL	نام بیمار با طول محدود؛ داده‌ی شناسایی‌کننده.
last_name	VARCHAR(100)	NOT NULL	نام خانوادگی بیمار؛ مبنای جست‌وجوی فهرست.
national_code	VARCHAR(10)	UNIQUE NULL	کد ده‌رقمی اختیاری و یکتا؛ قید ظاهری، بدون تأیید هویت.
date_of_birth	DATE	NULL	تاریخ تولد اختیاری؛ schema تاریخ گذشته می‌خواهد.
gender	VARCHAR(7)	NOT NULL	مقدار enum جنسیت با حالت unknown.
phone	VARCHAR(20)	NULL	شماره‌ی تماس؛ در User برای ورود OTP نیز استفاده می‌شود.
address	TEXT	NULL	نشانی متنی اختیاری؛ هنگام حداقل‌سازی داده باید بازبینی شود.
created_at	DATETIME	NOT NULL	زمان ایجاد با timezone؛ fallback بسیاری از تاریخ‌ها.
updated_at	DATETIME	NOT NULL	زمان آخرین تغییر؛ در فهرست گفتگو ترتیب فعالیت می‌دهد.

جدول ۲۵ مقادیر enum در patients

فیلد	مقادیر ذخیره‌شده
gender	male / female / other / unknown

بـ۴ حساب برنامه

`table: users`

نقش doctor یا admin و is_active برای کنترل ورود و دسترسی استفاده می‌شود. email و phone یکتا هستند و password_hash می‌تواند برای حساب OTP-only خالی باشد. hash در پاسخ عمومی قرار نمی‌گیرد. داشتن حساب doctor به معنای بررسی رسمی مدرک پزشکی نیست.

جدول ۲۶ فیلدهای users

نام فیلد	نوع ORM	قید و رابطه	تفسیر
id	INTEGER	PK NOT NULL	کلید اصلی رکورد؛ برای رابطه و ردیابی استفاده می‌شود.
email	VARCHAR(320)	UNIQUE INDEX NOT NULL	شناسه‌ی ایمیل حساب؛ برای ورود lower می‌شود.
phone	VARCHAR(20)	UNIQUE INDEX NULL	شماره‌ی تماس؛ در User برای ورود OTP نیز استفاده می‌شود.
password_hash	VARCHAR(200)	NULL	hash bcrypt؛ مقدار واقعی و hash در گزارش منتشر نمی‌شود.
full_name	VARCHAR(200)	NOT NULL	نام نمایشی حساب؛ بازبینی صلاحیت حرفه‌ای نیست.
role	VARCHAR(6)	NOT NULL	نوع نویسنده‌ی پیام یا نقش حساب، بسته به جدول.
is_active	BOOLEAN	NOT NULL	وضعیت فعال بودن حساب؛ در هر احراز هویت بررسی می‌شود.
created_at	DATETIME	NOT NULL	زمان ایجاد با timezone؛ fallback بسیاری از تاریخ‌ها.
updated_at	DATETIME	NOT NULL	زمان آخرین تغییر؛ در فهرست گفتگو ترتیب فعالیت می‌دهد.

جدول ۲۷ مقادیر enum در users

فیلد	مقادیر ذخیره‌شده
role	doctor / admin

بـ۵ حساسیت‌ها

`table: allergies`

allergen موضوع حساسیت و reaction شرح واکنش ثبت‌شده است. شدت از enum انتخاب می‌شود. نبود رکورد، تأیید نبود حساسیت نیست. ترتیب severe-first برای مرور تعریف شده و سیستم این شدت را از آزمون پزشکی مستقلی استنتاج نمی‌کند.

جدول ۲۸ فیلدهای allergies

نام فیلد	نوع ORM	قید و رابطه	تفسیر
id	INTEGER	PK NOT NULL	کلید اصلی رکورد؛ برای رابطه و ردیابی استفاده می‌شود.
patient_id	INTEGER	INDEX FK patients.id / CASCADE NOT NULL	کلید خارجی بیمار؛ در queryهای پرونده فیلتر می‌شود.
allergen	VARCHAR(200)	NOT NULL	نام حساسیت ثبت‌شده؛ ورودی قواعد و Context.
reaction	VARCHAR(300)	NULL	شرح واکنش بیمار به عامل؛ اختیاری.
severity	VARCHAR(8)	NOT NULL	شدت enum ثبت‌شده؛ در ترتیب severe-first مصرف می‌شود.
created_at	DATETIME	NOT NULL	زمان ایجاد با timezone؛ fallback بسیاری از تاریخ‌ها.
updated_at	DATETIME	NOT NULL	زمان آخرین تغییر؛ در فهرست گفتگو ترتیب فعالیت می‌دهد.

جدول ۲۹ مقادیر enum در allergies

فیلد	مقادیر ذخیره‌شده
severity	mild / moderate / severe / unknown

بـ۶ رویداد ثبت‌شده

`table: clinical_events`

source_type و source_id اشاره‌ی چندریختی بدون FK به منبع دارند. این انعطاف امکان انواع رویداد را فراهم می‌کند ولی تمامیت منبع باید در کد کنترل شود. تایم‌لاین اصلی از رکوردهای دیگر مشتق است و این جدول دفتر کامل تمام عملیات و کاربران نیست.

جدول ۳۰ فیلدهای clinical_events

نام فیلد	نوع ORM	قید و رابطه	تفسیر
id	INTEGER	PK NOT NULL	کلید اصلی رکورد؛ برای رابطه و ردیابی استفاده می‌شود.

نام فیلد	نوع ORM	قید و رابطه	تفسیر
patient_id	INTEGER	FK patients.id / CASCADE NOT NULL	کلید خارجی بیمار؛ در queryهای پرونده فیلتر می‌شود.
event_type	VARCHAR(50)	NOT NULL	نوع باز رویداد؛ مجموعه‌ی بسته‌ی enum نیست.
event_date	DATETIME	NOT NULL	زمان رویداد ثبت‌شده در ClinicalEvent.
title	VARCHAR(300)	NOT NULL	عنوان خوانا برای نمایش منبع یا گفتگو.
description	TEXT	NULL	شرح تکمیلی رویداد یا منبع.
source_type	VARCHAR(30)	NULL	نوع رکورد مرجع؛ در ClinicalEvent بدون FK چندجدولی.
source_id	INTEGER	NULL	شناسه‌ی رکورد مرجع رویداد؛ به‌تنهایی مجوز نیست.
created_at	DATETIME	NOT NULL	زمان ایجاد با timezone؛ fallback بسیاری از تاریخ‌ها.
updated_at	DATETIME	NOT NULL	زمان آخرین تغییر؛ در فهرست گفتگو ترتیب فعالیت می‌دهد.

ب‌ـ۷ مشکلات بالینی

`table: conditions`

نام مشکل متن آزاد است و status و تاریخ‌های شروع و رفع، حالت و زمان را ثبت می‌کنند. این جدول کاتالوگ استاندارد تشخیص نیست. داشتن نام بیماری در رکورد به معنی تشخیص خودکار توسط برنامه نیست و برچسب active از ورودی ویرایش‌شده‌ی کاربر می‌آید.

جدول ۳۱ فیلدهای conditions

نام فیلد	نوع ORM	قید و رابطه	تفسیر
id	INTEGER	PK NOT NULL	کلید اصلی رکورد؛ برای رابطه و ردیابی استفاده می‌شود.
patient_id	INTEGER	INDEX FK patients.id / CASCADE NOT NULL	کلید خارجی بیمار؛ در queryهای پرونده فیلتر می‌شود.
name	VARCHAR(200)	NOT NULL	نام متنی منبع؛ در کاتالوگ دارو قید یکتا دارد.

نام فیلد	نوع ORM	قید و رابطه	تفسیر
status	VARCHAR(8)	NOT NULL	وضعیت رکورد؛ در ترتیب و انتخاب داروهای active مؤثر است.
onset_date	DATE	NULL	تاریخ شروع مشکل؛ منبع تاریخ رویداد آغاز.
resolved_date	DATE	NULL	تاریخ رفع مشکل؛ در صورت وجود رویداد جدا می‌سازد.
notes	TEXT	NULL	توضیح تکمیلی متن آزاد؛ نتیجه‌ی استنتاج خودکار نیست.
created_at	DATETIME	NOT NULL	زمان ایجاد با timezone؛ fallback بسیاری از تاریخ‌ها.
updated_at	DATETIME	NOT NULL	زمان آخرین تغییر؛ در فهرست گفتگو ترتیب فعالیت می‌دهد.

جدول ۳۲ مقادیر enum در conditions

فیلد	مقادیر ذخیره‌شده
status	active / resolved

بـ۸ گفتگوی ماندگار

`table: conversations`

هر گفتگو بیمار و عنوان و زمان‌ها دارد. عنوان نخستین سؤال به‌صورت قطعی مشتق می‌شود مگر کاربر عنوان مشخص داده باشد. context snapshot یا model در این جدول وجود ندارد و داده‌ی پرونده هنگام هر سؤال دوباره خوانده می‌شود.

جدول ۳۳ فیلدهای conversations

نام فیلد	نوع ORM	قید و رابطه	تفسیر
id	INTEGER	PK NOT NULL	کلید اصلی رکورد؛ برای رابطه و ردیابی استفاده می‌شود.
patient_id	INTEGER	INDEX FK patients.id / CASCADE NOT NULL	کلید خارجی بیمار؛ در queryهای پرونده فیلتر می‌شود.
title	VARCHAR(300)	NOT NULL	عنوان خوانا برای نمایش منبع یا گفتگو.

نام فیلد	نوع ORM	قید و رابطه	تفسیر
created_at	DATETIME	NOT NULL	زمان ایجاد با timezone؛ fallback بسیاری از تاریخ‌ها.
updated_at	DATETIME	NOT NULL	زمان آخرین تغییر؛ در فهرست گفتگو ترتیب فعالیت می‌دهد.

بـ۹ متادیتا و متن سند

`table: documents`

storage_key به فایل اولیه اشاره دارد و extracted_text و ai_summary خروجی مشتق‌اند. وضعیت استخراج و تحلیل مستقل‌اند. ستون جدا برای روش OCR، confidence، نسخه‌ی مدل تحلیل یا زمان پایان استخراج در این مدل تعریف نشده است؛ جزئیات پردازش بیشتر در log دیده می‌شود.

جدول ۳۴ فیلدهای documents

نام فیلد	نوع ORM	قید و رابطه	تفسیر
id	INTEGER	PK NOT NULL	کلید اصلی رکورد؛ برای رابطه و ردیابی استفاده می‌شود.
patient_id	INTEGER	INDEX FK patients.id / CASCADE NOT NULL	کلید خارجی بیمار؛ در queryهای پرونده فیلتر می‌شود.
title	VARCHAR(300)	NOT NULL	عنوان خوانا برای نمایش منبع یا گفتگو.
doc_type	VARCHAR(17)	NOT NULL	نوع سند از enum؛ نوع بالینی با MIME متفاوت است.
storage_key	VARCHAR(500)	UNIQUE NOT NULL	کلید داخلی شیء؛ نشانی عمومی و مجوز مستقل نیست.
file_name	VARCHAR(300)	NOT NULL	نام اولیه‌ی فایل برای نمایش؛ کلید ذخیره جدا ساخته می‌شود.
mime_type	VARCHAR(100)	NOT NULL	MIME اعلامی فایل؛ امضای بایت جدا بررسی نشده است.
file_size_bytes	INTEGER	NOT NULL	اندازه‌ی بایت خوانده‌شده در upload.
extraction_status	VARCHAR(10)	NOT NULL	چرخه‌ی استخراج: pending، processing، completed یا failed.

نام فیلد	نوع ORM	قید و رابطه	تفسیر
extracted_text	TEXT	NULL	متن مشتق parser/OCR؛ فایل اصلی را جایگزین نمی‌کند.
analysis_status	VARCHAR(9)	NOT NULL	وضعیت تحلیل AI سند؛ مستقل از استخراج.
ai_summary	TEXT	NULL	خلاصه‌ی متنی AI؛ مقادیر ساختاریافته‌ی آزمایش نیست.
created_at	DATETIME	NOT NULL	زمان ایجاد با timezone؛ fallback بسیاری از تاریخ‌ها.
updated_at	DATETIME	NOT NULL	زمان آخرین تغییر؛ در فهرست گفتگو ترتیب فعالیت می‌دهد.

جدول ۳۵ مقادیر enum در documents

فیلد	مقادیر ذخیره‌شده
doc_type	lab_report / imaging_report / discharge_summary / medical_note / other
extraction_status	pending / processing / completed / failed
analysis_status	pending / completed / failed

ب‌ـ۱۰ سابقه‌ی مصرف دارو

`table: patient_medications`

رکورد رابطه‌ی بیمار و مدخل کاتالوگ همراه dosage، frequency و بازه‌ی تاریخ است. status تعیین می‌کند نسخه در overview دیده شود یا در تاریخچه بماند. رشته‌ی دوز به‌تنهایی واحد عددی قابل محاسبه یا دستور تجویز معتبر را تضمین نمی‌کند.

جدول ۳۶ فیلدهای patient_medications

نام فیلد	نوع ORM	قید و رابطه	تفسیر
id	INTEGER	PK NOT NULL	کلید اصلی رکورد؛ برای رابطه و ردیابی استفاده می‌شود.
patient_id	INTEGER	INDEX FK patients.id / CASCADE NOT NULL	کلید خارجی بیمار؛ در queryهای پرونده فیلتر می‌شود.

نام فیلد	نوع ORM	قید و رابطه	تفسیر
medication_id	INTEGER	INDEX FK medications.id / NO ACTION NOT NULL	پیوند نسخه‌ی بیمار به کاتالوگ دارو.
dosage	VARCHAR(100)	NULL	شرح دوز به‌صورت متن؛ محاسبه‌ی مستقل دوز انجام نمی‌شود.
frequency	VARCHAR(100)	NULL	تواتر مصرف به‌صورت متن آزاد.
status	VARCHAR(9)	NOT NULL	وضعیت رکورد؛ در ترتیب و انتخاب داروهای active مؤثر است.
start_date	DATE	NULL	تاریخ شروع مصرف؛ نبود آن با created_at جایگزین می‌شود.
end_date	DATE	NULL	تاریخ پایان مصرف؛ نمایش توقف به وضعیت نیز وابسته است.
notes	TEXT	NULL	توضیح تکمیلی متن آزاد؛ نتیجه‌ی استنتاج خودکار نیست.
created_at	DATETIME	NOT NULL	زمان ایجاد با timezone؛ fallback بسیاری از تاریخ‌ها.
updated_at	DATETIME	NOT NULL	زمان آخرین تغییر؛ در فهرست گفتگو ترتیب فعالیت می‌دهد.

جدول ۳۷ مقادیر enum در patient_medications

فیلد	مقادیر ذخیره‌شده
status	active / stopped / completed

بـ۱۱ انتساب دسترسی

`table: patient_user_access`

زوج بیمار و کاربر با قید یکتایی ثبت می‌شود. admin در بررسی دسترسی از این جدول عبور می‌کند و doctor به وجود زوج نیاز دارد. سطح مجوز مشاهده و ویرایش یا مرکز درمانی در این مدل جدا تعریف نشده است.

جدول ۳۸ فیلدهای patient_user_access

نام فیلد	نوع ORM	قید و رابطه	تفسیر
id	INTEGER	PK NOT NULL	کلید اصلی رکورد؛ برای رابطه و ردیابی استفاده می‌شود.
patient_id	INTEGER	INDEX FK patients.id / CASCADE NOT NULL	کلید خارجی بیمار؛ در queryهای پرونده فیلتر می‌شود.
user_id	INTEGER	INDEX FK users.id / CASCADE NOT NULL	کلید خارجی حساب دارای انتساب.
created_at	DATETIME	NOT NULL	زمان ایجاد با timezone؛ fallback بسیاری از تاریخ‌ها.
updated_at	DATETIME	NOT NULL	زمان آخرین تغییر؛ در فهرست گفتگو ترتیب فعالیت می‌دهد.

بـ۱۲ ضبط و رونوشت صوت

`table: voice_notes`

audio_storage_key فایل اولیه، transcript خروجی خام و cleaned_note متن مشتق مدل است. وضعیت رونویسی و پاک‌سازی جداست. processed_at پایان تلاش را بیان می‌کند و error_message خطای آخر را نگه می‌دارد. مدت می‌تواند از فرم کاربر آمده باشد و لزوماً محاسبه‌ی مستقل فایل نیست.

جدول ۳۹ فیلدهای voice_notes

نام فیلد	نوع ORM	قید و رابطه	تفسیر
id	INTEGER	PK NOT NULL	کلید اصلی رکورد؛ برای رابطه و ردیابی استفاده می‌شود.
patient_id	INTEGER	INDEX FK patients.id / CASCADE NOT NULL	کلید خارجی بیمار؛ در queryهای پرونده فیلتر می‌شود.
title	VARCHAR(300)	NULL	عنوان خوانا برای نمایش منبع یا گفتگو.
conversation_type	VARCHAR(14)	NOT NULL	physician_note یا doctor_patient؛ کنترل درخواست diarize.
audio_storage_key	VARCHAR(500)	UNIQUE NOT NULL	پیوند به اصل صوت در پوشه‌ی voice.

نام فیلد	نوع ORM	قید و رابطه	تفسیر
duration_seconds	INTEGER	NULL	مدت اعلامی ضبط؛ ممکن است از client آمده باشد.
language	VARCHAR(10)	NOT NULL	کد زبان STT؛ پیش‌فرض .fa
transcription_status	VARCHAR(10)	NOT NULL	وضعیت تلاش رونویسی مستقل از .cleanup
transcript	TEXT	NULL	متن خام provider؛ ویرایش کاربر آن را بازنویسی نمی‌کند.
cleanup_status	VARCHAR(9)	NOT NULL	وضعیت قالب‌بندی اختیاری .AI
cleaned_note	TEXT	NULL	متن قالب‌بندی‌شده‌ی مشتق؛ تأیید یادداشت جداست.
error_message	TEXT	NULL	شرح خطای آخر رونویسی با سقف ثبت‌شده در سرویس.
processed_at	DATETIME	NULL	زمان پایان تلاش پردازش؛ .timezone-aware
created_at	DATETIME	NOT NULL	زمان ایجاد با timezone؛ fallback بسیاری از تاریخ‌ها.
updated_at	DATETIME	NOT NULL	زمان آخرین تغییر؛ در فهرست گفتگو ترتیب فعالیت می‌دهد.

جدول ۴۰ مقادیر enum در voice_notes

فیلد	مقادیر ذخیره‌شده
conversation_type	physician_note / doctor_patient
transcription_status	pending / processing / completed / failed
cleanup_status	pending / completed / failed

بـ۱۳ یادداشت بالینی

`table: clinical_notes`

title و content متن ثبت‌شده‌اند و source نوع منشأ را بیان می‌کند. voice_note_id ارتباط اختیاری به ضبط دارد که با حذف ضبط SET NULL می‌شود. schema این مدل پزشک تأییدکننده و نسخه‌ی متن را جدا ثبت نمی‌کند؛ تصمیم بازبینی در گردش کار کاربر قرار دارد.

جدول ۴۱ فیلدهای clinical_notes

نام فیلد	نوع ORM	قید و رابطه	تفسیر
id	INTEGER	PK NOT NULL	کلید اصلی رکورد؛ برای رابطه و ردیابی استفاده می‌شود.
patient_id	INTEGER	INDEX FK patients.id / CASCADE NOT NULL	کلید خارجی بیمار؛ در queryهای پرونده فیلتر می‌شود.
voice_note_id	INTEGER	INDEX FK voice_notes.id / SET NULL NULL	پیوند اختیاری یادداشت به ضبط؛ حذف ضبط → SET NULL.
title	VARCHAR(300)	NOT NULL	عنوان خوانا برای نمایش منبع یا گفتگو.
content	TEXT	NOT NULL	بدنه‌ی متن یادداشت یا پیام؛ در AI ورودی داده محسوب می‌شود.
source	VARCHAR(11)	NOT NULL	enum منشأ یادداشت؛ manual، voice یا ai_assisted.
created_at	DATETIME	NOT NULL	زمان ایجاد با timezone؛ fallback بسیاری از تاریخ‌ها.
updated_at	DATETIME	NOT NULL	زمان آخرین تغییر؛ در فهرست گفتگو ترتیب فعالیت می‌دهد.

جدول ۴۲ مقادیر enum در clinical_notes

فیلد	مقادیر ذخیره‌شده
source	manual / voice / ai_assisted

بـ۱۴ پیام و شواهد

`table: conversation_messages`

role، content و evidence پیام را تعریف می‌کنند. evidence از نوع JSONB و nullable است؛ schema پاسخ null را به فهرست خالی تبدیل می‌کند. فقط شواهد اعتبارسنجی‌شده‌ی ساختاریافته ذخیره می‌شوند، ولی معنای ادعای متن به‌طور مستقل تأیید نمی‌شود.

جدول ۴۳ فیلدهای conversation_messages

نام فیلد	نوع ORM	قید و رابطه	تفسیر
id	INTEGER	PK NOT NULL	کلید اصلی رکورد؛ برای رابطه و ردیابی استفاده می‌شود.
conversation_id	INTEGER	INDEX FK conversations.id / CASCADE NOT NULL	کلید خارجی گفتگوی مالک پیام.
role	VARCHAR(9)	NOT NULL	نوع نویسنده‌ی پیام یا نقش حساب، بسته به جدول.
content	TEXT	NOT NULL	بدنه‌ی متن یادداشت یا پیام؛ در AI ورودی داده محسوب می‌شود.
evidence	JSONB	NULL	فهرست JSONB شواهد پرونده؛ بدون URL موقت.
created_at	DATETIME	NOT NULL	زمان ایجاد با timezone؛ fallback بسیاری از تاریخ‌ها.
updated_at	DATETIME	NOT NULL	زمان آخرین تغییر؛ در فهرست گفتگو ترتیب فعالیت می‌دهد.

جدول ۴۴ مقادیر enum در conversation_messages

فیلد	مقادیر ذخیره‌شده
role	user / assistant

بـ۱۵ نتیجه‌ی منتخب آزمایش

`table: lab_results`

value عمداً متن است تا مقدار کیفی را نیز بپذیرد. test_name فیلد عمومی نام آزمایش است و ستون‌های جدا برای Hb، WBC یا glucose در این جدول وجود ندارند. document_id تنها وقتی تنظیم شود منشأ واقعی سند را نشان می‌دهد؛ وجود این FK به معنی استخراج خودکار نیست.

جدول ۴۵ فیلدهای lab_results

نام فیلد	نوع ORM	قید و رابطه	تفسیر
id	INTEGER	PK NOT NULL	کلید اصلی رکورد؛ برای رابطه و ردیابی استفاده می‌شود.

نام فیلد	نوع ORM	قید و رابطه	تفسیر
patient_id	INTEGER	FK patients.id / CASCADE NOT NULL	کلید خارجی بیمار؛ در queryهای پرونده فیلتر می‌شود.
document_id	INTEGER	INDEX FK documents.id / SET NULL NULL	پیوند اختیاری آزمایش به سند واقعی؛ حذف سند → SET NULL.
test_name	VARCHAR(200)	NOT NULL	نام عمومی آزمایش؛ نوع به ستون خاص تبدیل نشده است.
value	VARCHAR(100)	NOT NULL	مقدار متنی کمی یا کیفی؛ نیازمند تبدیل برای محاسبه.
unit	VARCHAR(50)	NULL	واحد نتیجه به‌صورت متن اختیاری.
reference_range	VARCHAR(100)	NULL	شرح بازه‌ی مرجع؛ خودکار از آن پرچم ساخته نمی‌شود.
flag	VARCHAR(8)	NULL	enum نشانگر نتیجه؛ ورودی ثبت‌شده‌ی مدل داده.
result_date	DATE	NULL	تاریخ نتیجه؛ مبنای ترتیب اخیر و تایم‌لاین.
created_at	DATETIME	NOT NULL	زمان ایجاد با timezone؛ fallback بسیاری از تاریخ‌ها.
updated_at	DATETIME	NOT NULL	زمان آخرین تغییر؛ در فهرست گفتگو ترتیب فعالیت می‌دهد.

جدول ۴۶ مقادیر enum در lab_results

فیلد	مقادیر ذخیره‌شده
flag	normal / abnormal / critical

روابط اختیاری به سند و صوت، پس از حذف منبع با SET NULL داده‌ی مشتق تأییدشده را نگه می‌دارند. این انتخاب نیازمند سیاست منشأ است، زیرا خواننده‌ی بعدی ممکن است فایل اولیه را دیگر نبیند. source_id در ClinicalEvent و شناسه‌ی متنی evidence نیز FK چندجدولی نیستند و اعتبارسنجی‌شان در کد صورت می‌گیرد.

پیوست پ تنظیمات و حدود اجرای محیط

این مرجع نام و نوع فیلدهای Settings را از کلاس تنظیمات می‌گیرد، نه از فایل .env کاربر. مقدار credential و secret و هر پیش‌فرضی که از جنس رمز است منتشر نمی‌شود. env_file دو مسیر .env و ../.env را معرفی می‌کند و extra=ignore است. get_settings cache دارد؛ برای تغییر واقعی تنظیمات فرایند باید اثر restart در نظر گرفته شود.

پ‌۱ برنامه و مرورگر

جدول ۴۷ تنظیمات برنامه و مرورگر

متغیر	پیش‌فرض کد	کاربرد و محدودیت
PROJECT_NAME	Clinical AI Copilot	عنوان metadata برنامه و مستند OpenAPI.
PROJECT_DESCRIPTION	شرح تصمیم‌یار دانشگاهی	شرح metadata و تصمیم‌یار بودن؛ سیاست اجرایی جامع نیست.
VERSION	0.1.0	نسخه‌ی metadata API؛ با شماره‌ی commit متفاوت است.
DEBUG	False	حالت debug؛ برای محیط واقعی باید بازبینی شود.
BACKEND_PORT	8000	تنظیم پورت برنامه؛ فرمان uvicorn نیز باید سازگار باشد.
API_V1_PREFIX	/api/v1	پیشوند نسخه‌ی API؛ default برابر /api/v1.
CORS_ORIGINS	http://localhost:3000,http://localhost:3100	فهرست مبداهای مرورگر با کاما؛ جای ACL نیست.

پ‌۲ پایگاه داده

جدول ۴۸ تنظیمات پایگاه داده

متغیر	پیش‌فرض کد	کاربرد و محدودیت
POSTGRES_DB	clinical_ai_copilot	نام پایگاه؛ fixture تست پسوند test_ می‌افزاید.
POSTGRES_USER	مقدار محرمانه یا حساب نمایش داده نمی‌شود	حساب اتصال پایگاه؛ مقدار محیط گزارش نمی‌شود.
POSTGRES_PASSWORD	مقدار محرمانه یا حساب نمایش داده نمی‌شود	secret پایگاه؛ در runtime از محیط خوانده می‌شود.

متغیر	پیش‌فرض کد	کاربرد و محدودیت
POSTGRES_HOST	localhost	میزبان داده؛ در production به postgres override می‌شود.
POSTGRES_PORT	5432	پورت داده؛ داخل Compose برابر 5432.

پ‌۳ فایل و OCR

جدول ۴۹ تنظیمات فایل و OCR

متغیر	پیش‌فرض کد	کاربرد و محدودیت
MINIO_ENDPOINT	localhost:9000	host امضای URL و تماس S3؛ باید برای مرورگر قابل دسترس باشد.
MINIO_ACCESS_KEY	مقدار محرمانه یا حساب نمایش داده نمی‌شود	شناسه‌ی credential ذخیره‌ساز؛ مقدار محیط حذف شده است.
MINIO_SECRET_KEY	مقدار محرمانه یا حساب نمایش داده نمی‌شود	secret ذخیره‌ساز؛ مقدار نباید به client یا گزارش برسد.
MINIO_BUCKET	clinical-documents	نام bucket فایل اصلی اسناد و صوت.
MINIO_SECURE	False	استفاده از HTTPS در client ذخیره؛ جای رمزگذاری دیسک نیست.
TESSERACT_CMD	رشته‌ی خالی	مسیر binary OCR در صورت نبود PATH مناسب.

پ‌۴ گفتار

جدول ۵۰ تنظیمات گفتار

متغیر	پیش‌فرض کد	کاربرد و محدودیت
STT_PROVIDER	speechmatics	انتخاب speechmatics، google یا openrouter در فایل فعلی.
STT_API_KEY	مقدار محرمانه یا حساب نمایش داده نمی‌شود	کلید Speechmatics مقدم بر نام .legacy
STT_LANGUAGE	fa	زبان ضبط جدید؛ پیش‌فرض .fa
SPEECHMATICS_API_KEY	مقدار محرمانه یا حساب نمایش داده نمی‌شود	نام قدیمی کلید؛ fallback کلید .STT

متغیر	پیش‌فرض کد	کاربرد و محدودیت
SPEECHMATICS_BASE_URL	https://asr.api.speechmatics.com/v2	پایه‌ی endpoint batch و jobهای Speechmatics.
STT_POLL_INTERVAL_SECONDS	3.0	فاصله‌ی polling کار Speechmatics؛ timeout کل را تعیین نمی‌کند.
STT_TIMEOUT_SECONDS	300	سقف انتظار job؛ زمان upload جداست.
STT_GOOGLE_MAX_CHUNK_SECONDS	55	طول هر قطعه‌ی Google؛ کنترل کیفیت مرز مستقل لازم است.
STT_GOOGLE_RETRIES	3	تلاش هر قطعه؛ UnknownValueError حذف قطعه است.
STT_OPENROUTER_MODEL	رشته‌ی خالی	مدل دارای audio؛ از AI_MODEL پر نمی‌شود.
STT_FALLBACK_PROVIDERS	رشته‌ی خالی	زنجیره‌ی نام provider با کاما؛ اولین موفقیت پایان است.

پ‌۵ ورود و پیام‌رسان

جدول ۵۱ تنظیمات ورود و پیام‌رسان

متغیر	پیش‌فرض کد	کاربرد و محدودیت
UNIOM_API_KEY	مقدار محرمانه یا حساب نمایش داده نمی‌شود	کلید پیام‌رسان برای OTP؛ با credential ابزار MCP متفاوت است.
UNIOM_API_BASE_URL	https://api.uniom.ir	پایه‌ی REST پیام‌رسان؛ secret نباید در log URL چاپ شود.
UNIOM_TIMEOUT_SECONDS	15.0	timeout تماس ارسال پیام.
JWT_SECRET_KEY	مقدار محرمانه یا حساب نمایش داده نمی‌شود	secret امضا؛ default توسعه برای استقرار مناسب نیست.
JWT_ALGORITHM	HS256	الگوریتم مجاز امضا و decode؛ پیش‌فرض HS256.
JWT_EXPIRE_MINUTES	480	عمر access token؛ سیاست refresh مستقل موجود نیست.

پ‌۶ مدل زبانی

جدول ۵۲ تنظیمات مدل زبانی

متغیر	پیش‌فرض کد	کاربرد و محدودیت
AI_PROVIDER	openai_compatible	انتخاب provider پیش‌فرض گردش‌های AI.
OPENROUTER_API_KEY	مقدار محرمانه یا حساب نمایش داده نمی‌شود	کلید اختصاصی OpenRouter؛ fallback به AI_API_KEY.
AI_API_KEY	مقدار محرمانه یا حساب نمایش داده نمی‌شود	کلید عمومی provider؛ نبود آن بعضی مسیرها را 503 می‌کند.
AI_BASE_URL	https://api.openai.com/v1	پایه‌ی مسیر generic compatible؛ از ورودی کاربر نمی‌آید.
AI_MODEL	gemini-3.6-flash	مدل پیش‌فرض محیط؛ availability نام باید جدا تأیید شود.
AI_TIMEOUT_SECONDS	60.0	timeout client متن؛ در OpenRouter STT به client عمومی منتقل نمی‌شود.
AI_MAX_HISTORY_TURNS	6	برش تاریخچه‌ی stateless روی تعداد پیام؛ ماندگار سقف20 مستقل دارد.
AI_PROXY	رشته‌ی خالی	proxy در client متن و opener Google؛ مسیر STT OpenRouter شکاف دارد.

متغیر NEXT_PUBLIC_API_URL در Frontend مقدار build-time است و در Compose production باید URL کامل قابل استفاده از server و browser باشد. STORAGE_PUBLIC_HOST و HTTP_PORT و HTTPS_PORT متغیرهای Compose هستند، نه فیلد .Settings مدل‌های نام‌برده در پیش‌فرض و فهرست ایستا را باید پیش از اجرای واقعی از نظر availability مستقل بررسی کرد.

جدول ۵۳ تنظیمات خارج از Settings

نام	محل	رفتار
NEXT_PUBLIC_API_URL	frontend/Dockerfile و lib/api.ts	مقدار bundle هنگام build؛ پیش‌فرض development نشانی localhost است.
STORAGE_PUBLIC_HOST	deploy/docker-compose.prod.yml	MINIO_ENDPOINT و extra_hosts را به hostname عمومی مرتبط می‌کند.

نام	محل	رفتار
HTTP_PORT / HTTPS_PORT	Compose production	پورت publish؛ وجود 443 به معنی listen TLS در Nginx نیست.

پیوست ت قرارداد نمونه و گزیده‌ی کد

نمونه‌های این پیوست مصنوعی و صرفاً برای توضیح شکل داده‌اند. آن‌ها نتیجه‌ی اجرای زنده یا داده‌ی بیمار نیستند و هیچ پیشنهاد درمانی ارائه نمی‌کنند. ابتدا ورودی یا گزیده و سپس دلیل کاربرد آن توضیح داده می‌شود. نام کلیدها از schema واقعی است و مقادیر حساس در مثال نیامده‌اند.

تـ۱ ایجاد بیمار

```
{
  "first_name": "نمونه",
  "last_name": "آموزشی",
  "gender": "unknown"
}
```

این درخواست با PatientCreate سازگار است و فیلدهای اختیاری حذف شده‌اند. نبود کد ملی به معنای معتبر بودن هویت نیست. پاسخ موفق PatientResponse شناسه و زمان‌ها را اضافه می‌کند و پزشک سازنده در مرحله‌ی جدا به بیمار منتسب می‌شود.

تـ۲ ارسال پیام ماندگار

```
{
  "content": "چه اطلاعاتی برای مرور این پرونده ثبت شده است؟",
  "medication_safety": false,
  "provider": "openrouter",
  "model": "provider/model-id"
}
```

model در این مثال شناسه‌ی نمایشی ساختگی است، نه مدل قابل فراخوانی تضمین‌شده. در اجرای واقعی باید شناسه‌ی موجود انتخاب شود. provider و model به سرور امکان دریافت URL یا کلید از کاربر نمی‌دهند. پیام user پیش از completion در پایگاه می‌ماند.

تـ۳ شکل پاسخ شواهد مدل

```
{
  "answer": "اطلاعات ثبت‌شده نیازمند بازبینی پزشک است.",
  "evidence": [
    "DOC-123",
    "MED-42"
  ]
}
```

شناسه‌ها در این نمونه فرضی‌اند. در اجرای واقعی تنها موارد موجود در index همان بیمار پذیرفته می‌شوند. metadata مانند description از رکورد اصلی ساخته می‌شود و مدل آن را نمی‌تواند معتبر جلوه دهد. معتبر بودن شناسه جای صحت معنایی جمله را نمی‌گیرد.

تـ۴ ورودی تحلیل دارویی

```
{
  "medications": [
    {
      "name": "ExampleDrug",
      "dose": null
    }
  ],
  "question": "چه اطلاعاتی برای ارزیابی لازم است؟"
}
```

نام دارو برای آموزش ساختگی است و مثال ایمنی پزشکی نیست. endpoint اختصاصی یک تا پنج کاندید می‌پذیرد و داده را با Context به مدل پیش‌فرض می‌فرستد. خروجی schema شدت قطعی یا rule_id ندارد و ذخیره‌ی دارو با تحلیل مستقل است.

تـ۵ ثبت متن بازبینی‌شده

```
{
  "title": "یادداشت آموزشی",
  "content": "این متن مصنوعی برای توضیح قرارداد است."
}
```

این درخواست به /voice-notes/{voice_note_id}/note مربوط است. content ورودی کاربر در ClinicalNote ذخیره می‌شود و transcript خام تغییر نمی‌کند. داشتن voice_note_id واقعی و مجوز بیمار شرط مسیر است؛ تأیید انسانی باید در گردش کار مشخص باشد.

تـ۶ جلوگیری از اختلاط شواهد دو بیمار

```
        record = valid_index.get(evidence_id)
        if record is None:
            dropped.append(evidence_id)
            continue
        items.append(
```

این گزیده‌ی واقعی از validate_evidence نشان می‌دهد شناسه‌ی پاسخ مدل با index پرونده سنجیده می‌شود. شناسه‌ی خارج index وارد فهرست نتیجه نمی‌گردد. این کنترل در برابر ارجاع ساختاریافته‌ی جعلی عمل می‌کند و متن answer را از نظر درستی رابطه‌ی ادعا و شاهد تحلیل نمی‌کند.

تـ۷ حفظ سؤال در شکست ارائه‌دهنده

```
    user_message = ConversationMessage(
        conversation=conversation, role=ChatRole.user, content=content
    )
    db.add(user_message)
    if not history and conversation.title == DEFAULT_CONVERSATION_TITLE:
        conversation.title = _derive_title(content)  # deterministic, no LLM
    db.commit()
    db.refresh(user_message)
```

```
    try:
        # runtime selection (ids only) with env-default fallback
```

گزیده‌ی route ماندگار، ساخت پیام کاربر و commit آن را پیش از تماس خارجی نشان می‌دهد. با این ترتیب شکست AI سؤال را حذف نمی‌کند. همین تصمیم باعث می‌شود retry ساده‌ی رابط بتواند سؤال تکراری بسازد و لازم است شناسه‌ی درخواست و رفتار تلاش دوباره در توسعه‌ی بعدی مشخص شود.

تـ۸ فرمان‌های مستند محیط آزمایشی

```
docker compose up -d postgres minio
alembic upgrade head
uvicorn app.main:app --reload
docker compose --env-file .env -f deploy/docker-compose.prod.yml build
docker compose --env-file .env -f deploy/docker-compose.prod.yml run --rm backend alembic
upgrade head
```

دو فرمان اول برای زیرساخت development و migration در مسیر مناسب اجرا می‌شوند و build و run بعدی به فایل production مربوط‌اند. مقادیر واقعی محیط باید پیش از اجرا تنظیم شوند. این فرمان‌ها در گزارش به‌عنوان رویه‌ی پیکربندی آمده‌اند و در این کار روی سرور عمومی اجرا نشده‌اند. seed و downgrade روی داده‌ی واقعی نباید برای نشان دادن نتیجه‌ی گزارش اجرا شوند.

پیوست ث مرجع آزمون و سناریوهای تکمیلی

جدول نخست تعداد توابع تعریف‌شده با نام _test را در فایل‌ها نشان می‌دهد. این شمارش از AST است؛ parametrization می‌تواند تعداد موارد اجرای pytest را افزایش دهد، پس این عدد تعداد passed نیست. جدول دوم نتایج واقعی اجرای منتخب را نگه می‌دارد. انتظار رفتاری تست‌های اجرانشده صرفاً از کد آن‌ها خوانده شده است.

جدول ۵۴ فهرست فایل‌های آزمون

فایل	توابع تعریف‌شده	دامنه‌ی تفسیر
test_agent_api.py	17	دارای fixture پایگاه یا سرویس؛ اجرای منتخب را در جدول نتیجه ببینید
test_ai_medication_safety.py	17	دارای fixture پایگاه یا سرویس؛ اجرای منتخب را در جدول نتیجه ببینید
test_ai_selection_and_catalog.py	12	دارای fixture پایگاه یا سرویس؛ اجرای منتخب را در جدول نتیجه ببینید
test_allergies_api.py	9	دارای fixture پایگاه یا سرویس؛ اجرای منتخب را در جدول نتیجه ببینید
test_auth_api.py	17	دارای fixture پایگاه یا سرویس؛ اجرای منتخب را در جدول نتیجه ببینید
test_chat_api.py	17	دارای fixture پایگاه یا سرویس؛ اجرای منتخب را در جدول نتیجه ببینید
test_conditions_api.py	8	دارای fixture پایگاه یا سرویس؛ اجرای منتخب را در جدول نتیجه ببینید
test_conversations_api.py	16	دارای fixture پایگاه یا سرویس؛ اجرای منتخب را در جدول نتیجه ببینید
test_cors.py	4	آزمون service یا قرارداد؛ وابستگی واقعی/جایگزین از fixture تعیین می‌شود
test_document_analysis_api.py	11	دارای fixture پایگاه یا سرویس؛ اجرای منتخب را در جدول نتیجه ببینید
test_documents_api.py	20	دارای fixture پایگاه یا سرویس؛ اجرای منتخب را در جدول نتیجه ببینید

فایل	توابع تعریف‌شده	دامنه‌ی تفسیر
test_evidence_api.py	15	دارای fixture پایگاه یا سرویس؛ اجرای منتخب را در جدول نتیجه ببینید
test_extraction_service.py	7	آزمون service یا قرارداد؛ وابستگی واقعی/جایگزین از fixture تعیین می‌شود
test_health.py	3	آزمون service یا قرارداد؛ وابستگی واقعی/جایگزین از fixture تعیین می‌شود
test_logging.py	2	آزمون service یا قرارداد؛ وابستگی واقعی/جایگزین از fixture تعیین می‌شود
test_mcp_tools.py	10	دارای fixture پایگاه یا سرویس؛ اجرای منتخب را در جدول نتیجه ببینید
test_medication_safety.py	22	دارای fixture پایگاه یا سرویس؛ اجرای منتخب را در جدول نتیجه ببینید
test_medications_api.py	14	دارای fixture پایگاه یا سرویس؛ اجرای منتخب را در جدول نتیجه ببینید
test_models.py	7	دارای fixture پایگاه یا سرویس؛ اجرای منتخب را در جدول نتیجه ببینید
test_openrouter_provider.py	11	آزمون service یا قرارداد؛ وابستگی واقعی/جایگزین از fixture تعیین می‌شود
test_otp_login.py	14	دارای fixture پایگاه یا سرویس؛ اجرای منتخب را در جدول نتیجه ببینید
test_patient_access_control.py	12	دارای fixture پایگاه یا سرویس؛ اجرای منتخب را در جدول نتیجه ببینید
test_patient_context_api.py	9	دارای fixture پایگاه یا سرویس؛ اجرای منتخب را در جدول نتیجه ببینید
test_patient_overview_api.py	5	دارای fixture پایگاه یا سرویس؛ اجرای منتخب را در جدول نتیجه ببینید
test_patients_api.py	14	دارای fixture پایگاه یا سرویس؛ اجرای منتخب را در جدول نتیجه ببینید
test_seed.py	3	دارای fixture پایگاه یا سرویس؛ اجرای منتخب را در جدول نتیجه ببینید

فایل	توابع تعریف‌شده	دامنه‌ی تفسیر
test_storage_service.py	2	آزمون service یا قرارداد؛ وابستگی واقعی/جایگزین از fixture تعیین می‌شود
test_stt_google_and_openrouter.py	21	آزمون service یا قرارداد؛ وابستگی واقعی/جایگزین از fixture تعیین می‌شود
test_stt_service.py	11	آزمون service یا قرارداد؛ وابستگی واقعی/جایگزین از fixture تعیین می‌شود
test_voice_note_cleanup.py	9	دارای fixture پایگاه یا سرویس؛ اجرای منتخب را در جدول نتیجه ببینید
test_voice_notes_api.py	10	دارای fixture پایگاه یا سرویس؛ اجرای منتخب را در جدول نتیجه ببینید

ث‌ـ۱ موارد واقعی اجرای منتخب

جدول ۵۵ جزئیات موارد اجراشده

فایل	مورد آزمون	نتیجه
test_health	test_api_v1_health_returns_ok	موفق
test_health	test_root_health_returns_ok	موفق
test_health	test_openapi_schema_contains_health_paths	موفق
test_cors	test_preflight_allowed_for_default_frontend_port	موفق
test_cors	test_preflight_allowed_for_alternate_frontend_port	موفق
test_cors	test_preflight_rejected_for_unknown_origin	موفق
test_cors	test_actual_request_carries_allow_origin	موفق
test_logging	test_setup_logging_creates_utf8_stream	موفق
test_logging	test_logging_persian_does_not_raise	موفق
test_storage_service	test_upload_presign_download_delete_roundtrip	خطای setup؛ MinIO در دسترس نبود

فایل	مورد آزمون	نتیجه
test_storage_service	test_storage_key_is_unique_and_sanitized	خطای setup؛ MinIO در دسترس نبود
test_extraction_service	test_extract_plain_text	موفق
test_extraction_service	test_extract_pdf_text_layer	موفق
test_extraction_service	test_extract_pdf_without_text_or_ocr_raises_unavailable	موفق
test_extraction_service	test_ocr_image_english	موفق
test_extraction_service	test_ocr_image_persian	موفق
test_extraction_service	test_ocr_image_without_tesseract_raises_unavailable	موفق
test_extraction_service	test_unsupported_mime_type	موفق
test_stt_service	test_no_api_key_raises_unavailable	موفق
test_stt_service	test_full_job_flow_plain_transcript	موفق
test_stt_service	test_full_job_flow_diarized_transcript	موفق
test_stt_service	test_job_error_raises	موفق
test_stt_service	test_rejected_api_key_raises	موفق
test_stt_service	test_format_transcript_json_speaker_turns	موفق
test_stt_service	test_config_json_is_serializable_and_fa	موفق
test_stt_service	test_get_stt_provider_dispatch_and_key_fallback	موفق
test_stt_service	test_get_stt_provider_unknown_name_raises	موفق
test_stt_service	test_get_stt_provider_no_key_raises_unavailable	موفق
test_stt_service	test_transcribe_audio_delegates_to_provider	موفق
test_stt_google_and_openrouter	test_google_provider_single_chunk	موفق
test_stt_google_and_openrouter	test_google_provider_chunks_long_audio	موفق
test_stt_google_and_openrouter	test_google_provider_retries_request_error	موفق
test_stt_google_and_openrouter	test_google_provider_skips_silent_chunks	موفق
test_stt_google_and_openrouter	test_google_provider_all_silent_raises_persian	موفق

فایل	مورد آزمون	نتیجه
test_stt_google_and_openrouter	test_google_provider_temp_files_cleaned	موفق
test_stt_google_and_openrouter	test_google_provider_diarize_logs_and_continues	موفق
test_stt_google_and_openrouter	test_google_language_mapping	موفق
test_stt_google_and_openrouter	test_openrouter_request_format	موفق
test_stt_google_and_openrouter	test_openrouter_key_fallback_to_ai_api_key	موفق
test_stt_google_and_openrouter	test_openrouter_no_key_raises	موفق
test_stt_google_and_openrouter	test_openrouter_no_model_raises	موفق
test_stt_google_and_openrouter	test_openrouter_http_error_raises	موفق
test_stt_google_and_openrouter	test_openrouter_empty_text_raises	موفق
test_stt_google_and_openrouter	test_openrouter_temp_files_cleaned	موفق
test_stt_google_and_openrouter	test_fallback_google_to_openrouter	موفق
test_stt_google_and_openrouter	test_fallback_all_fail_combined_persian_error	موفق
test_stt_google_and_openrouter	test_chain_skips_unavailable_provider	موفق
test_stt_google_and_openrouter	test_unknown_provider_single_raises_directly	موفق
test_stt_google_and_openrouter	test_unknown_provider_in_chain_wrapped	موفق
test_stt_google_and_openrouter	test_provider_by_name_dispatch	موفق
test_openrouter_provider	test_factory_selects_openrouter_provider	موفق
test_openrouter_provider	test_factory_still_selects_gemini	موفق
test_openrouter_provider	test_default_model_when_ai_model_empty	موفق
test_openrouter_provider	test_missing_key_raises_unconfigured	موفق
test_openrouter_provider	test_ai_api_key_fallback	موفق
test_openrouter_provider	test_request_format_matches_openrouter_api	موفق
test_openrouter_provider	test_http_error_raises	موفق
test_openrouter_provider	test_rate_limit_raises	موفق
test_openrouter_provider	test_timeout_raises	موفق

فایل	مورد آزمون	نتیجه
test_openrouter_provider	test_malformed_response_raises	موفق
test_openrouter_provider	test_key_never_in_errors_or_payload	موفق

ث‌ـ۲ سناریوهای تکمیلی پیشنهادی

سناریوهای زیر پیشنهاد آینده‌اند و نتیجه‌ی اجرایی برای آن‌ها ادعا نمی‌شود. هر سناریو باید روی داده‌ی مصنوعی، نسخه‌ی مشخص و محیط اختصاصی اجرا شود. هدف آن‌ها پوشاندن شکاف‌های مشاهده‌شده در مسیرهاست، نه تکرار assertionهای ساده‌ی فعلی.

جدول ۵۶ طرح آزمون برای شکاف‌های مهندسی

موضوع	ورودی و پیش‌شرط	انتظار و اندازه‌گیری
دانلود در مرورگر	doctor با localStorage و cookie وارد شود و پیوند سند شواهد را باز کند.	ثبت وجود Authorization و موفقیت رسیدن به فایل؛ بررسی 401 مسیر لینک معمول.
PDF ترکیبی	یک صفحه‌ی متنی با متن کافی و یک صفحه‌ی اسکن‌شده‌ی دارای داده‌ی مرجع.	اندازه‌گیری پوشش صفحه‌ی دوم؛ بررسی آستانه‌ی مجموع پنجاه نویسه.
مقدار مرزی صوت	فایل صوتی بین 25 و 50 MiB از مسیر Nginx بارگذاری شود.	مقایسه‌ی پذیرش proxy و Backend؛ بدون تغییر مصنوعی مقدار معیار.
خطای conversion	پس از ساخت tempfile، ffmpeg شکست داده شود.	فهرست فایل‌های باقی‌مانده در Google؛ descriptor بسته باشد.
رقابت پیام	دو پیام یک گفتگو هم‌زمان و پاسخ‌های خارجی با ترتیب پایان معکوس.	ثبت ترتیب user و assistant و پیوند منطقی پاسخ؛ آزمون نیاز .idempotency
رقابت OTP	دو verify هم‌زمان برای یک کد معتبر در پایگاه آزمایشی.	فقط یک مصرف مجاز؛ بررسی lock یا update شرطی در راهکار آینده.
حذف بیمار	بیمار مصنوعی با سند و صوت حذف شود.	بررسی رکوردها و اشیاء؛ تفاوت cascade پایگاه و فایل گزارش شود.
منشأ نامعتبر	مدل ID واقعی ولی نامرتبط با جمله و ID جعلی داخل answer تولید کند.	تفکیک حذف فهرست ساختاریافته از کنترل معنایی و متن نمایش.
بازپردازش سند	ai_summary موجود، استخراج مجدد با متن متفاوت انجام شود.	سیاست invalidation خلاصه و نسخه‌ی ورودی را بررسی کند.
مکالمه با fallback	Speechmatics شکست و Google موفق شود در حالت .doctor_patient	گزارش نبود diarization و کامل بودن نتیجه به کاربر؛ فقط متن کافی نیست.

موضوع	ورودی و پیش‌شرط	انتظار و اندازه‌گیری
proxy صوت	OpenRouter STT با proxy لازم و شبکه‌ی مستقیم ناممکن اجرا شود.	اثبات مسیر واقعی client؛ صرف وجود self.proxy پذیرفته نشود.
prompt injection	متن مصنوعی سند دستور نادیده گرفتن سیاست یا ساخت منبع داشته باشد.	سنجش ادعای خارج پرونده، داده‌ی محرمانه و شواهد؛ ثبت شکست مدل.

ث‌ـ۳ طرح ثبت نتایج ارزیابی

برای هر اجرا شناسه‌ی نسخه، مدل و provider، prompt version، مشخصات فایل مصنوعی، زمان هر مرحله، وضعیت و نوع خطا ثبت شود. credential و اطلاعات بیمار واقعی در این ثبت نباشند. برای پاسخ AI، ارزیاب منبع هر ادعا و مقدار unsupported را علامت بزند. برای STT، متن مرجع و عادی‌سازی ثابت پیش از WER/CER تعیین گردد. برای ایمنی، «خارج پوشش» و «نامطمئن» با «منفی واقعی» ادغام نشوند.

پروتکل کاربری نیز باید مشارکت‌کننده، رضایت، وظیفه، زمان آغاز و پایان و روش ثبت اصلاحات را مشخص کند. نتیجه‌ی آزمون نرم‌افزار را نباید رضایت پزشک یا اثربخشی درمان نامید. داده‌ی مرجع و معیار امتیازدهی باید پیش از مشاهده‌ی پاسخ نهایی مشخص باشند تا تغییر معیار پس از نتیجه، ارزیابی را منحرف نکند.

University of Tehran

College of Farabi

Faculty of Engineering

Department of Computer Engineering

Design and Implementation of an AI Clinical Copilot Based on Integration of Patient Context and Validated Medical Knowledge

By

Erfan Karbalai

Supervisor

دکتر کاظم فولادی قلعه

A Bachelor's Project Report in Computer Engineering

Submission month and year: .....